

**PENGEMBANGAN *FRAMEWORK* SISTEM DETEKSI
PELANGGARAN 9-BALL POOL MENGGUNAKAN YOLO
DAN *RULE-BASED REASONING* PADA RASPBERRY PI**

***DEVELOPMENT OF 9-BALL POOL FOUL DETECTION
FRAMEWORK USING YOLO AND RULE-BASED
REASONING ON RASPBERRY PI***

**TUGAS AKHIR
UNTUK MEMENUHI SEBAGIAN DARI SYARAT-SYARAT
GUNA MENCAPAI GELAR SARJANA INFORMATIKA**



Diajukan Oleh:

David Tulus Halomoan Haryanto
NIM: 2210101027

**PROGRAM STUDI INFORMATIKA
UNIVERSITAS PRADITA
JAKARTA
2026**

LEMBAR PERSETUJUAN TUGAS AKHIR

Nama : David Tulus Halomoan Haryanto

NIM : 2210101027

Program Studi : Informatika

Bentuk Tugas Akhir : Skripsi

Peminatan Tugas Akhir : Artificial Intelligence

Judul Tugas Akhir : PENGEMBANGAN FRAMEWORK SISTEM DETEKSI
PELANGGARAN 9-BALL POOL MENGGUNAKAN YOLO
DAN RULE-BASED REASONING PADA RASPBERRY PI

Kab. Tangerang, 23 Juni 2026

Menyetujui

Pembimbing Skripsi

A handwritten signature in black ink, appearing to read 'Handri', with a stylized flourish underneath.

Dr. Eng. Handri Santoso, S. Si., M.Eng.

LEMBAR PERNYATAAN TIDAK PLAGIAT

Yang bertanda tangan dibawah ini:

Nama : David Tulus Halomoan Haryanto

NIM : 2210101027

Program Studi : Informatika

Judul Tugas Akhir : PENGEMBANGAN FRAMEWORK SISTEM DETEKSI
PELANGGARAN 9-BALL POOL MENGGUNAKAN YOLO DAN RULE-BASED
REASONING PADA RASPBERRY PI

Kab. Tangerang, 25 Juni 2026



David Tulus Halomoan Haryanto

2210101027

LEMBAR PENGESAHAN TUGAS AKHIR

Nama: : David Tulus Halomoan Haryanto
NIM : 2210101027
Program Studi : Informatika
Bentuk Tugas Akhir : Skripsi
Peminatan Tugas Akhir : Artificial Intelligence
Judul Tugas Akhir : PENGEMBANGAN FRAMEWORK SISTEM DETEKSI
PELANGGARAN 9-BALL POOL MENGGUNAKAN YOLO DAN RULE-BASED
REASONING PADA RASPBERRY PI

Mengajukan pada tanggal Kamis, 25 Juni 2026

Telah disetujui oleh:

Pembimbing

Penguji

Dr. Eng. Handri Santoso, S.Si., M.Eng

(Nama Penguji)

Ketua Sidang,

(Nama Ketua Sidang)

Disahkan oleh:

Kepala Program Studi Informatika

Erick Dazki, S.Kom., M.Kom.

LEMBAR PERNYATAAN PERSETUJUAN PUBLIKASI UNTUK KEPENTINGAN
AKADEMIS

Dengan ini saya sebagai civitas akademik Universitas Pradita yang bertandatangan di bawah ini

Nama : David Tulus Halomoan Haryanto

NIM : 2210101027

Program Studi : Informatika

Bentuk Tugas Akhir : Skripsi

untuk meningkatkan pengembangan ilmu pengetahuan, memberikan skripsi/tugas akhir kepada Universitas Pradita Hak Bebas Royalti Non-eksklusif (*Non-exclusive Royalty Free Right*) dengan judul:

**PENGEMBANGAN FRAMEWORK SISTEM DETEKSI PELANGGARAN 9-BALL
POOL MENGGUNAKAN YOLO DAN RULE-BASED REASONING PADA
RASPBERRY PI**

Beserta dokumen tugas akhir yang ada sesuai ketentuan yang berlaku. Dengan Hak Bebas Royalti Non-eksklusif (*Non-exclusive Royalty Free Right*) ini, maka Universitas Pradita berhak menyimpan dan mengelola dalam bentuk database, dan mempublikasikan tugas akhir ini dengan tetap mencantumkan nama saya sebagai penulis tugas akhir ini sebagai penulis/pencipta dan pemilik hak cipta

Demikianlah pernyataan ini saya buat dengan sebenarnya,

Kab. Tangerang, 25 Juni 2026



David Tulus Halomoan

Haryanto

2210101027

KATA PENGANTAR

Puji dan syukur peneliti panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunianya sehingga peneliti dapat menyelesaikan tugas akhir ini dengan baik. Tugas akhir dengan judul **“PENGEMBANGAN FRAMEWORK SISTEM DETEKSI PELANGGARAN 9-BALL POOL MENGGUNAKAN YOLO DAN RULE-BASED REASONING PADA RASPBERRY PI”** disusun sebagai salah satu syarat untuk penyusunan tugas akhir dalam rangka menyelesaikan Program Studi Informatika Jenjang Pendidikan Strata-1 (S1) Universitas Pradita.

Penyusunan laporan tugas akhir ini tidak terlepas dari bantuan, bimbingan, dan dukungan dari berbagai pihak. Oleh karena itu, penulis ingin mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Prof. Dr. Ir. Richardus Eko Indrajit, DBA. Dr(Pend), Dr(Han), M.Sc ,M.B.A, M.Si, MA ,M.I.T, M.Phil selaku Rektor Universitas Pradita.
2. Bapak Hendro Susanto, selaku wakil Rektor 1 Bidang Akademik Universitas Pradita.
3. Dr. Eng., Handri Santoso, S.Si., M.Eng. selaku Dekan Fakultas Sains dan Teknologi Universitas Pradita. Selaku dosen pembimbing yang selalu memotivasi, membimbing peneliti dari tahap awal penyusunan hingga skripsi ini selesai, serta memberikan arahan dan klarifikasi yang diperlukan selama proses penulisan.
4. Bapak Erick Dazki, S.Kom, M.Kom. selaku Ketua Program Studi Informatika Universitas Pradita.
5. Orang tua yang membantu menasehati peneliti dan memberikan *resource* yang dibutuhkan untuk dapat mengerjakan tugas akhir ini.
6. Keluarga yang tanpa lelah memberi semangat pada peneliti dan bantu memberikan *input* dan kebutuhan.
7. Teman-teman mahasiswa seperjuangan peneliti yang selalu memberikan dukungan dan menemani peneliti sepanjang berjalannya tugas akhir ini.

Peneliti menyadari bahwa laporan tugas akhir ini masih memiliki kekurangan. Oleh karena itu, peneliti sangat mengharapkan kritik dan saran demi perbaikan dan penyempurnaan penelitian ini di masa mendatang. Semoga penelitian ini dapat bermanfaat bagi pemain biliar, pemilik lapangan biliar, dan bagi peneliti lainnya yang mencari ilmu di bidang visi komputer dan *object detection* berbasis YOLO.

Kab. Tangerang, 25 Juni 2026

A handwritten signature in brown ink, appearing to read 'David' followed by a stylized flourish.

David Tulus Halomoan

Haryanto

2210101027

ABSTRAK

Pelanggaran dalam permainan *9-ball pool* berpengaruh langsung terhadap keadilan permainan dan hasil pertandingan. Dalam praktiknya, pengamatan manual oleh wasit masih dapat dipengaruhi oleh pengalaman, sudut pandang, dan konsistensi pengamatan terhadap kejadian di atas meja. Perkembangan visi komputer dan *edge computing* memungkinkan sistem bantu dikembangkan untuk menganalisis objek permainan secara otomatis melalui kamera. Penelitian ini mengembangkan sistem deteksi pelanggaran *9-ball pool* berbasis visi komputer yang diimplementasikan pada Raspberry Pi 5 dengan akselerator Hailo-8L. Sistem menggunakan model YOLO untuk mendeteksi *cue ball*, bola bernomor, dan *pocket*, kemudian memanfaatkan hasil deteksi berupa kelas objek dan *bounding box* sebagai dasar logika deteksi pelanggaran berbasis koordinat. Jenis pelanggaran yang dideteksi dibatasi pada *cue ball scratch*, *no legal hit*, *cue ball off field*, dan *no rail contact on shot*. Dataset dibangun dari video publik pertandingan *9-ball* dan video pribadi menggunakan meja biliard sederhana. Tiga model YOLO varian *nano*, yaitu YOLOv8n, YOLOv10n, dan YOLO11n, dilatih dan dibandingkan. Hasil evaluasi menunjukkan bahwa YOLOv10n memiliki keseimbangan terbaik antara akurasi dan performa. Pada CPU Raspberry Pi 5, YOLOv10n memperoleh mAP@50 sebesar 0,976 dan mAP@50-95 sebesar 0,738. Setelah dijalankan pada Hailo-8L, model memperoleh mAP@50 sebesar 0,953 dengan performa *end-to-end* meningkat dari 4,06 FPS menjadi 51,41 FPS. Pengujian terhadap 50 video menghasilkan akurasi deteksi pelanggaran sebesar 84% pada dengan model Hailo, dan 92% dengan model YOLO *original*.

Kata kunci: *9-ball pool*, deteksi pelanggaran, visi komputer, YOLOv10n, Hailo-8L

ABSTRACT

Fouls in 9-ball pool directly affect the fairness of gameplay and match outcomes. In practice, manual observation by referees can still be influenced by experience, viewing angle, and consistency in observing events on the table. The development of computer vision and edge computing enables an assistive system to analyze game objects automatically through a camera. This study develops a computer vision-based foul detection system for 9-ball pool implemented on a Raspberry Pi 5 with a Hailo-8L accelerator. The system uses YOLO models to detect the cue ball, numbered balls, and pockets, then utilizes object classes and bounding boxes as the basis for coordinate-based foul detection logic. The detected foul types are limited to cue ball scratch, no legal hit, cue ball off field, and no rail contact on shot. The dataset was built from public 9-ball match videos and privately recorded videos using a simple billiard table setup. Three nano YOLO variants, namely YOLOv8n, YOLOv10n, and YOLO11n, were trained and compared. The evaluation results show that YOLOv10n provides the best balance between accuracy and performance. On the Raspberry Pi 5 CPU, YOLOv10n achieved an mAP@50 of 0.976 and an mAP@50-95 of 0.738. When executed on Hailo-8L, the model achieved an mAP@50 of 0.953, while end-to-end performance increased from 4.06 FPS to 51.41 FPS. Testing on 50 videos produced an overall foul detection accuracy of 84% on the Hailo model, and 92% on the original YOLO model.

Keywords: 9-ball pool, foul detection, computer vision, YOLOv10n, Hailo-8L.

DAFTAR ISI

DAFTAR TABEL	xii
DAFTAR GAMBAR.....	xiii
DAFTAR LAMPIRAN.....	xiv
BAB I PENDAHULUAN.....	1
1.1. Latar Belakang.....	1
1.2. Rumusan Masalah	3
1.3. Tujuan Penelitian.....	3
1.4. Manfaat Penelitian.....	3
1.4.1. Subjektif.....	3
1.4.2. Objektif.....	4
1.5. Ruang Lingkup Pembahasan.....	4
1.6. Metode Penelitian.....	4
1.7. Sistematika Penulisan	5
BAB II TINJAUAN PUSTAKA	7
2.1. Kerangka Teoritis	7
2.1.1. Permainan 9-Ball Pool dan Pelanggaran	7
2.1.2. Deteksi Pelanggaran Berbasis Visi Komputer.....	9
2.1.3. Dataset dan Kebutuhan Pelatihan Model.....	9
2.1.4. Implementasi Pada Edge Device	10
2.2. Tinjauan Teoritis	10
2.2.1. Computer Vision.....	10
2.2.2. Image Classification, Image Segmentation, dan Object Detection	11
2.2.3. Object Detection dengan YOLO	12
2.2.4. Dataset Annotation dan Data Augmentation	12
2.2.5. Edge Computing dan AI Accelerator	13
2.2.6. Metrik Evaluasi Object Detection	15
2.3. Studi Preseden.....	15
BAB III METODOLOGI PENELITIAN	19
3.1. Gambaran Umum Objek Penelitian	19
3.1.1. Objek Penelitian	19
3.1.2. Perangkat dan Bahan Penelitian	19
3.1.3. Gambaran Umum Alur Penelitian	20

3.1.4.	Persiapan Data	21
3.1.5.	Pengolahan Dataset.....	22
3.1.6.	Pelatihan Model Deteksi Objek.....	23
3.1.7.	Pengembangan Logika Deteksi Pelanggaran	25
3.1.8.	Konversi Model ke Format Hailo Executable Format.....	29
3.1.9.	Implementasi dan Pengujian Model pada Raspberry Pi 5.....	30
3.2.	Metode Penelitian.....	30
3.2.1.	Metode Pengambilan Data.....	30
3.2.2.	Metode Pengolahan Data.....	31
3.2.3.	Metode Analisis Data	31
3.2.4.	Metode Evaluasi Data.....	32
BAB IV ANALISIS DAN PEMBAHASAN.....		34
4.1.	Hasil Pelatihan Model Deteksi Objek	34
4.2.	Evaluasi Deteksi Objek per Kelas	35
4.3.	Perbandingan Performa CPU Raspberry Pi 5 dan Hailo-8L	36
4.4.	Hasil Implementasi Sistem Deteksi Pelanggaran.....	38
4.5.	Hasil Pengujian Deteksi Pelanggaran	40
BAB V KESIMPULAN DAN SARAN.....		42
5.1.	Kesimpulan.....	42
5.2.	Saran	42
DAFTAR PUSTAKA.....		43
LAMPIRAN.....		46

DAFTAR TABEL

Tabel 1. 1. Pelanggaran yang akan diimplementasikan	2
Tabel 2. 1. Peraturan Pelanggaran 9-ball Berdasarkan BCA.....	7
Tabel 2. 2. Peraturan Pelanggaran Permainan Ball Berdasarkan BCA.	8
Tabel 2. 3. Pelanggaran dan indikator visual	8
Tabel 3. 1. Setiap kelas yang <i>ditraining</i>	22
Tabel 3. 2. Perbandingan kinerja model YOLO <i>nano</i>	24
Tabel 3. 3. Konfigurasi Pelatihan	24
Tabel 4. 1. Hasil evaluasi model deteksi objek pada CPU Raspberry Pi 5.....	34
Tabel 4. 2. Hasil evaluasi model deteksi objek pada Hailo-8L	34
Tabel 4. 3. Hasil evaluasi per kelas YOLOv10n pada CPU Raspberry Pi 5	35
Tabel 4. 4. Hasil evaluasi per kelas YOLOv10n pada Hailo-8L	36
Tabel 4. 5. Perbandingan performa CPU Raspberry Pi 5 dan Hailo-8L.....	37
Tabel 4. 6. Hasil pengujian deteksi pelanggaran menggunakan model .hef.....	40
Tabel 4. 7. Hasil pengujian deteksi menggunakan model .pt	40

DAFTAR GAMBAR

Gambar 2. 1. Komponen Studi Preseden	16
Gambar 3. 1 Alur Metodologi Penelitian.....	21
Gambar 3. 2 Top-down view meja biliar	25
Gambar 3. 3. Flowchart sistem deteksi pelanggaran 9-ball pool	26
Gambar 3. 4. Alur Konversi Model YOLO ke Format .hef.....	29
Gambar 4. 1. Tampilan hasil deteksi objek.....	38
Gambar 4. 2. Tampilan area pembantu pada sistem	39
Gambar 4. 3. Tampilan sistem saat mendeteksi pelanggaran	39

DAFTAR LAMPIRAN

LAMPIRAN 1 (Main_Camera_Code.py).....	46
LAMPIRAN 2 (Model_Testing.py)	81
LAMPIRAN 3 (Augmentation.ipynb).....	101
LAMPIRAN 4 (Training.ipynb).....	105
LAMPIRAN 5 (ONNX_Conversion.py)	106
LAMPIRAN 6 (Hailo Compilation Command)	106

BAB I

PENDAHULUAN

1.1. Latar Belakang

Olahraga merupakan bagian penting dari kehidupan manusia yang melibatkan aktivitas fisik, baik untuk kompetisi maupun rekreasi. Selain bermanfaat bagi kesehatan, olahraga juga menjadi sarana hiburan yang dilakukan banyak orang di seluruh dunia. Untuk menjaga keadilan dalam pertandingan, setiap cabang olahraga memiliki serangkaian aturan yang harus dipatuhi. Ketika seorang pemain melanggar aturan tersebut, hal itu disebut pelanggaran atau foul. Salah satu cabang olahraga yang membutuhkan konsentrasi, presisi dan strategi adalah biliar. Olahraga ini dimainkan di atas sebuah meja khusus dengan menggunakan stik untuk menyodok bola sesuai dengan peraturan bermain yang berlaku pada jenis permainan. Dalam berbagai cabang olahraga, *foul* atau pelanggaran merupakan bagian penting karena berpengaruh langsung terhadap keadilan permainan, perolehan poin, dan hasil akhir pertandingan. Dalam praktiknya, pelanggaran atau insiden penting dalam olahraga tidak selalu terdeteksi secara sempurna oleh wasit karena keterbatasan persepsi dan kondisi pertandingan. Pada olahraga seperti sepak bola, hal ini mendorong diperkenalkannya *Video Assistant Referee* (VAR), yaitu bantuan video untuk mengoreksi “*clear and obvious errors*” pada insiden yang berpotensi mengubah jalannya pertandingan. Studi Spitz et al. (2021) menganalisis 2195 pertandingan kompetitif dari 13 asosiasi sepak bola nasional dari periode 1 Januari 2017 hingga 1 Juni 2018, dan melaporkan bahwa akurasi keputusan awal wasit 92,1% meningkat menjadi 98,3% setelah intervensi VAR (Spitz et al., 2021).

Seiring dengan kemajuan teknologi, banyak peneliti telah mencoba menerapkan sistem otomatis untuk membantu tugas wasit dalam berbagai cabang olahraga. Upaya untuk menggunakan teknologi, contohnya kecerdasan buatan (Xu et al., 2021), dan khususnya visi komputer, untuk mendeteksi berbagai kejadian dalam permainan biliar juga bukanlah hal yang baru. Junqin Yu et al., mengusulkan sebuah sistem untuk mendeteksi kejadian dalam video permainan snooker (salah satu jenis permainan biliar) dengan pendekatan multimodal (Yu et al., 2018). Untuk mendeteksi *foul*, dilakukan analisis pada scoreboard yang tampil pada video permainan. Analisis yang dilakukan menggunakan *canny edge detection* untuk melihat tepian atau garis batas objek dalam sebuah gambar. Dengan informasi yang diekstrak dari papan skor, peneliti mendeteksi terjadinya pelanggaran dengan menganalisis perubahan data di antara dua rekaman informasi yang berdekatan. Seorang pemain dapat diidentifikasi ketika gilirannya untuk memukul berpindah ke pemain lawan, yang disertai dengan penambahan skor pada lawannya. Berdasarkan hasil eksperimen dalam penelitian tersebut, metode ini terbukti sangat baik, dengan keberhasilan mendeteksi kejadian pelanggaran secara benar, mencapai tingkat akurasi sebesar 100% (terdeteksi 28/28 pelanggaran). Salah satu keterbatasan dari metode yang diusulkan adalah deteksi foul dilakukan secara tidak langsung melalui perubahan informasi pada scoreboard, bukan melalui analisis langsung terhadap interaksi objek di atas meja, seperti pergerakan cue ball, kontak pertama dengan bola sasaran, posisi pocket, maupun kontak bola dengan

rail. Oleh karena itu, pendekatan tersebut belum secara khusus membahas deteksi pelanggaran berdasarkan hubungan visual antar objek permainan yang terjadi di atas meja biliar.

Berbeda dari metode yang mengandalkan analisis video dan papan skor, penelitian yang dilakukan oleh Zheng dan Yan (2025) memperkenalkan TCGA-Pool, sebuah kerangka analisis video khusus biliar yang tujuannya untuk mendeteksi *clear shot* (*pot* yang berhasil tanpa pelanggaran), identifikasi momen permainan berakhir, dan perhitungan bola yang masuk (Zheng & Yan, 2025). TCGA-Pool memakai ResNet-50 sebagai *frame encoder* untuk mengekstrak fitur setiap *frame*, kemudian modul TCGA menyaring fitur berdasarkan konteks seluruh *clip*, menurunkan yang kurang penting, dan menonjolkan konteks yang relevan, dan merangkum waktunya untuk klasifikasi. TCGA-Pool memberikan peningkatan signifikan pada semua metrik evaluasi dibanding dengan metode lainnya, sebesar 4.7% pada *clear shot detection* (akurasi sebesar 87.4%). Penelitian ini relevan karena sama-sama meneliti 9-ball, namun belum fokus pada hal *foul detection* dan implementasi pada *edge device*.

Berdasarkan latar belakang tersebut, penelitian ini mengusulkan sebuah kerangka deteksi pelanggaran pada permainan biliar yang berlandaskan aturan resmi dan dievaluasi pada rekaman pertandingan nyata. Pendekatan ini diharapkan dapat meningkatkan akurasi pemutusan pelanggaran dan konsistensi keputusan wasit.

Secara keseluruhan, penelitian ini diawali dengan pengumpulan data dan proses anotasi pada setiap bola, stik, serta elemen meja yang relevan. Setelah itu, dataset diperluas melalui augmentasi (Wang et al., 2025) yang mempertahankan label, misalnya perubahan perspektif, penyesuaian kontras, penambahan blur, rotasi, dan teknik lainnya. Data yang telah siap kemudian digunakan untuk melatih sebuah model visi komputer yang akan bekerja sebagai cara untuk “melihat” meja biliar dan mengklasifikasikan benda yang berada di atas meja. Tahap berikutnya adalah menyusun modul aturan foul dengan Python. Seluruh hasil dievaluasi pada rekaman pertandingan nyata (Padilla et al., 2020), dan pada tahap akhir sistem diimplementasikan pada sebuah *edge device* yang mampu menjalankan model dan sistem deteksi *foul*.

Penelitian ini tidak mencakup seluruh bentuk pelanggaran dalam aturan resmi 9-ball, melainkan dibatasi pada pelanggaran yang dapat dianalisis melalui informasi visual dari kamera dan hasil deteksi objek. Pembatasan ini dilakukan agar sistem dikembangkan tetap sesuai dengan kemampuan model deteksi objek, logika program yang berbasis koordinat, dan juga kebutuhan implementasi pada *edge device*. Pelanggaran yang akan dideteksi dalam penelitian ini ditampilkan pada Tabel 1.1.

Tabel 1. 1. Pelanggaran yang akan diimplementasikan

Jenis Pelanggaran	Indikator Visual
<i>Cue ball scratch</i>	<i>Cue ball</i> terdeteksi berada di area <i>pocket</i> atau masuk ke <i>pocket</i> .

<i>No legal hit</i>	<i>Cue ball</i> tidak mengenai bola sasaran yang sah. ini mencakup <i>cue ball</i> yang tidak mengenai bola apa pun atau <i>cue ball</i> pertama kali mengenai bola yang bukan bola bernomor terendah di atas meja.
<i>Cue ball off field</i>	<i>Cue ball</i> keluar dari area permainan.
<i>No rail contact on shot</i>	Tidak ada bola yang masuk ke <i>pocket</i> dan tidak ada bola yang menyentuh rail.

1.2. Rumusan Masalah

Berdasarkan latar belakang yang telah tertulis di atas mengenai kebutuhannya sebuah sistem pendeteksi pelanggaran dalam permainan biliard, berikut adalah rumusan masalah yang dapat ditarik untuk penelitian yang akan dilaksanakan.

1. Bagaimana membangun dan merancang arsitektur model visi komputer yang mampu mendeteksi dan melokalisasi bola biliard, *pocket*, dan elemen meja lainnya pada permainan *9-ball pool*?
2. Bagaimana mengembangkan logika deteksi pelanggaran pada permainan *9-ball pool* berdasarkan hasil deteksi objek dari model yang telah dilatih?
3. Bagaimana sistem deteksi pelanggaran *9-ball* dapat diimplementasikan pada *platform edge computing*?

1.3. Tujuan Penelitian

Berdasarkan rumusan masalah yang tertulis di atas, berikut adalah tujuan dari penelitian yang akan dilaksanakan:

1. Membangun dan merancang arsitektur model visi komputer yang mampu mendeteksi dan melokalisasi bola biliard, *pocket* dan elemen relevan lainnya di atas meja.
2. Mengembangkan logika deteksi pelanggaran pada permainan *9-ball pool* berdasarkan hasil deteksi objek dari model yang telah dilatih.
3. Mengimplementasikan sistem deteksi pelanggaran *9-ball* pada *platform edge computing*.

1.4. Manfaat Penelitian

1.4.1. Subjektif

1. Bagi peneliti, penelitian ini menjadi sumber untuk menerapkan dan memperdalam pengetahuan dalam bidang kecerdasan buatan, khususnya visi komputer, dan implementasi sistem pada perangkat keras.
2. Penelitian ini menjadi kontribusi bagi Universitas Pradita dengan menambah referensi ilmiah di bidang penerapan AI, *deep learning*,

dan visi komputer, khususnya terkait deteksi bola permainan *billiard*.

1.4.2. Objektif

1. Sistem dapat menjadi alat bantu wasit untuk meningkatkan akurasi dan objektivitas saat mengambil keputusan pada 9-ball.
2. Sistem dapat memberikan informasi deteksi pelanggaran secara otomatis berdasarkan hasil analisis visual, sehingga dapat membantu proses evaluasi rekaman pertandingan secara lebih konsisten..

1.5. Ruang Lingkup Pembahasan

Pada penelitian ini, pembahasan yang akan menjadi fokus utama adalah sebagai berikut:

1. Mengevaluasi hasil *training* visi komputer.
2. Mengevaluasi akurasi dari sistem deteksi pelanggaran *9-ball* menggunakan model yang terlatih.
3. Mengevaluasi performa sistem deteksi pelanggaran *9-ball* pada *edge device*.

Penelitian ini akan dilaksanakan secara berkelanjutan hingga ketiga poin evaluasi tersebut tercapai guna memastikan tujuan penelitian terpenuhi. Batasan masalah yang ditetapkan dalam penelitian ini adalah sebagai berikut:

- Penelitian ini difokuskan pada permainan 9-ball pool dan tidak mencakup deteksi pelanggaran pada permainan 8-ball pool
- Sistem deteksi pelanggaran yang dikembangkan hanya mencakup empat jenis pelanggaran, yaitu *cue ball scratch*, *no legal hit*, *cue ball* keluar dari area permainan, dan tidak adanya kontak dengan *rail* setelah sebuah pukulan.
- Sistem tidak mendeteksi seluruh jenis pelanggaran resmi pada permainan 9-ball, seperti *three consecutive fouls*, *illegal jump shot*, *illegal masse*, dan pelanggaran akibat sentuhan tangan atau stik terhadap bola.
- Deteksi pelanggaran dilakukan berdasarkan hasil deteksi objek, posisi *bounding box*, pergerakan bola, interaksi antara *cue ball*, *object ball*, *pocket*, dan *rail*.
- *Dataset* yang dibangun berasal dari video pertandingan *public* 9-ball untuk mendapatkan akurasi tinggi pada *edge device* serta video yang diambil secara pribadi menggunakan meja biliar mainan.

1.6. Metode Penelitian

Penelitian ini menggunakan pendekatan System Development Life Cycle (SDLC), dengan tahapan sebagai berikut:

1. Studi Literatur

Meninjau aturan resmi BCA untuk 9-ball, riset terkait deteksi peristiwa/foul pada biliar, konsep inti visi komputer, model visi komputer yang layak untuk sistem

pendeteksi pelanggaran, metrik evaluasi, serta kelayakan deployment di perangkat *edge*.

2. Analisis Kebutuhan

Merumuskan kebutuhan fungsional: kelas objek (bola bernomor, *cue ball*, stik, *rail*, kantong), kebutuhan data (multi-sudut, variasi pencahayaan/gerak), target kinerja (akurasi deteksi tinggi untuk mendukung logika *foul*, serta kebutuhan non-fungsional (portabilitas model ke perangkat berdaya rendah).

3. Perancangan Sistem

Mendesain arsitektur modular: pipeline data (anotasi, lalu augmentasi), pelatihan model visi komputer dengan skema validasi, modul aturan foul berbasis Python yang mengonsumsi output deteksi (bbox + kelas), menampilkan hasil deteksi pelanggaran pada antarmuka sistem dan menyiapkan integrasi ke *edge device* untuk inferensi, serta rencana integrasi ke *edge device* untuk inferensi.

4. Implementasi dan Pengembangan

Membangun dataset beranotasi, menerapkan augmentasi yang relevan (contohnya perubahan perspektif, pencahayaan, *blur*, rotasi), melatih model visi komputer, mengembangkan modul aturan foul dan integrasi I/O video, lalu melakukan optimisasi model agar muat memori/komputasi *edge device*.

5. Pengujian dan Evaluasi

Sistem dievaluasi menggunakan dataset uji dan video pengujian yang terdiri dari video publik serta video pribadi dari setup meja biliar sederhana. Yang dievaluasi performa deteksi (*Precision*, *Recall*, mAP), akurasi keputusan *foul*, serta metrik sistemik (FPS/latensi) di perangkat target.

6. Analisis Hasil dan Pembahasan

Menulis capaian metrik vs target, membahas kegagalan umum, *trade-off* akurasi-kecepatan, dan merumuskan perbaikan serta arah kerja lanjut.

1.7. Sistematika Penulisan

Penulisan laporan Tugas Akhir ini terbagi dalam beberapa bab yang tersusun sebagai berikut:

BAB I – PENDAHULUAN

Bab ini berisi latar belakang penelitian mengenai kebutuhan sistem bantu deteksi pelanggaran pada permainan 9-ball pool. Pembahasan diawali dengan penjelasan mengenai pentingnya objektivitas dalam pengambilan keputusan pelanggaran, keterbatasan pengamatan manual oleh wasit, dan juga pemanfaatan visi komputer untuk membantu proses deteksi. Terdapat juga dalam bab ini rumusan masalah, tujuan

penelitian, manfaat penelitian, ruang lingkup pembahasan, metode penelitian secara umum, dan sistematika penulisan. Pada bagian ruang lingkup pembahasan, penelitian dibatasi pada permainan 9-ball dan 4 jenis pelanggaran, yaitu *cue ball scratch*, *no legal hit*, *cue ball off field*, dan *no rail contact on shot*.

BAB II – TINJAUAN PUSTAKA

Pada bab ini, teori-teori yang menjadi dasar dalam pengembangan sistem deteksi pelanggaran 9-ball pool. Pembahasan mengandung aturan dasar dan jenis pelanggaran dalam permainan 9-ball, konsep kecerdasan buatan dan visi komputer, metode deteksi objek, arsitektur YOLO sebagai *one stage detector*, proses anotasi dan augmentasi dataset, dan konsep *edge computing*. Bab ini juga membahas beberapa penelitian terdahulu yang berkaitan dengan deteksi objek, analisis video olahraga, dan sistem berbasis visi komputer.

BAB III – METODOLOGI PENELITIAN

Pada bab ini, dijelaskan tahapan-tahapan yang dilakukan dalam pengembangan sistem deteksi pelanggaran 9-ball pool. Tahapan tersebut meliputi pengumpulan data video, pengambilan *frame*, anotasi objek, pengolahan dataset, augmentasi data, pelatihan model, serta konversi model agar dapat masuk kepada *edge device*. Bab ini juga menjelaskan perancangan logika deteksi pelanggaran berdasarkan hasil deteksi objek, seperti posisi *cue ball*, *object ball*, *pocket*, *rail*, dan area permainan. Selain itu, bab ini memuat metode evaluasi yang digunakan untuk mengukur kinerja model deteksi objek, akurasi deteksi pelanggaran, serta performa sistem pada *platform edge computing*.

BAB IV – ANALISIS DAN PEMBAHASAN

Bab ini berisi hasil implementasi dan pengujian sistem deteksi pelanggaran pada permainan 9-ball. Pembahasan berisi hasil pelatihan model deteksi objek, evaluasi performa model menggunakan metrik *precision*, *recall*, dan *mean Average Precision* (mAP), dan juga kemampuan sistem dalam mendeteksi empat jenis pelanggaran yang menjadi batasan penelitian.

BAB V – KESIMPULAN DAN SARAN

Bab ini berisi kesimpulan dari hasil penelitian berdasarkan rumusan masalah dan tujuan yang telah ditetapkan. Kesimpulan menjelaskan sejauh mana sistem mampu mendeteksi objek dan pelanggaran pada permainan 9-ball, serta bagaimana performa sistem ketika dijalankan pada perangkat *edge*. Bab ini juga memberikan saran untuk pengembangan selanjutnya.

BAB II TINJAUAN PUSTAKA

2.1. Kerangka Teoritis

Berdasarkan Encyclopaedia Britannica, biliar merujuk pada berbagai permainan yang dimainkan di atas meja berbentuk persegi panjang dengan sejumlah bola kecil dan tongkat panjang yang disebut *cue*. Permainan biliar utama termasuk *snooker*, *pocket billiards*, atau *pool*, dimainkan pada meja yang memiliki enam lubang (*pocket*), satu di setiap sudut dan satu di masing-masing sisi panjang. Terdapat banyak varian untuk setiap permainan tersebut.

2.1.1. Permainan 9-Ball Pool dan Pelanggaran

Seperti olahraga lain, setiap varian biliar memiliki kumpulan aturan sendiri. Penelitian ini berfokus pada permainan biliar 9-ball pool. Ringkasan definisi pelanggaran untuk 9-ball disajikan pada Tabel 2.1 (Billiards Congress of America, 2026). Tabel 2.2 menjelaskan aturan umum yang merupakan pelanggaran dalam biliar (Billiards Congress of America, 2026).

Tabel 2. 1. Peraturan Pelanggaran 9-ball Berdasarkan BCA.

Pelanggaran	Penalty
Gagal memukul bola 1 terlebih dahulu saat break dan gagal memasukkan bola atau membuat lebih dari empat bola menyentuh ban. Bola putih masuk lubang atau keluar meja saat break. Bola sasaran terlempar keluar meja saat break.	<i>Foul</i> , pemain berikutnya mendapat bola bebas. Bola sasaran yang terlempar keluar saat <i>break</i> tidak dikembalikan kecuali bola 9 yang harus dikembalikan.
Pemain tidak memukul bola dengan nomor terkecil terlebih dahulu.	<i>Foul</i> , pemain berikutnya mendapat bola bebas.
Pemain memukul bola yang benar, tetapi tidak ada bola yang menyentuh <i>rail</i> .	<i>Foul</i> , pemain berikutnya mendapat bola bebas.
Membuat bola sasaran terlempar keluar dari meja.	<i>Foul</i> , bola yang keluar tidak dikembalikan, kecuali bola 9. Lawan mendapat bola bebas.
Mencoba melakukan <i>jump shot</i> atau <i>masse</i> melewati atau memutari bola penghalang, dan bola penghalang tersebut bergerak.	Pelanggaran <i>cue ball</i> , bola bebas untuk lawan.
Melakukan <i>foul</i> tiga kali berturut-turut dalam permainan yang sama.	Pemain kalah dalam permainan tersebut.
Melakukan <i>foul</i> saat memasukkan bola 9.	Jika bola 9 masuk pada pukulan yang merupakan <i>foul</i> , bola 9 dikembalikan ke titik dan lawan mendapat bola bebas.

Tabel 2. 2. Peraturan Pelanggaran Permainan Ball Berdasarkan BCA.

Pelanggaran	Penalty
<i>Cue ball</i> gagal menyentuh bola sasaran yang sah.	<i>Foul</i> , pemain berikutnya mendapat bola bebas.
<i>Cue ball</i> masuk ke lubang saat melakukan pukulan.	<i>Foul</i> , pemain berikutnya mendapat bola bebas.
Memukul, menyentuh, atau melakukan kontak dengan <i>cue ball</i> atau bola sasaran apa pun dalam permainan, kecuali dinyatakan aman untuk dilakukan.	<i>Foul</i> , pemain berikutnya mendapat bola bebas.
Menyentuh bola sasaran apa pun dengan bola putih saat <i>cue ball</i> sedang dalam posisi bebas.	<i>Foul</i> , pemain berikutnya mendapat bola bebas.
Bola berhenti di permukaan selain alas meja setelah pukulan.	<i>Foul</i> , pemain berikutnya mendapat bola bebas.

Penelitian ini tidak mendeteksi seluruh jenis pelanggaran resmi. Penelitian dibatasi pada pelanggaran yang dapat dianalisis melalui informasi visual kamera dan hasil deteksi objek. Oleh karena itu, pelanggaran yang menjadi fokus penelitian adalah *cue ball scratch*, *no legal hit*, *cue ball off field*, dan *no rail contact on shot*. Tabel 2.3 menyimpulkan *foul* atau pelanggaran apa saja yang akan terdeteksi dan indikator visualnya.

Tabel 2. 3. Pelanggaran dan indikator visual

Pelanggaran	Indikator visual
<i>Cue ball scratch</i>	<i>Cue ball</i> masuk ke area pocket atau tidak terdeteksi kembali setelah berada dekat pocket.
<i>No legal hit</i>	<i>Cue ball</i> tidak mengenai bola atau pertama kali mengenai bola yang bukan bola bernomor terendah di atas meja.
<i>Cue ball off field</i>	<i>Cue ball</i> tidak terdapat pada area permainan.
<i>No rail contact on shot</i>	Setelah pukulan terjadi, tidak ada bola yang masuk ke pocket dan tidak ada bola yang menyentuh rail.

Pembatasan tersebut penting karena sistem yang dikembangkan bergantung pada data visual berupa posisi objek, perubahan posisi objek, dan hubungan antarobjek

dalam rangkaian frame video. Dengan demikian, pelanggaran yang membutuhkan informasi non-visual, seperti sentuhan tangan terhadap bola atau pelanggaran teknis tertentu yang tidak terlihat jelas oleh kamera, tidak termasuk dalam ruang lingkup penelitian ini.

2.1.2. Deteksi Pelanggaran Berbasis Visi Komputer

Untuk secara otomatis mengidentifikasi berbagai pelanggaran yang didefinisikan dalam aturan, sistem harus mampu ‘melihat’ dan menafsirkan keadaan permainan dari rangkaian *frame* video. Kemampuan ini disediakan dengan adanya kecerdasan buatan (*artificial intelligence*) dan khususnya sub-bidang visi komputer (*computer vision*) (Xu et al., 2021). Visi komputer memungkinkan mesin mengekstrak informasi bermakna dari masukan visual, yang dalam penelitian ini melibatkan analisis aliran video dari meja biliar. Untuk bisa menafsirkan permainan, sistem visi komputer harus mampu mengidentifikasi dan melacak berbagai elemen kunci di atas meja. Hal ini diperlukan untuk menentukan interaksi antara *cue ball* dan bola-bola objek. Meskipun informasi ini cukup untuk membangun pendeteksi biliar sederhana, pendeteksi *foul* yang kuat membutuhkan juga informasi mengenai sisi meja (*rail*) dan masing-masing *pocket*.

Deteksi pelanggaran pada 9-ball tidak cukup hanya dengan mengetahui objek secara terpisah. Sistem juga perlu mengetahui hubungan antarobjek. Sebagai contoh, untuk mendeteksi *no legal hit*, sistem perlu mengetahui posisi *cue ball*, bola bernomor terendah, dan bola pertama yang terkena setelah pukulan. Untuk mendeteksi *cue ball scratch*, sistem perlu mengetahui hubungan antara posisi *cue ball* dan area *pocket*. Sementara itu, untuk mendeteksi *no rail contact on shot*, sistem perlu mengetahui apakah setelah pukulan terdapat bola yang menyentuh *rail* atau masuk ke *pocket* dan mengetahui lokasi kedua hal tersebut (*rail* dan *pocket*).

Berdasarkan kebutuhan tersebut, metode deteksi objek menjadi pendekatan yang sesuai karena dapat menghasilkan dua informasi utama, yaitu kelas objek dan posisi objek dalam bentuk bounding box. Informasi tersebut dapat digunakan sebagai dasar untuk membangun logika deteksi pelanggaran berbasis koordinat. Dengan kata lain, model visi komputer berfungsi sebagai komponen yang “melihat” objek permainan, sedangkan modul aturan berbasis Python digunakan untuk menafsirkan hasil deteksi tersebut menjadi keputusan pelanggaran.

2.1.3. Dataset dan Kebutuhan Pelatihan Model

Agar model deteksi objek mampu mengenal elemen permainan secara baik, diperlukan dataset yang berisi berbagai kondisi visual permainan. Dataset tersebut perlu memiliki variasi perspektif kamera, *lighting*, warna meja, posisi bola, serta kondisi sebelum dan sesudah pukulan. *Variasi data diperlukan agar model tidak hanya mengenali objek pada kondisi tertentu, tetapi juga mampu bekerja pada kondisi permainan yang berbeda.*

Setiap gambar dalam dataset perlu diberi anotasi agar model dapat belajar mengenali objek yang relevan. Anotasi pada penelitian ini dilakukan dalam bentuk *bounding box*, yaitu kotak pembatas yang menunjukkan lokasi objek pada gambar. Objek yang perlu dianotasi meliputi bola bernomor, *cue ball*, *pocket*, dan elemen lain yang diperlukan oleh sistem. Selain anotasi, augmentasi data juga digunakan untuk memperluas variasi data pelatihan dengan cara memodifikasi gambar yang sudah ada, misalnya melalui perubahan kecerahan, kontras, *blur*, rotasi, atau perubahan perspektif (Wang et al., 2025).

Dataset yang telah disiapkan kemudian digunakan untuk melatih model deteksi objek. Model yang dihasilkan akan dievaluasi berdasarkan kemampuannya dalam mendeteksi kelas objek dan menentukan lokasi objek secara akurat. Hasil deteksi tersebut menjadi dasar bagi sistem untuk membaca pergerakan bola dan menentukan terjadinya pelanggaran.

2.1.4. Implementasi Pada Edge Device

Selain mendeteksi pelanggaran berdasarkan video, penelitian ini juga diarahkan pada implementasi sistem di perangkat edge. *Edge computing* memungkinkan pemrosesan dilakukan secara lokal pada perangkat, tanpa harus selalu bergantung pada *server*. Pendekatan ini sesuai untuk sistem deteksi berbasis kamera karena proses inferensi dapat dilakukan langsung pada perangkat yang terhubung dengan kamera.

Perangkat seperti Raspberry Pi memiliki kelebihan dari sisi ukuran, konsumsi daya, dan portabilitas. Namun, perangkat tersebut memiliki keterbatasan komputasi jika dibandingkan dengan komputer yang memiliki GPU khusus. Berdasarkan penelitian Minott et al. (2025), Raspberry Pi 5 masih sangat bergantung pada CPU saat menjalankan beban kerja AI, sehingga penggunaan akselerator dapat membantu mengurangi beban pemrosesan dan meningkatkan performa inference (Minott et al., 2025). Oleh karena itu, penelitian ini menggunakan Raspberry Pi 5 dengan akselerator Hailo-8L sebagai platform implementasi sistem. Dengan adanya edge device dan akselerator AI, sistem diharapkan dapat menjalankan model deteksi objek secara lebih efisien. Evaluasi pada tahap ini tidak hanya melihat akurasi deteksi objek, tetapi juga performa komputasi seperti kecepatan inference, penggunaan sumber daya, dan kelayakan sistem untuk dijalankan pada perangkat edge.

2.2. Tinjauan Teoritis

2.2.1. Computer Vision

Computer vision merupakan bidang dalam kecerdasan buatan yang memungkinkan komputer untuk memperoleh, memproses, dan memahami informasi visual dari gambar maupun video. Dalam penelitian ini, *computer vision* digunakan untuk membaca keadaan permainan 9-ball pool berdasarkan frame video. Informasi visual yang

diproses mencakup keberadaan bola, *cue ball*, *cue*, *pocket*, serta posisi objek-objek tersebut di atas meja.

Menurut Szeliski, *computer vision* berkaitan dengan bagaimana komputer dapat menafsirkan gambar digital untuk memahami struktur, properti, bentuk, dan informasi dari suatu adegan visual (Richard Szeliski, 2021). Pada sistem deteksi pelanggaran *9-ball*, kemampuan ini diperlukan karena pelanggaran tidak hanya ditentukan oleh keberadaan objek, tetapi juga oleh hubungan antarobjek. Sebagai contoh, sistem perlu mengetahui posisi *cue ball* terhadap *pocket*, kontak *cue ball* dengan bola bernomor, serta kemungkinan bola menyentuh *rail* setelah pukulan. Dengan demikian, *computer vision* menjadi dasar utama dalam penelitian ini karena seluruh proses deteksi pelanggaran bergantung pada kemampuan sistem dalam membaca objek permainan dan perubahan posisinya dari waktu ke waktu.

2.2.2. Image Classification, Image Segmentation, dan Object Detection

Dalam *computer vision*, terdapat beberapa pendekatan utama untuk memahami informasi visual, di antaranya *image classification*, *image segmentation*, dan *object detection*. Ketiga pendekatan ini memiliki karakteristik yang berbeda sesuai dengan jenis informasi yang ingin diperoleh dari gambar. *Image classification* merupakan proses pemberian label kelas terhadap sebuah gambar secara keseluruhan. *Image classification* dapat dipahami sebagai proses mengelompokkan gambar ke dalam kategori tertentu berdasarkan fitur visual yang dimiliki (Mohamed et al., 2022). Pendekatan ini berguna ketika sistem hanya perlu mengetahui isi utama suatu gambar. Namun, *image classification* kurang sesuai untuk penelitian ini karena sistem tidak cukup hanya mengetahui bahwa terdapat bola biliar pada frame, tetapi juga perlu mengetahui lokasi bola tersebut.

Image segmentation merupakan proses membagi gambar menjadi beberapa bagian berdasarkan objek atau kategori tertentu. Berdasarkan survei Minaee et al., *image segmentation* dapat dilakukan dalam bentuk *semantic segmentation*, *instance segmentation*, maupun *panoptic segmentation* (Minaee et al., 2022). Segmentasi mampu memberikan informasi spasial yang lebih detail karena bekerja pada tingkat pixel. Meskipun demikian, pendekatan ini membutuhkan proses anotasi yang lebih kompleks dan beban komputasi yang lebih besar. Oleh karena itu, *image segmentation* tidak menjadi pendekatan utama dalam penelitian ini.

Object detection merupakan pendekatan yang tidak hanya mengenali kelas objek, tetapi juga menentukan lokasi objek dalam gambar. Menurut Budiharto et al. (2018), *object detection* mampu menggabungkan proses klasifikasi dan lokalisasi objek dalam satu sistem (Budiharto et al., 2018). *Output* dari *object detection* umumnya berupa kelas objek, *bounding box*, dan *confidence score*. Pendekatan ini paling sesuai untuk penelitian ini karena sistem membutuhkan informasi jenis objek dan posisi objek untuk membangun logika deteksi pelanggaran berbasis koordinat.

2.2.3. Object Detection dengan YOLO

YOLO atau *You Only Look Once* merupakan salah satu model *object detection* yang banyak digunakan karena memiliki keseimbangan antara kecepatan dan akurasi. Menurut Terven et al. (2023), YOLO dirancang sebagai model deteksi objek yang mampu melakukan prediksi *bounding box* dan kelas objek secara cepat melalui satu proses inferensi (Terven et al., 2023). Hal ini berbeda dari pendekatan dua tahap yang umumnya memisahkan proses pencarian region dan klasifikasi objek.

Sebagai *one-stage detector*, YOLO cocok digunakan pada pemrosesan video karena setiap *frame* dapat diproses secara langsung untuk menghasilkan prediksi objek. Dalam penelitian ini, YOLO digunakan untuk mendeteksi objek-objek pada permainan 9-ball, seperti *cue ball*, bola bernomor, *cue stick*, *pocket*, dan *rack*. Namun, tidak seluruh kelas deteksi digunakan dalam logika pelanggaran. Pada penelitian ini, keputusan foul hanya didasarkan pada kelas dan area visual yang relevan terhadap empat jenis pelanggaran, yaitu *cue ball*, bola bernomor, *pocket*, dan *rail zone*. Hasil deteksi tersebut kemudian digunakan sebagai input bagi modul aturan deteksi pelanggaran.

Kelebihan YOLO terletak pada kemampuannya menghasilkan deteksi dalam waktu cepat, sehingga cocok untuk sistem yang akan berjalan pada perangkat edge. Selain itu, terdapat varian YOLO berukuran kecil seperti YOLOv8n, YOLOv9t, YOLOv10n, dan YOLO11n yang diperuntukan penggunaan lebih ringan dibandingkan varian normalnya. Model berukuran ringan lebih sesuai untuk perangkat dengan sumber daya terbatas karena membutuhkan komputasi yang lebih rendah.

Dalam penelitian ini, YOLO tidak hanya digunakan untuk mendeteksi objek, tetapi juga menjadi dasar untuk membaca hubungan antarobjek. *Bounding box* yang dihasilkan YOLO digunakan untuk menghitung titik tengah objek, memantau perubahan posisi bola, mendeteksi interaksi antara *cue ball* dan *object ball*, serta menentukan hubungan objek dengan *pocket* dan *rail zone*.

2.2.4. Dataset Annotation dan Data Augmentation

Pelatihan model *object detection* memerlukan dataset yang telah dianotasi. Anotasi dataset adalah proses pemberian label pada setiap objek yang terdapat dalam gambar. Pada *object detection*, anotasi dilakukan dengan *bounding box*, yaitu kotak pembatas yang menunjukkan lokasi objek. Setiap *bounding box* memiliki informasi kelas objek dan koordinat posisi objek pada gambar.

Dalam konteks penelitian ini, anotasi diperlukan agar model dapat belajar mengenali objek-objek permainan 9-ball. Objek yang dianotasi meliputi bola bernomor, *cue ball*, *cue*, *pocket*, dan objek lain yang dibutuhkan oleh sistem. Format anotasi yang digunakan perlu disesuaikan dengan model deteksi objek yang dilatih.

Karena penelitian ini menggunakan YOLO, maka format anotasi yang digunakan adalah format YOLO.

Selain anotasi, *data augmentation* juga diperlukan untuk meningkatkan variasi *dataset*. Augmentasi data merupakan teknik untuk menghasilkan variasi data baru dari data yang sudah ada dengan tetap mempertahankan label aslinya (Wang et al., 2025). Augmentasi dapat membantu model menjadi lebih tahan terhadap variasi kondisi visual, seperti perbedaan pencahayaan, sudut kamera, *blur*, rotasi, dan perubahan perspektif.

Augmentasi data dapat dilakukan dengan mengubah tampilan gambar agar model terbiasa menghadapi kondisi visual yang berbeda. Perubahan *brightness* dan *contrast* digunakan untuk memperkenalkan variasi pencahayaan dan tingkat kejelasan objek. Variasi *brightness* membantu model mengenali objek pada kondisi cahaya yang berbeda, sedangkan *contrast* membantu model belajar mengenali perbedaan antara objek dan latar belakang tanpa butuh mendapatkan gambar yang memiliki satu kondisi visual yang dibutuhkan (Park et al., 2025).

Selain perubahan pencahayaan, augmentasi juga dapat dilakukan dengan mengubah posisi, ukuran, dan sudut tampilan objek. *Horizontal flip* dapat menambah variasi arah gambar dan membantu mengurangi risiko *overfitting*. *Shift*, *scale*, dan *rotate* membuat objek muncul pada posisi, ukuran, dan kemiringan yang berbeda. *Random scale* dapat membantu model mengenali objek seperti terlihat dari jarak kamera yang berbeda, sedangkan *perspective transformation* dapat membantu model menghadapi perubahan sudut pandang kamera (Park et al., 2025).

Augmentasi juga dapat digunakan untuk meniru penurunan kualitas gambar yang sering terjadi pada misalkan kamera. *Motion blur* dapat menggambarkan kondisi ketika objek bergerak cepat atau kamera kurang stabil, sedangkan *gaussian blur* membuat gambar menjadi lebih lembut atau kurang tajam. *Noise* dan kompresi gambar juga relevan karena video dapat mengalami penurunan kualitas akibat kondisi kamera, pencahayaan yang rendah, atau proses kompresi file (Gholinavaz et al., 2025).

CLAHE atau *Contrast Limited Adaptive Histogram Equalization* merupakan teknik yang digunakan untuk meningkatkan kontras lokal pada gambar. Teknik ini dapat membantu memperjelas detail struktur pada gambar, utamanya di bagian gambar yang memiliki perbedaan intensitas rendah (Inoue et al., 2024). Sementara itu, variasi bayangan dapat terjadi akibat posisi sumber cahaya, posisi kamera, atau objek di sekitar area pengambilan gambar.

2.2.5. Edge Computing dan AI Accelerator

Edge computing merupakan pendekatan komputasi yang memproses data secara lokal pada perangkat, bukan sepenuhnya bergantung pada *server*. Dalam sistem berbasis kamera, *edge computing* memungkinkan proses inferensi dilakukan langsung pada perangkat yang berada dekat dengan sumber data. Pendekatan ini dapat

mengurangi ketergantungan terhadap koneksi internet dan membuat sistem lebih portabel.

Raspberry Pi merupakan salah satu perangkat *edge* yang banyak digunakan karena berukuran kecil, relatif hemat daya, dan dapat menjalankan berbagai aplikasi komputasi. Berdasarkan Alqahtani et al. (2024), Raspberry Pi 5 memiliki peningkatan performa dibandingkan Raspberry Pi 4, termasuk dalam hal CPU, GPU, dan bandwidth I/O (Alqahtani et al., 2024). Raspberry Pi 5, menggunakan RAM 4 GB LPDDR4x, terbukti lebih unggul dalam hal pemrosesan, di mana waktu inferensi menggunakan model SSD lebih cepat (93ms) dibandingkan dengan Raspberry Pi 4 (209ms). Namun, meskipun performanya meningkat, Raspberry Pi tetap memiliki keterbatasan dibandingkan komputer dengan GPU khusus.

Pada beban kerja AI, penggunaan akselerator dapat membantu meningkatkan kecepatan inferensi. Minott et al. (2025) menjelaskan bahwa Raspberry Pi 5 masih sangat bergantung pada CPU ketika menjalankan beban kerja AI, sehingga penggunaan akselerator menjadi penting untuk mengurangi beban pemrosesan (Minott et al., 2025). Dalam penelitian ini, akselerator yang digunakan adalah Hailo-8L, yaitu *Neural Processing Unit* (NPU) yang dapat menjalankan model hasil kompilasi dalam format Hailo *Executable Format* atau .hef (Hailo AI, 2026). Berdasarkan dokumentasi resmi Raspberry Pi untuk AI Kit/AI HAT+ yang memakai akselerator Hailo di Raspberry Pi 5, Hailo-8/Hailo-8L berperan sebagai NPU (Neural Processing Unit), yaitu akselerator AI untuk mempercepat inference, yang dipasang ke Raspberry Pi 5 lewat antarmuka PCIe. Raspberry Pi menjelaskan bahwa varian AI Kit berbasis Hailo-8L 13 TOPS, sedangkan AI HAT+ tersedia dalam varian 13 TOPS (Hailo-8L) dan 26 TOPS (Hailo-8). Alur yang umum digunakan untuk model kustom adalah melakukan optimasi atau kuantisasi dan mengompilasi model menjadi file HEF (Hailo Executable Format) menggunakan Hailo *Dataflow Compiler*, lalu menjalankan *inference* dengan HailoRT.

Penggunaan Hailo-8L bertujuan untuk mempercepat inferensi model deteksi objek pada Raspberry Pi 5. Dengan demikian, evaluasi sistem tidak hanya berfokus pada akurasi deteksi, tetapi juga pada performa komputasi, seperti kecepatan inferensi, latensi, dan kelayakan model untuk dijalankan pada edge device.

Di sisi lain terdapat Coral USB Accelerator dari Google. Berdasarkan dokumentasi resmi dari Coral, Coral adalah platform Edge AI dari Google yang menggabungkan hardware Edge TPU dan toolchain software agar model dapat dijalankan secara lokal di edge device dengan efisien (konsumsi daya rendah) namun tetap cepat untuk kebutuhan inference. Salah satu produk Coral yang kompatibel dengan Raspberry Pi (3/4) di Linux adalah ‘Coral USB Accelerator’, yaitu modul USB yang menambahkan Edge TPU ke host sehingga inference bisa jauh lebih kencang dibanding CPU saja. Sayangnya Edge TPU hanya mendukung model TensorFlow Lite yang fully 8-bit quantized dan biasanya perlu dikompilasi khusus memakai Edge TPU Compiler. Dokumentasi Coral juga menyebut 1 Edge TPU mampu sekitar 4 TOPS dan efisiensinya sekitar 0,5 watt per TOPS, sehingga cocok untuk kebutuhan visi komputer real-time di perangkat kecil.

Pemilihan akselerator AI perlu disesuaikan dengan jenis model yang digunakan. Pada model YOLO, Raspberry Pi 5 dengan AI HAT+ lebih sesuai dibandingkan Coral USB TPU karena mampu mempertahankan ukuran input 640×640 , sedangkan deployment YOLO pada Coral TPU menggunakan ukuran input 320×320 . Coral TPU memang memiliki waktu inferensi yang lebih cepat, tetapi penurunan ukuran input tersebut dapat menurunkan akurasi YOLO (Alqahtani et al., 2026). Sebaliknya, AI HAT+ mampu mempertahankan akurasi YOLO dengan lebih baik dan memiliki konsumsi energi per request yang lebih rendah, sehingga lebih sesuai untuk sistem deteksi objek berbasis YOLO pada edge device. (Alqahtani et al., 2026)

2.2.6. Metrik Evaluasi Object Detection

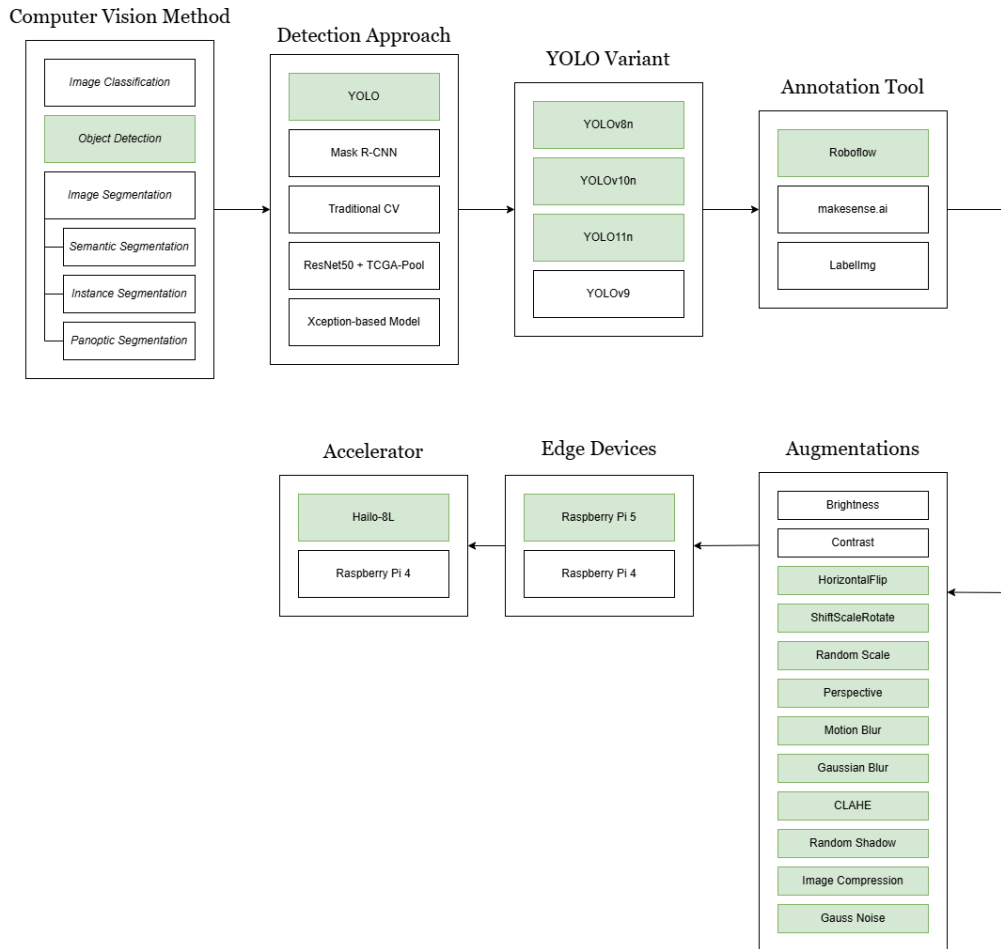
Evaluasi object detection dilakukan untuk mengetahui seberapa baik model dapat mendeteksi objek dan menentukan lokasinya. Menurut Padilla et al. (2020), evaluasi object detection umumnya melibatkan metrik seperti *precision*, *recall*, *Intersection over Union*, dan *mean Average Precision* (Padilla et al., 2020).

Precision menunjukkan seberapa banyak prediksi model yang benar dibandingkan dengan seluruh prediksi yang dihasilkan. Nilai *precision* yang tinggi menunjukkan bahwa model jarang menghasilkan deteksi yang salah. *Recall* menunjukkan seberapa banyak objek sebenarnya yang berhasil dideteksi oleh model. Nilai *recall* yang tinggi menunjukkan bahwa model mampu menemukan sebagian besar objek yang ada pada gambar.

Intersection over Union atau IoU digunakan untuk mengukur tingkat tumpang tindih antara *bounding box* hasil prediksi dan *bounding box* ground truth. Semakin besar nilai IoU, semakin baik posisi *bounding box* yang diprediksi oleh model. *Mean Average Precision* atau mAP digunakan untuk mengukur performa deteksi secara keseluruhan dengan mempertimbangkan *precision* dan *recall* pada berbagai ambang IoU.

Dalam penelitian ini, metrik evaluasi yang digunakan meliputi *precision*, *recall*, mAP@50, dan mAP@50-95. *Precision* dan *recall* digunakan untuk melihat kemampuan model dalam menghasilkan prediksi yang benar dan menemukan objek yang relevan. mAP@50 digunakan untuk menilai akurasi deteksi pada ambang IoU 0,50, sedangkan mAP@50-95 digunakan untuk menilai performa model secara lebih ketat pada berbagai ambang IoU. Selain metrik deteksi objek, sistem juga dievaluasi berdasarkan kemampuan mendeteksi empat jenis pelanggaran serta performa inferensi pada Raspberry Pi 5 dan Hailo-8L.

2.3. Studi Preseden



Gambar 2. 1. Komponen Studi Preseden

Menurut penelitian J. Yu, Y. Huang, dan Y. He yang berjudul "Snooker video event detection using multimodal features", sistem yang dikembangkan bertujuan untuk mendeteksi dan memberi anotasi sembilan kejadian spesifik seperti nice shots, *fouls*, dan high breaks dalam pertandingan snooker. Metode yang digunakan meliputi *Canny edge detection*, OCR, dan Hidden Markov Model (HMM). Hasil pengujian menunjukkan bahwa deteksi kejadian frames, *high breaks*, *defensive counterattacks*, dan *fouls* berhasil mencapai akurasi 100 persen, sedangkan *nice shots* mendapatkan akurasi 93.96 persen. Namun, performa deteksi untuk faults dan kejadian lucu terbukti masih buruk dalam sistem ini.

Menurut penelitian A. Zheng dan W. Q. Yan yang berjudul "Attention-Pool: 9-Ball Game Video Analytics with Object Attention and Temporal Context Gated Attention", fokus penelitian adalah mendeteksi *clear shot*, identifikasi kondisi kemenangan, dan perhitungan bola yang masuk ke *pocket*. Metode yang diterapkan menggunakan arsitektur TCGA-Pool dengan *Temporal Context Gated Attention* (TCGA) dan *Frame Encoder* berbasis backbone ResNet-50 yang dilatih pada dataset 58.078 frame video 9-ball pool. Hasil pengujian menunjukkan tingkat akurasi sebesar 87.4 persen untuk deteksi clear shot, 94.8 persen untuk identifikasi kondisi kemenangan, dan 82.1 persen untuk penghitungan bola yang masuk. Selain itu, model ini terbukti efisien karena hanya menggunakan 27,3 juta *parameter* dan 13,9 G FLOPS.

Menurut penelitian W. K. Pan, D. Zhu, J. Wang, dan H. Zhu yang berjudul "*Effective recognition design in 8-ball billiards vision systems for training purposes based on Xception network modified by improved Chaos African Vulture Optimizer*", penelitian ini bertujuan mendeteksi objek dan melakukan *pattern recognition* untuk membedakan bola solid dengan bola *stripe*. Metode yang digunakan terdiri dari *Improved Chaos African Vulture Optimizer* (ICAVO), *color space transformation*, dan *thresholding*. Proses pengujian model dilakukan pada dataset yang berisi 100 gambar. Berdasarkan hasil eksperimen, sistem ini berhasil mencapai tingkat akurasi yang sangat tinggi yaitu 98.65 persen untuk pengenalan bola *solid* dan 98.927 persen untuk bola *stripe*.

Menurut penelitian Y. Tian yang berjudul "*A Multimodal Intelligent Assessment Method and Solution for Enhancing Billiard Skills*", tujuan utama proyek ini adalah mengembangkan metode evaluasi cerdas berbasis data *multimodal* yang menilai aspek akurasi pukulan dan kontrol bola, serta membuat program pelatihan yang bersifat *personalized*. Metode yang diajukan diimplementasikan melalui pembuatan arsitektur *transformer* baru yang dapat menangani data *multimodal* dengan lebih baik. Hasil pengujian menunjukkan bahwa kelompok eksperimen mengalami peningkatan signifikan dibandingkan kelompok kontrol dengan latihan tradisional. Secara spesifik, kelompok eksperimen unggul dalam akurasi pukulan sebesar 85 persen berbanding 78 persen, pengurangan kesalahan kontrol sebesar 5.8 persen berbanding 8.9 persen, serta pengambilan keputusan sebesar 83 persen berbanding 74 persen.

Menurut penelitian M. Buric, M. Pobar, dan M. Ivasic-Kos yang berjudul "*Ball detection using YOLO and mask R-CNN*", penelitian ini membandingkan kinerja YOLO dengan mask R-CNN dalam hal deteksi objek bola non-biliar. Metode yang dijalankan melibatkan proses pelatihan arsitektur R-CNN dan YOLOv2, khususnya varian *tiny-YOLO*. Hasil perbandingan menunjukkan bahwa model YOLOv2 memiliki kecepatan inferensi yang hampir 20 kali lebih cepat dibandingkan dengan R-CNN. Selain itu, terdapat peningkatan *F1-score* pada YOLOv2 dan R-CNN dari kisaran 6 hingga 34 persen berbanding 8 hingga 18 persen, serta peningkatan *recall* yang berdampak pada penurunan nilai *precision*.

Menurut penelitian J. K. Park dan J. Park yang berjudul "*Intelligent Carom Billiards Assistive System for automatic solution path generation and actual path prediction with principal component analysis-based ball motion detection*", proyek ini bertujuan mengembangkan *Intelligent Carom Billiards Assistive System* (ICBAS) sebagai sistem bantu pemain pemula carom billiards agar dapat melihat jalur solusi cue ball dan jalur solusi pukulan secara otomatis. Metode computer vision ini memanfaatkan kamera monokular untuk mendeteksi meja, posisi bola, dan cue stick menggunakan *HSV thresholding*, *contour extraction*, *circle Hough transform*, *homography*, dan *principal component analysis* untuk mendeteksi pergerakan bola. Hasil penelitian menunjukkan bahwa sistem bantu tersebut mampu mendeteksi posisi dan gerakan bola secara tepat. Sistem ini juga berhasil menghitung jalur solusi permainan dan memproyeksikan jalur prediksi secara langsung ke atas meja biliar.

Menurut penelitian Z. Zhang, W. Zhang, dan W. Chen yang berjudul "*Design of Intelligent Assistant System for Billiards Hit Training*", sistem ini dirancang untuk menjadi alat bantu latihan pukul biliar bagi para pemain pemula. Tujuan utama sistem adalah mendeteksi posisi bag, bola biliar, dan *white ball*, lalu memberikan saran arah dan kekuatan yang dibutuhkan pukulan agar bola dapat masuk dengan lebih mudah. Metode yang diterapkan meliputi *bimodal gray histogram threshold*, *sobel edge detection*, *hough transform*, serta *normalized RGB* untuk memisahkan bola dari background, ditambah persamaan kinetik untuk memberikan saran pukulan dan simulasi jalur bola. Hasil penelitian menunjukkan bahwa metode pengenalan objek dan simulasi jalur dapat berjalan dengan baik untuk membantu latihan biliar, meskipun data hasil tidak mengandung nilai numerik seperti akurasi atau persentase keberhasilan.

Menurut penelitian I. Silva, R. Chavez, J. L. Castillo-Sequera, dan L. Wong yang berjudul "*System for Table Run Strategy in Peruvian Billiards with Real-Time Object Detection using YOLOv8n*", sistem bernama Billiard Sense dikembangkan untuk membantu pemain biliar menentukan strategi table run atau closing the table. Sistem ini memberikan rekomendasi pukulan dan posisi cue ball berikutnya agar pemain dapat memasukkan beberapa bola secara berurutan dalam satu giliran. Metode yang digunakan adalah deteksi objek real-time menggunakan YOLOv8n untuk mendeteksi bola, *cue ball*, dan *cue stick* dengan bantuan GoPro Hero 10 sebagai kamera, OpenCV untuk pengolahan gambar, serta algoritma *shot suggestion* dan *shot prediction*. Ketika sistem diuji pada 24 pemain, performanya mampu menurunkan jumlah rata-rata pukulan menjadi 22 dibanding pukulan rata-rata pemain profesional yaitu 47, dengan tingkat penerimaan rekomendasi pukulan mencapai 100 persen untuk pemula, 95.1 persen untuk pemain menengah, dan 95.6 persen untuk pemain profesional.

BAB III

METODOLOGI PENELITIAN

3.1. Gambaran Umum Objek Penelitian

3.1.1. Objek Penelitian

Objek dalam penelitian ini adalah sistem deteksi pelanggaran pada permainan *9-ball pool* berbasis visi komputer. Sistem ini dikembangkan untuk mengenali objek-objek penting dalam permainan, seperti *cue ball*, bola bernomor, *cue*, *pocket*, dan *rail*. Informasi visual dari objek-objek tersebut kemudian digunakan sebagai dasar untuk menentukan apakah suatu kejadian dalam permainan termasuk sebuah pelanggaran atau tidak.

Penelitian ini tidak mencakup seluruh jenis pelanggaran dalam aturan resmi 9-ball. Pelanggaran yang menjadi fokus penelitian dibatasi pada empat jenis, yaitu *cue ball scratch*, *no legal hitt*, *cue ball off field*, dan *no rail contact on shot*. Pembatasan ini dilakukan karena keempat jenis pelanggaran tersebut masih dapat dianalisis melalui informasi visual seperti posisi objek, perubahan posisi objek, interaksi antar objek, dan keberadaan objek dalam area permainan.

3.1.2. Perangkat dan Bahan Penelitian

Penelitian ini menggunakan beberapa perangkat keras, perangkat lunak, dan bahan pendukung untuk membangun sistem deteksi pelanggaran *9-ball pool*

1. Perangkat Keras

- Raspberry Pi 5
- Hailo-8L AI Accelerator
- Webcam Logitech C922 Pro Stream (1920x1080, 30FPS)
- Laptop Axioo RTX 4060
- Meja biliar mainan
- Bola biliar
- Tripod/holder kamera

2. Perangkat Lunak

- Python (versi 3.12.7)
- Roboflow
- Ultralytics YOLO (versi 8.4.67)0.3
- OpenCV (versi 4.10.0)
- Hailo Dataflow Compiler/ HailoRT
- VLC media player

3. Bahan/data Penelitian

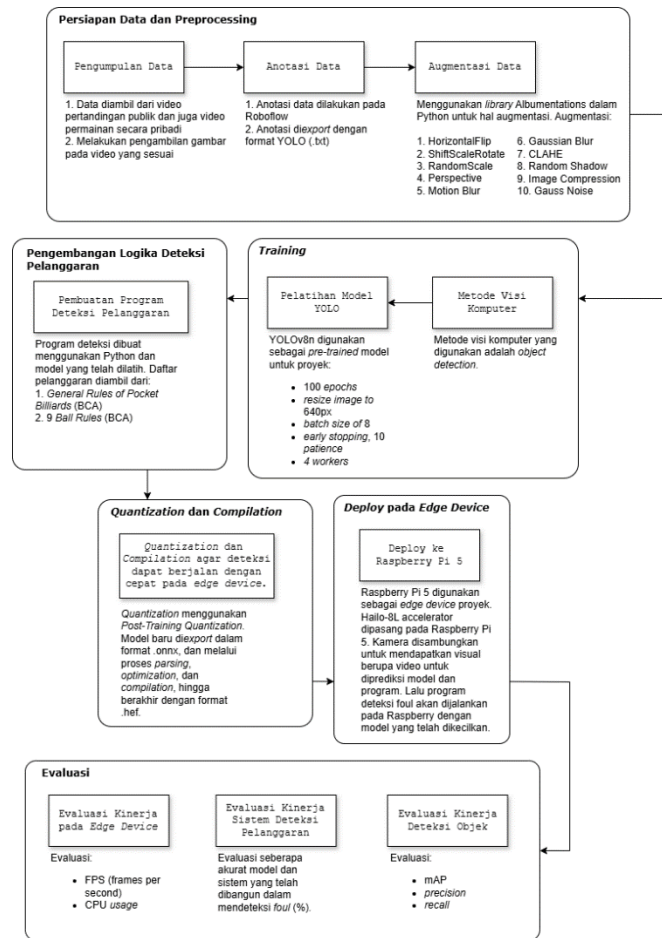
- Video publik dari YouTube
- Video pribadi dari meja biliar mainan

- Dataset anotasi YOLO
- Model .pt dan .hef

3.1.3. Gambaran Umum Alur Penelitian

Secara umum, penelitian ini terdiri dari beberapa tahapan utama, yaitu persiapan data, pengolahan dataset, pelatihan model deteksi objek, pengembangan logika deteksi pelanggaran, konversi model ke *format* Hailo Executable Format atau .hef, dan evaluasi sistem pada perangkat *edge*. Tahapan tersebut disusun agar sistem tidak hanya mampu mendeteksi objek, tetapi juga mampu menggunakan hasil deteksi objek untuk mengambil keputusan pelanggaran.

Gambar 3.1 menunjukkan alur umum metodologi penelitian. Alur penelitian dimulai dari pengumpulan video permainan *9-ball*, baik dari video publik maupun hasil perekaman pribadi. Video tersebut kemudian dipecah menjadi *frame-frame* gambar individual. *Frame* yang telah dipilih kemudian dianotasi dengan *bounding box* untuk menandai objek-objek yang relevan. *Dataset* yang telah dianotasi digunakan untuk melatih model YOLO, kemudian hasil model tersebut diterapkan dalam sistem deteksi pelanggaran berbasis aturan.



Gambar 3. 1 Alur Metodologi Penelitian

3.1.4. Persiapan Data

Tahap persiapan data dimulai dengan pengumpulan video permainan *9-ball pool*. Data yang digunakan terdiri dari data sekunder dan data primer. Data sekunder diperoleh dari rekaman pertandingan *9-ball* yang tersedia secara publik, sedangkan data primer diperoleh melalui perekaman pribadi menggunakan meja biliar mainan. Penggunaan kedua jenis data ini bertujuan untuk memperkaya variasi visual dalam dataset.

Video yang dikumpulkan memiliki variasi kondisi pencahayaan, sudut pandang kamera, kualitas video, warna taplak meja, dan posisi objek di atas meja. Variasi ini diperlukan agar model yang dilatih tidak hanya mengenali objek dalam satu kondisi tertentu, tetapi juga mampu melakukan generalisasi terhadap kondisi permainan yang berbeda. Beberapa momen penting yang diperhatikan dalam proses pemilihan data meliputi kondisi sebelum pukulan, saat *cue* mengenai *cue ball*, pergerakan bola setelah pukulan, bola mendekati *pocket*, dan kondisi bola setelah berhenti. Setelah video terkumpul, bagian dari video akan secara manual diambil sebagai gambar individual dengan menggunakan *VLC media player*.

3.1.5. Pengolahan Dataset

Frame yang telah dipilih kemudian melalui proses anotasi. Anotasi dilakukan dengan memberikan label kelas dan bounding box pada objek-objek yang dibutuhkan oleh sistem. Objek yang dianotasi meliputi *cue ball*, bola bernomor, *cue*, dan *pocket*. Proses anotasi dilakukan menggunakan Roboflow karena platform tersebut menyediakan fitur anotasi, manajemen dataset, penyimpanan *progress*, serta ekspor dataset ke *format* YOLO. Dalam format ini, setiap gambar memiliki file anotasi dengan *extension* .txt. Setiap baris dalam file anotasi berisi ID kelas objek dan koordinat *bounding box* yang telah dinormalisasi terhadap ukuran gambar. Format ini digunakan karena sesuai dengan kebutuhan pelatihan model YOLO.

Tabel 3.1 menunjukkan daftar kelas objek yang digunakan dalam proses anotasi pelatihan model deteksi objek. Setiap kelas diberi ID numerik yang sesuai dengan format YOLO. Pada penelitian ini, *rail* tidak dijadikan sebagai kelas objek dalam proses anotasi. Area *rail* dihitung secara otomatis berdasarkan titik tengah *pocket* yang terdeteksi oleh model.

Tabel 3. 1. Setiap kelas yang *ditraining*

ID Kelas	Nama Kelas	Kuantitas
0	1-ball	607
1	2-ball	650
2	3-ball	898
3	4-ball	950
4	5-ball	854
5	6-ball	889
6	7-ball	954
7	8-ball	1,065
8	9-ball	1,169
9	cue_ball	1,254
10	cue_stick	237
11	pocket	7,016
12	rack	46

Setelah dataset siap, data dibagi menjadi *training set*, *validation set*, dan *test set*. Pembagian dataset ini dilakukan agar evaluasi model lebih objektif. Rasio pembagian data adalah 7:2:1:

1. *Training Set*: Sebagian besar data (70%) dialokasikan untuk melatih model. Model akan belajar mengenali pola dan fitur dari set data ini.
2. *Validation Set*: Sebagian kecil data (20%) digunakan untuk memvalidasi dan menyetel *parameter* model selama proses pelatihan. Data ini membantu memantau kinerja model pada data yang belum pernah dilihat dan bantu dalam mencegah *overfitting*.
3. *Test Set*: Sisa data (10%) disimpan dan hanya digunakan saat model selesai dilatih. Set ini memberikan evaluasi akhir yang objektif tentang seberapa baik model dapat berkinerja pada data dunia nyata yang sepenuhnya baru.

Tahap berikutnya adalah augmentasi data. Augmentasi dilakukan untuk menambah variasi data *training* tanpa perlu mengambil data baru. Pada penelitian ini, augmentasi hanya diterapkan pada *training set*. *Validation set* dan *test set* tidak diaugmentasi agar proses evaluasi dilakukan pada data asli.

Augmentasi yang digunakan adalah *horizontal flip*, *ShiftScaleRotate*, *random scale*, *perspective motion blur*, *gaussian noise*, CLAHE, *random shadow*, *image compression*, dan *gauss noise*. Horizontal flip dipakai untuk menambah variasi arah tampilan gambar. ShiftScaleRotate dipakai agar model tidak terbiasa pada posisi dan kemiringan objek tertentu. Random scale digunakan untuk meniru kamera yang kemungkinan tidak selalu sama jaraknya dengan data latihan. *Motion blur* dan *gaussian blur* untuk meniru kondisi gambar yang kurang tajam akibat pergerakan bola, pergerakan kamera, atau kualitas video. Random shadow digunakan untuk meniru keadaan nyata, dimana mungkin muncul sebuah bayangan pada meja akibat posisi kamera dan pencahayaan. *Image compression* dan *gauss noise* digunakan untuk meniru penurunan kualitas kamera yang bisa terjadi pada kamera pada saat rekaman.

Beberapa augmentasi tidak digunakan, seperti *brightness* atau *contrast*. Ini dikarenakan *brightness* dapat membuat warna bola tertentu menjadi terlalu terang. Bola 4-ball berisiko tertukan dengan *cue ball* karena warnanya yang mirip. *Contrast* juga dapat membuat bola seperti 7-ball terlihat lebih kuning, sehingga berisiko tertukar dengan 1-ball. Oleh karena itu, augmentasi yang dipilih difokuskan pada perubahan posisi, sudut pandang, kualitas gambar, bayangan, dan noise tanpa mengubah warna utama dari bola.

3.1.6. Pelatihan Model Deteksi Objek

Model deteksi objek yang digunakan dalam penelitian ini berasal dari keluarga YOLO. YOLO dipilih karena mampu melakukan klasifikasi objek dan lokalisasi objek

dalam satu proses inferensi. Kemampuan ini sesuai dengan kebutuhan sistem deteksi pelanggaran 9-ball pool, karena sistem tidak hanya perlu mengetahui jenis objek yang muncul pada *frame*, tetapi juga posisi objek tersebut di atas meja.

Penelitian sebelumnya yang membandingkan YOLOv8s, YOLOv9s, YOLOv10s, dan YOLO11s menunjukkan bahwa YOLOv9s mendapatkan akurasi tertinggi, dengan *precision* 97.6% *recall* 97.6%, dan mAP 98.7%, sedangkan YOLO11s memiliki waktu inferensi tercepat sebesar 13,8ms. YOLOv9 tidak digunakan dalam penelitian karena penelitian berarah pada varian *nano* yang lebih sebanding untuk implementasi edge. Model yang akhirnya digunakan adalah YOLOv8n, YOLOv10n, dan YOLO11n. Selain itu, varian ringan YOLOv9 yang tersedia lebih diketahui sebagai *tiny*, bukan *nano* (YOLOv9t) (Bento et al., 2025). Ketiga model ini dipilih karena termasuk varian ringan (*nano*) dari keluarga YOLO, dan juga merupakan beberapa model yang didukung oleh toolchain Hailo untuk proses kompilasi ke format .hef. Penggunaan beberapa varian model dilakukan untuk membandingkan kemampuan masing-masing model dalam mendeteksi objek permainan 9-ball, baik dari sisi akurasi maupun performa komputasi. Tabel 3.2 menampilkan perbandingan kinerja model yang dipilih untuk dilatih (Ultralytics, 2026).

Tabel 3. 2. Perbandingan kinerja model YOLO *nano*

Metrik	YOLOv8n	YOLOv10n	YOLO11n
mAP@50-90	37.3	39.5	39.5
Parameters	3.2 million	2.3 million	2.6 million
FLOPs	8.7 billion	6.7 billion	6.5 billion
Dukungan Hailo	✓	✓	✓

Setiap model dilatih menggunakan dataset yang sama agar hasil perbandingan lebih adil. Dataset yang digunakan telah melalui proses anotasi, pembagian data, dan augmentasi. Setelah pelatihan selesai, setiap model menghasilkan file bobot dengan format .pt. File tersebut kemudian dievaluasi menggunakan metrik *precision*, *recall*, mAP@50, dan mAP@50-95.

Ketiga model dilatih menggunakan konfigurasi *training* yang sama agar hasil perbandingan adil. Tabel 3.3 menampilkan konfigurasi *training*.

Tabel 3. 3. Konfigurasi Pelatihan

Parameter	Nilai
Epochs	100
Image Size	640
Batch Size	8
Patience	10

Workers	4
---------	---

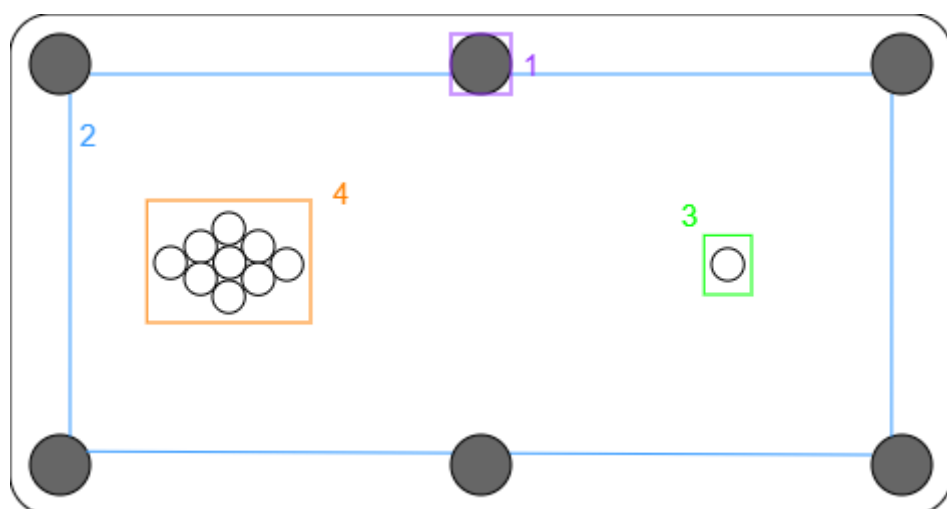
Parameter *epochs* ditetapkan sebesar 100 agar model mendapat kesempatan belajar yang cukup dari dataset. *Imgsz* atau *image size* sebesar 640 digunakan karena ukuran ini yang umum digunakan pada model YOLO dan sesuai untuk kebutuhan deteksi objek kecil seperti bola biliar. Parameter batch sebesar 8 dipilih dengan mempertimbangkan keterbatasan memori *device* saat *training*. *Patience* sebesar 10 digunakan untuk menghentikan *training* lebih awal jika tidak ada peningkatan performa dalam beberapa *epoch*. *Workers* sebesar 4 digunakan untuk mempercepat proses pembuatan data saat *training*.

Hasil perbandingan ketiga model dianalisis untuk melihat kelebihan dan keterbatasan masing-masing model. Sebuah model dapat memiliki akurasi yang lebih tinggi, tetapi membutuhkan waktu inferensi yang lebih besar. Sebaliknya, model lain dapat memiliki kecepatan inferensi yang lebih baik, tetapi dengan akurasi yang sedikit lebih rendah. Oleh karena itu, pembahasan hasil tidak diarahkan pada penentuan model terbaik secara mutlak, melainkan pada analisis hubungan antara akurasi, kecepatan, dan efisiensi komputasi.

3.1.7. Pengembangan Logika Deteksi Pelanggaran

Setelah model deteksi objek selesai dilatih, tahap selanjutnya adalah mengembangkan logika deteksi pelanggaran memakai Python. Logika ini menggunakan output YOLO berupa kelas objek, koordinat bounding box, dan *confidence score* pada setiap *frame*. Informasi tersebut lalu digunakan untuk membaca posisi dan pergerakan objek pada area permainan.

Gambar 3.2 menunjukkan ilustrasi tamplan atas meja biliar. Ilustrasi ini menggambarkan area utama dan objek yang digunakan oleh sistem sebagai acuan dalam mendeteksi pelanggaran.



Gambar 3. 2 Top-down view meja biliar

Keterangan pada Gambar 3.2 adalah sebagai berikut:

1. Pocket

Pocket merupakan area lubang pada meja biliar. Area ini digunakan untuk mendeteksi kemungkinan bola masuk ke lubang. Selain itu, posisi *pocket* juga digunakan sebagai petunjuk untuk membentuk area *rail* pada sistem.

2. Rail zone

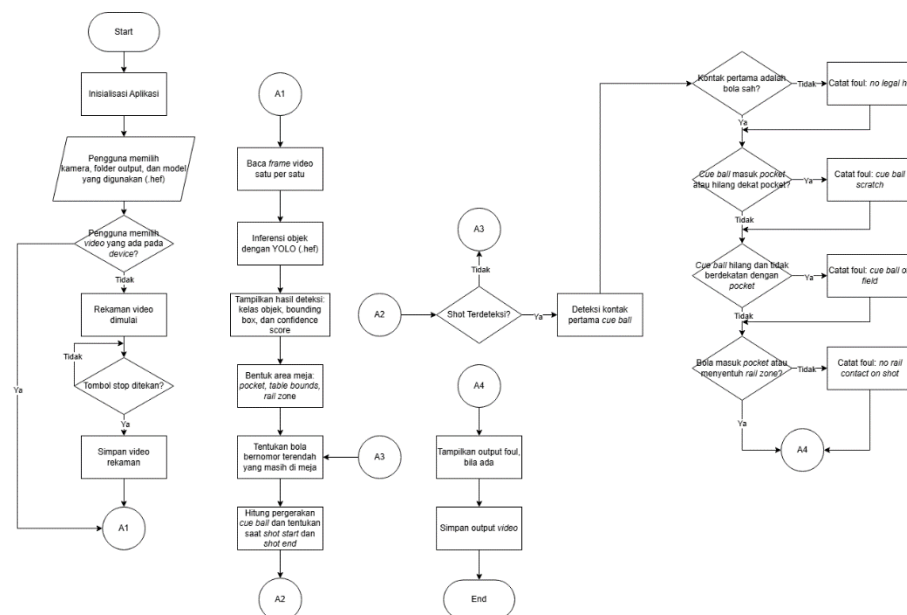
Rail zone merupakan area bantu yang digunakan untuk mendeteksi kontak bola dengan sisi meja. Area ini dibentuk dari titik tengah *pocket* yang terdeteksi oleh model. Titik tengah pocket tersebut dihubungkan menjadi sebuah *polygon* yang mengikuti bentuk area permainan. Pada pembentukan *polygon* ini, *pocket* tengah bagian bawah tidak digunakan agar bentuk *rail zone* tetap mengikuti sisi meja yang dibutuhkan dalam tampilan kamera.

3. Cue ball

Cue ball merupakan bola putih yang menjadi objek utama dalam proses deteksi pelanggaran. Pergerakan *cue ball* digunakan untuk menentukan awal pukulan, memantau kontak pertama dengan *object ball*, serta mendeteksi beberapa kondisi pelanggaran seperti *cue ball scratch*, *no legal hit*, dan kondisi ketika *cue ball* tidak terdeteksi oleh sistem (*cue ball off field*).

4. Object balls

Object balls merupakan bola-bola bernomor yang menjadi target permainan 9-ball. Pada ilustrasi, *object balls* ditampilkan dalam susunan diamond sesuai dengan posisi *rack* awal permainan 9-ball. Posisi *object balls* digunakan untuk menentukan bola bernomor terendah yang masih berada di atas meja.



Gambar 3. 3. Flowchart sistem deteksi pelanggaran 9-ball pool

Gambar 3.3 menampilkan *flowchart* dari program deteksi pelanggaran *9-ball pool*. Flowchart ini menjelaskan alur kerja sistem saat aplikasi dijalankan, mulai dari pemilihan sumber video, proses inferensi model YOLO, pembentukan area meja, pelacakan pergerakan bola, hingga pengecekan pelanggaran. Alur ini digunakan untuk memperjelas bagaimana hasil deteksi objek dari model diolah menjadi keputusan foul berdasarkan aturan yang telah ditentukan.

Saat aplikasi dijalankan, pengguna terlebih dahulu memilih sumber video, folder *output*, dan model YOLO berformat *.hef* yang akan digunakan. Sumber video dapat berasal dari file video yang sudah tersedia pada komputer atau dari hasil rekaman kamera. Jika pengguna memilih *file* video, sistem akan langsung memproses video tersebut. Jika pengguna memilih kamera, sistem akan melakukan perekaman terlebih dahulu hingga pengguna menghentikan proses rekaman. Setelah video tersedia, sistem melanjutkan proses ke tahap deteksi objek dan analisis pelanggaran.

Pada tahap pemrosesan, video dibaca secara *frame by frame*. Pada setiap *frame*, model YOLO melakukan inferensi melalui Hailo-8L untuk mendeteksi objek di atas meja, seperti *cue ball*, *object ball*, *pocket*, dan *cue stick*. Hasil deteksi yang diperoleh berupa kelas objek, *bounding box*, dan *confidence score*. Pada tahap inferensi, sistem menggunakan *confidence threshold* sebesar 0,35.

Setelah objek terdeteksi, sistem mengelompokkan hasil deteksi menjadi beberapa objek utama, yaitu *cue ball*, *object balls*, *pocket*, dan *cue stick*. Untuk setiap bola, sistem menghitung titik tengah *bounding box* sebagai posisi bola pada *frame* tersebut. Posisi ini disimpan dari *frame* ke *frame* untuk membaca pergerakan bola dan menghitung kecepatan *cue ball*. Selain itu, sistem juga menentukan bola bernomor terendah yang masih terdeteksi di atas meja sebagai target sah pada pukulan tersebut.

Sistem kemudian membentuk area penting pada meja, yaitu *pocket* dan *rail zone*. Area *pocket* didapatkan dari *bounding box pocket* yang terdeteksi oleh model. Dari setiap *bounding box pocket*, sistem menghitung titik tengah sebagai titik acuan. Titik-titik tengah *pocket* tersebut kemudian dihubungkan untuk membentuk *polygon rail zone*. Pada proses ini, *pocket* tengah bagian bawah tidak digunakan dalam pembentukan *polygon*. *Rail zone* yang terbentuk digunakan untuk mengecek apakah suatu bola menyentuh *rail* setelah pukulan terjadi.

Satu pukulan dianggap mulai ketika *cue ball* bergerak melewati ambang kecepatan tertentu. Saat pukulan dimulai, sistem menyimpan kondisi awal bola, seperti posisi awal, status bola terhadap *rail*, dan bola bernomor terendah yang menjadi target. Pukulan dianggap selesai ketika *cue ball* berada di bawah ambang kecepatan selama beberapa *frame*. Setelah pukulan selesai, sistem masih memberi jeda tambahan sebelum mengevaluasi *rail contact* agar bola yang masih bergerak tetap memiliki waktu untuk mencapai *rail* atau *pocket*.

Cue ball scratch dideteksi ketika posisi *cue ball* berada di dalam area *pocket* atau tumpang tindih dengan *bounding box pocket*. Dalam sistem ini, *cue ball* tidak perlu menghilang dari frame untuk diklasifikasikan sebagai *scratch*. Selama *cue ball* terdeteksi pada area *pocket*, sistem langsung mengklasifikasikan kejadian tersebut sebagai *cue ball scratch*.

No legal hit dideteksi dengan memperkirakan bola pertama yang terkena setelah *cue ball* bergerak. Sistem terlebih dahulu menentukan bola bernomor terendah yang masih berada di atas meja pada awal pukulan sebagai target sah. Setelah pukulan dimulai, sistem memeriksa kemungkinan kontak dengan beberapa cara, yaitu melalui tumpang tindih *bounding box* antara *cue ball* dan *object ball*, serta melalui lintasan pergerakan *cue ball*. Pemeriksaan lintasan digunakan untuk mengatasi kondisi ketika *cue ball* bergerak cepat sehingga kontak fisik tidak selalu terlihat jelas pada satu *frame*.

Kontak yang terdeteksi tidak langsung dianggap *valid*. Sistem terlebih dahulu menunggu beberapa *frame* untuk melihat apakah bola yang diduga terkena benar-benar mengalami perpindahan posisi. Jika bola tersebut bergerak melewati ambang perpindahan tertentu, maka kontak dianggap terjadi. Setelah kontak pertama ditentukan, kelas bola tersebut dibandingkan dengan bola bernomor terendah yang telah disimpan pada awal pukulan. Jika bola pertama yang terkena bukan bola bernomor terendah, maka sistem mengklasifikasikan kejadian tersebut sebagai *no legal hit*.

Selain itu, sistem juga menggunakan *fallback* berdasarkan bola yang pertama kali bergerak. Jika kontak tidak berhasil dikonfirmasi melalui tumpang tindih *bounding box* atau lintasan *cue ball*, sistem akan mencari *object ball* yang paling awal mengalami perpindahan. Untuk mengurangi kesalahan pada kondisi ketika bola lain bergerak akibat dorongan berantai, sistem memeriksa garis pandang antara *cue ball* dan bola yang bergerak pertama. Jika terdapat bola lain yang berada di antara *cue ball* dan bola tersebut, maka bola yang menghalangi garis pandang dianggap sebagai kandidat kontak pertama. Jika setelah seluruh proses tersebut tidak ada kontak yang dapat dikonfirmasi, sistem akan mengklasifikasikan pukulan sebagai *no legal hit*.

Cue ball off field dideteksi ketika *cue ball* tidak lagi terdeteksi pada area permainan. Untuk membedakannya dari *scratch*, sistem memeriksa posisi terakhir *cue ball* sebelum hilang. Jika posisi terakhir *cue ball* berada di sekitar area *pocket*, kejadian diklasifikasikan sebagai *scratch*. Jika tidak berkaitan dengan area *pocket*, kejadian diklasifikasikan sebagai *cue ball off field*.

No rail contact on shot dideteksi setelah *cue ball* menyentuh bola targetnya. Sistem akan memeriksa apakah terdapat bola yang masuk ke *pocket* atau menyentuh *rail zone* setelah terjadinya kontak tersebut. Jika sampai pukulan selesai dan belum ada bola yang masuk ataupun mengenai *rail zone*, maka sistem mengklasifikasikan kejadian sebagai *no rail contact on shot*.

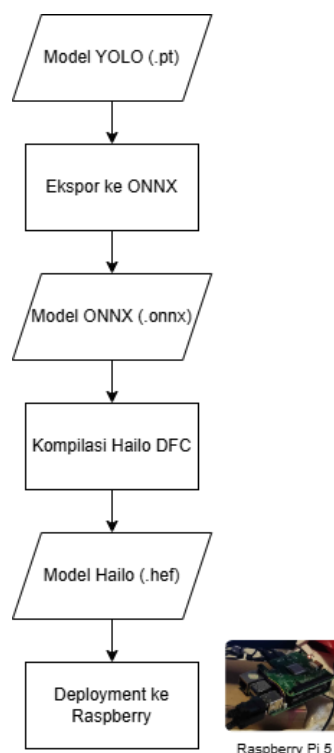
Hasil akhir dari proses ini ditampilkan pada layar dalam bentuk *overlay*, seperti *bounding box* objek, label kelas, target bola sah, *rail zone*, serta informasi foul yang

terdeteksi. Selain itu, sistem juga menyimpan video hasil anotasi dan file *log foul* ke dalam folder *output* yang telah dipilih oleh pengguna.

3.1.8. Konversi Model ke Format Hailo Executable Format

Setelah model YOLO selesai dilatih dan menghasilkan file *weights* dengan format .pt, tahap berikutnya adalah melakukan konversi model agar dapat dijalankan pada akselerator Hailo-8L. File .pt hasil pelatihan belum dapat langsung digunakan pada Hailo-8L karena format tersebut masih berupa format PyTorch. Agar model dapat digunakan pada akselerator Hailo-8L, model perlu melalui proses konversi dan kompilasi hingga menghasilkan file Hailo *Executable Format* atau .hef.

Proses konversi model dalam penelitian ini ditunjukkan pada Gambar 3.4. Alur dimulai dari model YOLO dalam format .pt, kemudian diekspor menjadi format ONNX. Format ONNX digunakan sebagai perantara agar model dapat dibaca *toolchain* Hailo. Setelah model berada dalam format ONNX, model diproses melalui beberapa tahap utama, yaitu *parsing*, optimasi, kuantisasi, dan kompilasi.



Gambar 3. 4. Alur Konversi Model YOLO ke Format .hef

Pada tahap parsing, struktur jaringan ONNX dibaca dan diterjemahkan ke dalam format Hailo, yang menghasilkan file HAR (Hailo Archive). Lalu, dilakukan optimasi dan kuantisasi, di mana diperlukan sejumlah *calibration images* dari dataset untuk

membantu menyesuaikan rentang nilai aktivasi model dengan data yang akan digunakan pada waktu inferensi. Tahap terakhir adalah kompilasi model menjadi file .hef. File .hef merupakan hasil akhir yang dapat dijalankan pada Raspberry Pi 5 dengan akselerator Hailo-8L. Dalam penelitian ini, proses kompilasi dilakukan pada tiga varian model, yaitu YOLOv8n, YOLOv10n, dan YOLO11n.

3.1.9. Implementasi dan Pengujian Model pada Raspberry Pi 5

Setelah model berhasil dikonversi ke format .hef, tahap berikutnya adalah mengimplementasikan model tersebut pada Raspberry Pi 5 untuk mengevaluasi kinerja model deteksi objek. Pada penelitian ini, inferensi tidak hanya digunakan untuk menjalankan sistem deteksi pelanggaran, tetapi juga digunakan untuk membandingkan performa model secara lebih luas antara CPU Raspberry Pi 5 dan akselerator Hailo-8L. Kedua pengujian yang dilakukan menggunakan *test set* yang sama agar hasil perbandingan tetap adil.

Pengujian pertama dilakukan dengan menjalankan model .pt menggunakan backend PyTorch pada CPU Raspberry Pi 5. Pengujian ini digunakan untuk mengetahui performa model tanpa menggunakan akselerator tambahan. Pada mode ini, model dievaluasi menggunakan fungsi validasi dari Ultralytics terhadap *dataset* uji. Hasil evaluasi yang diperoleh mencakup *precision*, *recall*, mAP@50, mAP@75, mAP@50-95, serta metrik per kelas.

Pengujian kedua dilakukan dengan menjalankan model .hef menggunakan HailoRT pada akselerator Hailo-8L. Pada mode ini, model yang telah dikompilasi dijalankan melalui Hailo NPU. Setiap gambar dari dataset uji dibaca, disesuaikan dengan ukuran *input* model, lalu diberikan ke *pipeline* inferensi Hailo. *Output* model kemudian diolah menjadi *bounding box*, kelas objek, dan *confidence score*. Hasil prediksi tersebut dibandingkan dengan *label ground truth* untuk menghitung metrik evaluasi deteksi objek. Metrik yang dihasilkan pada pengujian ini sama dengan pengujian pertama.

Hasil dari tahap ini digunakan untuk melihat pengaruh penggunaan Hailo-8L terhadap kinerja model deteksi objek, serta pengaruh kuantisasi dan optimasi terhadap akurasi model yang telah dikompilasi. Perbandingan ini digunakan untuk menilai apakah peningkatan kecepatan inferensi masih sebanding dengan perubahan akurasi yang terjadi setelah model dikompilasi ke format .hef.

3.2. Metode Penelitian

3.2.1. Metode Pengambilan Data

Data yang digunakan dalam penelitian ini berjenis sekunder, yang berarti data tidak diambil secara langsung di tempat, melainkan melalui suatu perantara, seperti contohnya *platform* publik seperti YouTube, dan primer, yang melibatkan pengambilan data pribadi. Untuk memastikan model yang *robust*, data akan diambil dari berbagai

sumber yang memiliki variasi dalam kondisi pencahayaan, kualitas video, dan sudut pandang kamera. Secara khusus, akan diupayakan untuk mendapatkan video dengan pandangan dari atas ke bawah (*top-down/overhead view*) yang ideal untuk pelacakan bola. Data sekunder akan diambil dari 8 rekaman pertandingan yang berbeda, dan akan diambil sekitar 500 atau lebih data primer.

3.2.2. Metode Pengolahan Data

Data video yang telah dikumpulkan diolah menjadi dataset gambar untuk pelatihan model deteksi objek. Proses pengambilan *frame* dilakukan secara manual dengan memilih frame tertentu menggunakan VLC media player. *Frame* yang dianggap relevan disimpan menggunakan fitur *save frame*.

Frame yang telah disimpan kemudian diunggah ke Roboflow untuk proses anotasi. Anotasi dilakukan dengan memberikan label kelas dan bounding box pada objek-objek yang dibutuhkan oleh sistem, seperti *cue ball*, bola bernomor, *pocket*, dan *rack*. Walaupun alat anotasi dasar seperti Makesense.ai praktis untuk digunakan, alat tersebut memiliki kelemahan karena tidak dapat menyimpan progress pekerjaan jika terjadi hal tidak diduga, seperti tab tertutup. Di sisi lain, LabelImg lebih aman untuk penggunaan labeling karena bersifat offline, namun fungsinya terbatas dan tidak terdapat fitur auto-label menggunakan model yang telah dilatih seperti Roboflow. Oleh karena itu, platform Roboflow yang digunakan dari mulai pelabelan, manajemen dataset, hingga pengeksportan ke format YOLO.

Dataset yang telah selesai dianotasi kemudian dibagi menjadi *training set*, *validation set*, dan *test set*. *Training set* digunakan untuk melatih model, *validation set* digunakan untuk memantau kinerja model selama pelatihan, sedangkan *test set* digunakan untuk evaluasi akhir. Pembagian dataset dilakukan dengan rasio 70% untuk *training set*, 20% untuk *validation set*, dan 10% untuk *test set*.

Selanjutnya, augmentasi diterapkan menggunakan Albumentations pada training set. Albumentations dipilih karena dapat melakukan transformasi gambar sekaligus menyesuaikan koordinat *bounding box* secara otomatis. Augmentasi yang digunakan meliputi *horizontal flip*, *ShiftScaleRotate*, *random scale*, *perspective*, *motion blur*, *gaussian blur*, CLAHE, *random shadow*, *image compression*, dan *gauss noise*. *Validation set* dan *test set* tidak diaugmentasi agar evaluasi model tetap dilakukan pada data asli.

3.2.3. Metode Analisis Data

Metode analisis data dilakukan terhadap seluruh data yang diperoleh dari hasil pelatihan, pengujian model, dan pengujian sistem deteksi pelanggaran. Analisis dilakukan untuk mengetahui kemampuan model dalam mendeteksi objek permainan 9-

ball, membandingkan performa model pada CPU dan Hailo-8L, serta menilai kemampuan sistem dalam mendeteksi pelanggaran berdasarkan hasil deteksi objek.

Analisis data meliputi:

1. Analisis hasil pelatihan model YOLOv8n, YOLOv10n, dan YOLO11n.
2. Analisis performa deteksi objek berdasarkan nilai *precision*, *recall*, mAP@50, dan mAP@50-95.
3. Analisis hasil deteksi per kelas, seperti *cue ball*, bola bernomor, *pocket*, dan *rack*.
4. Analisis perbandingan performa model .pt pada CPU Raspberry Pi 5 dan model .hef pada akselerator Hailo-8L.
5. Analisis kecepatan inferensi, penggunaan CPU, dan perubahan akurasi setelah proses konversi model.
6. Analisis hasil deteksi empat jenis pelanggaran, yaitu *cue ball scratch*, *no legal hit*, *cue ball off field*, dan *no rail contact on shot*.
7. Analisis kesalahan deteksi yang terjadi, seperti objek tidak terdeteksi, *bounding box* kurang akurat, atau objek tertutup oleh objek lain.

3.2.4. Metode Evaluasi Data

Metode evaluasi data dilakukan untuk mengukur keberhasilan model deteksi objek dan sistem deteksi pelanggaran yang dikembangkan.

Evaluasi data yang dilakukan:

1. Evaluasi model deteksi objek YOLOv8n, YOLOv10n, dan YOLO11n menggunakan metrik *precision*, *recall*, mAP@50, dan mAP@50-95.
2. Evaluasi hasil deteksi per kelas objek, seperti *cue ball*, bola bernomor, *pocket*, dan *rack*, untuk mengetahui kelas mana yang sudah terdeteksi dengan baik dan kelas mana yang masih sering mengalami kesalahan.
3. Evaluasi perbandingan model .pt pada CPU Raspberry Pi 5 dan model .hef pada akselerator Hailo-8L menggunakan test set yang sama.
4. Evaluasi performa inferensi sistem berdasarkan FPS, waktu inferensi per gambar, waktu end-to-end, dan penggunaan CPU.
5. Evaluasi kemampuan sistem dalam mendeteksi empat jenis pelanggaran, yaitu *cue ball scratch*, *no legal hit*, *cue ball off field*, dan *no rail contact on shot*.
6. Evaluasi hasil deteksi pelanggaran dilakukan dengan membandingkan output sistem terhadap ground truth yang ditentukan secara manual dari video pengujian.
7. Evaluasi kesalahan dilakukan dengan mengamati kondisi yang menyebabkan sistem gagal mendeteksi pelanggaran, seperti objek tidak terdeteksi, *bounding box* tidak akurat, *cue ball* tertutup objek lain, atau perubahan pencahayaan pada video.

Hasil evaluasi digunakan untuk menilai sejauh mana sistem yang dikembangkan mampu mencapai tujuan penelitian, yaitu melakukan deteksi objek, mendeteksi pelanggaran 9-ball, dan diimplementasikan pada Raspberry Pi 5 dengan akselerator Hailo-8L.

BAB IV ANALISIS DAN PEMBAHASAN

4.1. Hasil Pelatihan Model Deteksi Objek

Tabel 4. 1. Hasil evaluasi model deteksi objek pada CPU Raspberry Pi 5

Model	Precision	Recall	mAP@50	mAP@75	mAP@50-95
YOLOv8n	0.927	0.940	0.958	0.840	0.718
YOLOv10n	0.936	0.952	0.976	0.860	0.738
YOLO11n	0.966	0.948	0.969	0.857	0.736

Berdasarkan Tabel 4.1, seluruh model yang dijalankan pada CPU Raspberry Pi 5 memperoleh nilai mAP@50 di atas 0.95. Hal ini menunjukkan bahwa model mampu mendeteksi objek permainan 9-ball dengan baik pada test set. YOLOv10n memperoleh nilai mAP@50 tertinggi, yaitu 0.9766, serta mAP@50-95 tertinggi, yaitu 0.7387. Sementara itu, YOLO11n memperoleh precision tertinggi sebesar 0.9666. Hasil ini menunjukkan bahwa performa ketiga model pada backend CPU cukup baik, dengan perbedaan nilai yang tidak terlalu besar.

Tabel 4. 2. Hasil evaluasi model deteksi objek pada Hailo-8L

Model	Precision	Recall	mAP@50	mAP@75	mAP@50-95
YOLOv8n	0.944	0.907	0.930	0.706	0.589
YOLOv10n	0.978	0.919	0.953	0.775	0.639
YOLO11n	0.927	0.924	0.928	0.673	0.587

Berdasarkan Tabel 4.2, model yang telah dikonversi ke format .hef dan dijalankan pada Hailo-8L masih memperoleh nilai mAP@50 yang cukup tinggi. YOLOv10n memperoleh nilai terbaik pada backend Hailo-8L, dengan *precision* sebesar 0.978, mAP@50 sebesar 0.953, dan mAP@50-95 sebesar 0.639. Dibandingkan hasil pada CPU, nilai mAP@50-95 mengalami penurunan setelah model dijalankan dalam format .hef. Ini terjadi karena proses optimasi dan kuantisasi pada tahap kompilasi model ke format Hailo. Yang menarik adalah, nilai precision pada backend Hailo-8L terlihat lebih tinggi dibandingkan CPU pada YOLOv8n dan YOLOv10n. Ini menunjukkan bahwa model .hef terkadang menghasilkan prediksi yang lebih selektif, hingga jumlah prediksi salah atau *false positive* menjadi lebih

sedikit. Peningkatan *precision* ini juga diikuti dengan penurunan *recall* pada beberapa model, hingga beberapa objek yang seharusnya terdeteksi menjadi lebih mudah terlewat.

4.2. Evaluasi Deteksi Objek per Kelas

Evaluasi per kelas dilakukan untuk mengetahui kemampuan model dalam mendeteksi setiap objek yang digunakan. Berdasarkan hasil evaluasi sebelumnya, YOLOv10n menunjukkan performa paling stabil dibanding model lainnya. Oleh karena itu, pembahasan evaluasi per kelas difokuskan pada hasil YOLOv10n menggunakan backend CPU dan Hailo-8L. Tabel 4.3 menampilkan hasil evaluasi YOLOv10n pada CPU dan tabel 4.4 menampilkan hasil evaluasi pada Hailo-8L.

Tabel 4. 3. Hasil evaluasi per kelas YOLOv10n pada CPU Raspberry Pi 5

Kelas	Precision	Recall	mAP@50	mAP@50-95
1-ball	0.963	1.000	0.995	0.845
2-ball	0.968	1.000	0.994	0.787
3-ball	0.980	0.947	0.982	0.777
4-ball	0.988	0.994	0.995	0.768
5-ball	0.990	1.000	0.995	0.809
6-ball	0.971	1.000	0.995	0.802
7-ball	0.985	0.990	0.995	0.753
8-ball	0.983	0.991	0.995	0.787
9-ball	0.949	0.990	0.995	0.744
cue	0.978	1.000	0.995	0.773
cue_stick	1.000	0.479	0.771	0.232
pocket	0.984	0.987	0.994	0.798
rack	0.438	1.000	0.995	0.730

Berdasarkan Tabel 4.3, sebagian besar kelas bola bernomor memperoleh nilai mAP@50 yang tinggi, yaitu berada di atas 0.98. Hal ini menunjukkan bahwa model mampu mengenali bola-bola pada permainan 9-ball dengan baik. Kelas cue juga memperoleh hasil yang baik dengan precision sebesar 0.9780, recall sebesar 1.0000, dan mAP@50 sebesar 0.9950. Hasil ini penting karena cue ball menjadi objek utama dalam proses deteksi pelanggaran.

Tabel 4. 4. Hasil evaluasi per kelas YOLOv10n pada Hailo-8L

Kelas	Precision	Recall	mAP@50	mAP@50-95
1-ball	0.979	0.855	0.931	0.669
2-ball	1.000	1.000	0.995	0.688
3-ball	0.986	0.908	0.969	0.670
4-ball	0.987	0.963	0.985	0.699
5-ball	1.000	1.000	0.995	0.706
6-ball	1.000	0.977	0.993	0.646
7-ball	0.990	0.960	0.989	0.641
8-ball	0.990	0.981	0.994	0.703
9-ball	0.899	0.961	0.956	0.545
cue	0.990	0.935	0.970	0.647
cue_stick	0.923	0.444	0.648	0.238
pocket	0.977	0.968	0.981	0.664
rack	1.000	1.000	0.995	0.736

Pada *backend* Hailo-8L, sebagian besar kelas tetap memiliki mAP@50 yang tinggi. Kelas 2-ball, 5-ball, 7-ball, 8-ball, pocket, dan rack masih berada pada nilai mAP@50 di atas 0.98. Kelas *cue* juga memperoleh hasil yang cukup baik dengan *precision* sebesar 0.990, recall sebesar 0.935, dan mAP@50 sebesar 0.970. Walau begitu, nilai mAP@50-95 pada *backend* Hailo-8L banyak yang lebih rendah dibandingkan CPU. Penurunan ini menunjukkan bahwa model yang telah dikompilasi ke format .hef masih mampu mendeteksi objek pada IoU 0,50, tapi menurun pada evaluasi IoU yang lebih ketat. Ini bisa dilihat di beberapa kelas seperti 9-ball, cue, pocket, dan cue_stick.

Secara keseluruhan, hasil evaluasi per kelas menunjukkan bahwa model sudah mampu mendeteksi objek utama yang dibutuhkan sistem deteksi pelanggaran. Kelas *cue ball*, bola bernomor, dan *pocket* memiliki performa yang cukup baik pada CPU maupun Hailo-8L.

4.3. Perbandingan Performa CPU Raspberry Pi 5 dan Hailo-8L

Setelah evaluasi akurasi model dilakukan, tahap berikutnya adalah membandingkan performa inferensi antara CPU Raspberry Pi 5 dan akselerator Hailo-8L. Pengujian dilakukan menggunakan *test set* yang sama, yaitu 108 gambar. Metrik performa yang dibandingkan meliputi *inference only*, *end-to-end FPS*, waktu pemrosesan per gambar, CPU *peak*, dan RAM *peak*.

Tabel 4. 5. Perbandingan performa CPU Raspberry Pi 5 dan Hailo-8L

Model	Backend	Inference	End-to-End	CPU Peak	RAM Peak
YOLOv8n	CPU	3.92FPS/255.42ms	3.88FPS/257.65ms	88%	3.35GB
YOLOv8n	Hailo-8L	82.99FPS/12.05ms	60.18FPS/16.62ms	92%	2.44GB
YOLOv10n	CPU	4.09FPS/244.33ms	4.06FPS/246.03ms	95%	3.35GB
YOLOv10n	Hailo-8L	67.79FPS/14.75ms	51.41FPS/19.45ms	92%	2.45GB
YOLO11n	CPU	4.16FPS/240.32ms	4.12FPS/242.57ms	75%	3.46GB
YOLO11n	Hailo-8L	60.09FPS/16.64ms	41.03FPS/24.37ms	98%	2.55GB

Berdasarkan Tabel 4.5, penggunaan Hailo-8L memberikan peningkatan kecepatan inferensi yang cukup besar pada seluruh model. Pada YOLOv8n, performa end-to-end meningkat dari 3.88 FPS pada CPU menjadi 60.18 FPS pada Hailo-8L, atau meningkat sekitar 1451.03%. YOLOv10n meningkat dari 4.06 FPS menjadi 51.41 FPS, atau meningkat sekitar 1166.26%. Sementara itu, YOLO11n meningkat dari 4.12 FPS menjadi 41.03 FPS, atau meningkat sekitar 895.87%. Ini menunjukkan bahwa penggunaan Hailo-8L mampu mempercepat proses inferensi model secara signifikan.

Peningkatan performa juga terlihat dari waktu pemrosesan per gambar. Pada CPU, ketiga model membutuhkan waktu sekitar 240 - 257 ms untuk memproses satu gambar secara *end-to-end*. Setelah dijalankan pada Hailo-8L, waktu pemrosesan turun menjadi sekitar 16–24 ms per gambar.

Dari sisi penggunaan RAM, model yang dijalankan pada Hailo-8L menggunakan memori yang lebih rendah dibandingkan *backend* CPU. Pada CPU, penggunaan RAM berada pada rentang 3.35 GB hingga 3.46 GB. Sementara itu, pada Hailo-8L penggunaan RAM berada pada rentang 2.44 GB hingga 2.55 GB. Penurunan ini menunjukkan bahwa penggunaan model .hef pada Hailo-8L lebih ringan dari sisi penggunaan memori dibandingkan model .pt yang dijalankan melalui PyTorch pada CPU.

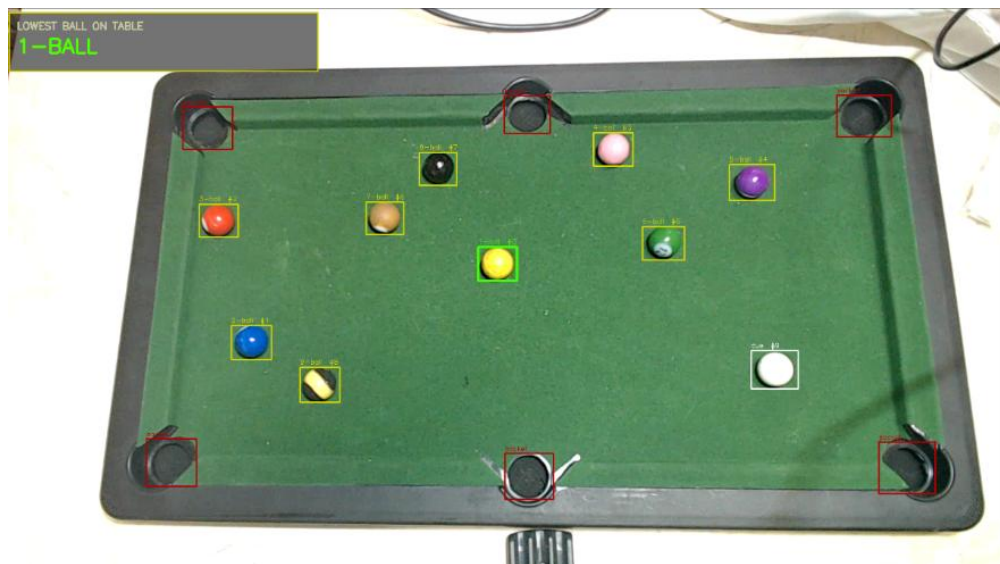
Penggunaan CPU *peak* pada *backend* Hailo-8L masih tetap tinggi. Ini menunjukkan bahwa walaupun proses inferensi dibantu oleh Hailo-8L, CPU Raspberry Pi 5 masih dipakai untuk proses lain, seperti pembacaan gambar, *pre-processing* dan *post-processing*. Dapat disimpulkan bahwa Hailo-8L tidak sepenuhnya menghilangkan beban CPU, tapi memindahkan beban utamanya ke akselerator.

Jika dilihat dari keseimbangan antara akurasi dan performa, YOLOv10n menjadi model yang cukup stabil. YOLOv10n memiliki performa *end-to-end* sebesar 51.41 FPS pada Hailo-8L, dengan mAP@50 sebesar 0.953 of dan mAP@50-95 sebesar 0.639. Walau

YOLOv8n memiliki FPS tertinggi, YOLOv10n memberikan hasil akurasi yang lebih baik setelah proses kompilasi ke format .hef.

4.4. Hasil Implementasi Sistem Deteksi Pelanggaran

Setelah model deteksi objek dikonversi ke format .hef, sistem deteksi pelanggaran 9-ball pool berhasil diimplementasikan pada Raspberry Pi 5 dengan akselerator Hailo-8L. Model yang digunakan pada implementasi akhir adalah YOLOv10n karena memiliki keseimbangan yang baik antara akurasi deteksi dan kecepatan inferensi berdasarkan hasil pengujian sebelumnya. Gambar 4.1 menampilkan hasil deteksi objek pada video, di mana objek permainan seperti *cue ball*, bola bernomor, dan *pocket* ditandai memakai *bounding box* dan label kelas.



Gambar 4. 1. Tampilan hasil deteksi objek

Hasil implementasi menunjukkan bahwa sistem mampu membaca video *input* dan menampilkan objek permainan, seperti *cue ball*, bola yang bernomor, dan *pocket*. Hasil deteksi kemudian digunakan untuk membentuk area seperti area *pocket* dan *rail zone*, yang menjadi pembantu dalam proses deteksi pelanggaran. Gambar 4.2 menampilkan *rail zone* yang dibentuk dari titik tengah *pocket* dan digunakan sebagai area bantu untuk membaca kontak bola dengan sisi meja.



Gambar 4. 2. Tampilan area pembantu pada sistem

Sistem juga mampu menghubungkan posisi dan pergerakan bola dengan logika pelanggaran. Sistem dapat menampilkan status kejadian, mau itu *foul* atau *no foul*, seperti *cue ball scratch*, *no legal ball hit*, *cue ball off field*, dan *no rail contact on shot*. Gambar 4.3 menampilkan contoh *output* ketika sistem berhasil mengidentifikasi salah satu jenis pelanggaran dan menampilkan status foul pada video. Pelanggaran yang ditampilkan adalah *no rail contact on shot* dan *no legal ball hit*. Pada deteksi *no legal hit*, sistem tidak hanya membaca tumpang tindih bounding box, tetapi juga memanfaatkan arah lintasan *cue ball* dan perubahan posisi *object ball* untuk memperkirakan bola pertama yang terkena.



Gambar 4. 3. Tampilan sistem saat mendeteksi pelanggaran

4.5. Hasil Pengujian Deteksi Pelanggaran

Pengujian sistem deteksi pelanggaran dilakukan untuk mengetahui kemampuan sistem dalam menentukan kejadian *foul* dan *non-foul* pada video pengujian. Evaluasi dilakukan menggunakan 50 video, dengan pembagian 10 video untuk setiap kategori kejadian. Kategori yang diuji terdiri dari *no foul*, *no legal hit*, *no rail contact on shot*, *cue ball off field*, dan *cue ball scratch*. Setiap hasil deteksi sistem dibandingkan dengan ground truth yang ditentukan secara manual berdasarkan isi video pengujian, sesuai dengan metode evaluasi yang telah dijelaskan pada Bab III. Tabel 4.6 menunjukkan hasil pengujian sistem deteksi pelanggaran terhadap 50 video menggunakan Hailo accelerator. Tabel 4.7 menunjukkan hasil pengujian menggunakan model .pt.

Tabel 4. 6. Hasil pengujian deteksi pelanggaran menggunakan model .hef

Kategori Pengujian	Jumlah Video	Deteksi Benar	Deteksi Salah	Akurasi
No foul	10	8	2	80%
No legal hit	10	8	2	80%
No rail contact on shot	10	9	1	90%
Cue ball off field	10	10	0	100%
Cue ball scratch	10	7	3	70%
Total	50	42	8	84%

Tabel 4. 7. Hasil pengujian deteksi menggunakan model .pt

Kategori Pengujian	Jumlah Video	Deteksi Benar	Deteksi Salah	Akurasi
No foul	10	9	1	90%
No legal hit	10	8	2	80%
No rail contact on shot	10	10	0	100%
Cue ball off field	10	10	0	100%
Cue ball scratch	10	9	1	90%
Total	50	46	4	92%

Berdasarkan Tabel 4.6 dan Tabel 4.7, sistem menghasilkan akurasi deteksi pelanggaran sebesar 84% ketika menggunakan model .hef pada Hailo-8L, sedangkan pengujian menggunakan model .pt menghasilkan akurasi sebesar 92%. Hasil ini menunjukkan bahwa model .pt memberikan hasil deteksi pelanggaran yang lebih tinggi dibandingkan model .hef, dengan selisih 8 poin persentase. Perbedaan ini dapat terjadi karena model .pt belum melalui proses kompilasi, optimasi, dan kuantisasi, sehingga hasil deteksi objek masih berasal dari bobot asli YOLO.

Peningkatan terbesar terlihat pada kategori *cue ball scratch*, yaitu dari 70% pada model .hef menjadi 90% pada model .pt. Hal ini menunjukkan bahwa deteksi *scratch* cukup sensitif terhadap kestabilan deteksi *cue ball* dan *pocket*. Jika *bounding box cue ball* atau *pocket* kurang stabil pada beberapa *frame*, sistem dapat gagal membaca kondisi *cue ball* yang berada di area *pocket*. Selain itu, kategori *no rail contact on shot* juga meningkat dari 90% menjadi 100%, yang menunjukkan bahwa deteksi posisi bola terhadap rail lebih konsisten pada model .pt.

Kategori *no legal hit* memperoleh hasil yang sama pada kedua pengujian, yaitu 80%. Hal ini menunjukkan bahwa kesalahan pada kategori tersebut tidak hanya dipengaruhi oleh format model, tetapi juga oleh kompleksitas logika kontak pertama. Deteksi *no legal hit* membutuhkan pembacaan hubungan antara *cue ball* dan *object ball*, seperti tumpang tindih *bounding box*, lintasan *cue ball*, perpindahan bola, dan bola pertama yang bergerak setelah pukulan.

Secara keseluruhan, model .pt menghasilkan akurasi deteksi pelanggaran yang lebih tinggi, sedangkan model .hef tetap lebih sesuai untuk implementasi pada edge device karena memiliki performa inferensi yang jauh lebih cepat. Dengan demikian, hasil pengujian menunjukkan adanya trade-off antara akurasi deteksi pelanggaran dan efisiensi komputasi setelah model dikonversi ke format .hef.

BAB V

KESIMPULAN DAN SARAN

5.1. Kesimpulan

Berdasarkan hasil penelitian yang telah dilakukan, sistem deteksi pelanggaran 9-ball pool berbasis visi komputer berhasil dikembangkan dan diimplementasikan pada Raspberry Pi 5 dengan akselerator Hailo-8L. Sistem mampu mendeteksi objek permainan seperti cue ball, bola bernomor, dan pocket, kemudian menggunakan informasi tersebut untuk menentukan kejadian pelanggaran berdasarkan aturan yang sudah dirancang.

Model YOLOv10n menjadi model yang paling seimbang untuk digunakan dalam implementasi akhir. Pada pengujian CPU Raspberry Pi 5, YOLOv10n mencapai mAP@50 sebesar 0.976 dan mAP@50-95 sebesar 0.738. Setelah dikonversi ke format .hef dan dijalankan pada Hailo-8L, model tetap memperoleh mAP@50 yang cukup tinggi, yaitu 0.953, walau terjadi penurunan pada mAP@50-95 akibat proses optimasi.

Penggunaan Hailo-8L terbukti meningkatkan performa inferensi secara signifikan. Pada YOLOv10n, performa *end-to-end* meningkat dari 4,06 FPS pada CPU menjadi 51,41 FPS pada Hailo-8L. Hal ini menunjukkan bahwa akselerator Hailo-8L mampu membantu Raspberry Pi 5 menjalankan model deteksi objek dengan lebih cepat dan efisien.

Pada pengujian deteksi pelanggaran, sistem berhasil mendeteksi 42 dari 50 video dengan benar, sehingga mendapatkan akurasi keseluruhan sebesar 84%. Hasil terbaik diperoleh pada kategori cue ball off field dengan akurasi 100%, sedangkan hasil terendah terdapat pada *cue ball scratch* dengan akurasi 70%. Secara keseluruhan, sistem sudah mampu menjalankan fungsi utama sesuai ruang lingkup penelitian, yaitu mendeteksi empat jenis pelanggaran 9-ball berdasarkan hasil deteksi objek dan logika aturan berbasis koordinat.

5.2. Saran

Pengembangan selanjutnya dapat dilakukan dengan menambah jumlah dan variasi data pengujian, khususnya untuk kondisi *cue ball scratch* dan *no legal hit*. Variasi perspektif kamera, pencahayaan, dan *ball in motion* perlu diperhatikan karena kondisi tersebut dapat memengaruhi kestabilan deteksi objek, terutama ketika bola bergerak cepat, tertutup objek lain, atau terkena bayangan dan refleksi cahaya. Sistem juga dapat dikembangkan dengan metode pelacakan objek yang lebih kuat agar pergerakan bola dari frame ke frame dapat terbaca dengan lebih konsisten. Ini dapat dikembangkan dengan menggunakan kamera dengan *shutter speed* yang lebih baik, resolusi yang lebih tinggi, dan pencahayaan yang lebih stabil. Pastikan juga penempatan kamera dibuat lebih stabil dan mendekati sudut top-down agar seluruh area meja dan objek di atasnya dapat terlihat dengan jelas.

Selain itu, penggunaan *bounding box* untuk keperluan analisis kontak masih memiliki keterbatasan, Tumpang tindih *bounding box* tidak selalu menunjukkan kontak fisik antarbola, sedangkan kontak yang terjadi sangat cepat dawat terlewat. Oleh karena itu,

penelitian berikutnya dapat menambahkan pendekatan berbasis kecepatan, perubahan posisi bola, atau metode lainnya untuk meningkatkan akurasi pembacaan kontak pertama dan kontak dengan *rail*.

Dataset juga perlu diperluas menggunakan meja biliar standar dan kondisi pertandingan nyata. Ini penting karena sebagian data primer dalam penelitian ini menggunakan meja biliar sederhana, sehingga terdapat perbedaan visual dengan meja pertandingan sebenarnya, seperti ukuran meja, bentuk *pocket*, tekstur permukaan, dan, warna meja. Selain itu, penelitian berikutnya dapat memperluas jenis pelanggaran yang dideteksi, seperti *jump en masse*, *three consecutive fouls*, atau pelanggaran lain yang masih memungkinkan dianalisis melalui kamera.

DAFTAR PUSTAKA

- Alqahtani, D. K., Cheema, A., & Toosi, A. N. (2024). *Benchmarking Deep Learning Models for Object Detection on Edge Computing Devices*. <http://arxiv.org/abs/2409.16808>
- Alqahtani, D. K., Cheema, M. A., Rodriguez, M. A., & Toosi, A. N. (2026). *A Comprehensive Evaluation of Deep Learning Object Detection Models on Heterogeneous Edge Devices*. <http://arxiv.org/abs/2409.16808>
- Bento, J., Paixão, T., & Alvarez, A. B. (2025). Performance Evaluation of YOLOv8, YOLOv9, YOLOv10, and YOLOv11 for Stamp Detection in Scanned Documents. *Applied Sciences (Switzerland)*, 15(6). <https://doi.org/10.3390/app15063154>
- Billiards Congress of America. (2026, June 27). *Official BCA 9 Ball Rules*. <https://www.billiards.com/blogs/articles/official-bca-9-ball-rules>
- Budiharto, W., Gunawan, A. A. S., Suroso, J. S., Chowanda, A., Patrik, A., & Utama, G. (2018). Fast Object Detection for Quadcopter Drone Using Deep Learning. *2018 3rd International Conference on Computer and Communication Systems (ICCCS)*, 192–195. <https://doi.org/10.1109/CCOMS.2018.8463284>
- Gholinavaz, S., saeedi, N., & Gharehveran, S. S. (2025). Robustness analysis of YOLO and faster R-CNN for object detection in realistic weather scenarios with noise augmentation. *Scientific Reports*, 15(1). <https://doi.org/10.1038/s41598-025-28737-5>
- Hailo AI. (2026, June 23). *Hailo Getting Started Guide*. https://hailo.ai/developer-zone/documentation/hailo-sw-suite-2025-10-for-hailo-8-8l/?sp_referrer=suite/suite_overview.html#where-to-begin
- Inoue, Y., Sultana, R., Nishino, Y., & Shimizu, I. (2024). Enhancing Daytime Train Image Detection with YOLOv8 through Data Augmentation Techniques. *2024 the 7th Artificial Intelligence and Cloud Computing Conference (AICCC)*

(AICCC 2024), December 1416, 2024, Tokyo, Japan, 1.
<https://doi.org/10.1145/3719384>

- Minaee, S., Boykov, Y., Porikli, F., Plaza, A., Kehtarnavaz, N., & Terzopoulos, D. (2022). Image Segmentation Using Deep Learning: A Survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7), 3523–3542. <https://doi.org/10.1109/TPAMI.2021.3059968>
- Minott, D., Siddiqui, S., & Haddad, R. J. (2025). Benchmarking Edge AI Platforms: Performance Analysis of NVIDIA Jetson and Raspberry Pi 5 with Coral TPU. *Conference Proceedings - IEEE SOUTHEASTCON*, 1384–1389. <https://doi.org/10.1109/SoutheastCon56624.2025.10971592>
- Mohamed, A. H., Refaat, M., & Hemeida, A. M. (2022). Image classification based deep learning: A Review. *Aswan University Journal of Science and Technology*, 2(1). <https://journals.aswu.edu.eg/stjournal>
- Padilla, R., Netto, S. L., & da Silva, E. A. B. (2020). A Survey on Performance Metrics for Object-Detection Algorithms. *2020 International Conference on Systems, Signals and Image Processing (IWSSIP)*, 237–242. <https://doi.org/10.1109/IWSSIP48289.2020.9145130>
- Park, S., Kim, J., Wang, S., & Kim, J. (2025). Effectiveness of Image Augmentation Techniques on Non-Protective Personal Equipment Detection Using YOLOv8. *Applied Sciences (Switzerland)*, 15(5). <https://doi.org/10.3390/app15052631>
- Richard Szeliski. (2021). *Computer Vision: Algorithms and Applications 2nd Edition*. Springer.
- Spitz, J., Wagemans, J., Memmert, D., Williams, A. M., & Helsen, W. F. (2021). Video assistant referees (VAR): The impact of technology on decision making in association football referees. *Journal of Sports Sciences*, 39(2), 147–153. <https://doi.org/10.1080/02640414.2020.1809163>
- Terven, J., Córdova-Esparza, D. M., & Romero-González, J. A. (2023). A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS. In *Machine Learning and Knowledge Extraction* (Vol. 5, Number 4, pp. 1680–1716). Multidisciplinary Digital Publishing Institute (MDPI). <https://doi.org/10.3390/make5040083>
- Ultralytics. (2026, June 23). *YOLO Object Detection & Segmentation*. <https://docs.ultralytics.com/models>
- Wang, Z., Wang, P., Liu, K., Wang, P., Fu, Y., Lu, C.-T., Aggarwal, C. C., Pei, J., & Zhou, Y. (2025). *A Comprehensive Survey on Data Augmentation*. <https://doi.org/10.48550/arXiv.2405.09591>
- Xu, Y., Liu, X., Cao, X., Huang, C., Liu, E., Qian, S., Liu, X., Wu, Y., Dong, F., Qiu, C. W., Qiu, J., Hua, K., Su, W., Wu, J., Xu, H., Han, Y., Fu, C., Yin, Z.,

Liu, M., ... Zhang, J. (2021). Artificial intelligence: A powerful paradigm for scientific research. In *Innovation* (Vol. 2, Number 4). Cell Press. <https://doi.org/10.1016/j.xinn.2021.100179>

Yu, J., Huang, Y., & He, Y. (2018). Snooker video event detection using multimodal features. *MMSports 2018 - Proceedings of the 1st International Workshop on Multimedia Content Analysis in Sports, Co-Located with MM 2018*, 3–10. <https://doi.org/10.1145/3265845.3265847>

Zheng, A., & Yan, W. Q. (2025). Attention-Pool: 9-Ball Game Video Analytics with Object Attention and Temporal Context Gated Attention. *Computers*, 14(9). <https://doi.org/10.3390/computers14090352>

LAMPIRAN

LAMPIRAN 1 (Main_Camera_Code.py)

```
import os
import time
import threading
import dataclasses
from datetime import datetime
from typing import Optional, List
from collections import deque, Counter, defaultdict

import cv2
import tkinter as tk
from tkinter import filedialog, messagebox
from PIL import Image, ImageTk
from ultralytics import YOLO
import numpy as np

os.environ['KMP_DUPLICATE_LIB_OK'] = 'TRUE'
_SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
MODELS_ROOT = os.path.join(_SCRIPT_DIR, "..", "Trained_Models")

def _model_pt_path(name):
    weights_dir = os.path.join(MODELS_ROOT, name, "weights")
    conventional = os.path.join(weights_dir, f"{name}.pt")
    fallback = None
    preferred = None
    try:
        for f in sorted(os.listdir(weights_dir)):
            stem, ext = os.path.splitext(f)
            if ext.lower() != ".pt":
                continue
            if stem.lower() == name.lower():
                return os.path.join(weights_dir, f)
            if stem.lower() in ("best", "last") and preferred is None:
                preferred = os.path.join(weights_dir, f)
            if fallback is None:
                fallback = os.path.join(weights_dir, f)
    except OSError:
        pass
    return preferred or fallback or conventional

DEFAULT_MODEL_NAME = "YOLO11n"
DEFAULT_MODEL_PATH = _model_pt_path(DEFAULT_MODEL_NAME)

CLASS_NAMES = {
    0: "Ball-1", 1: "Ball-2", 2: "Ball-3", 3: "Ball-4", 4: "Ball-5",
```

```

5: "Ball-6", 6: "Ball-7", 7: "Ball-8", 8: "Ball-9",
9: "cue", 10: "cue_stick", 11: "pocket", 12: "rack",
}
BALL_CLASSES = set(range(9))
CUE_CLASS = 9
POCKET_CLASS = 11
RACK_CLASS = 12

BALL_COLOR_ID = {
    0: "Kuning", 1: "Biru", 2: "Merah",
    3: "Pink", 4: "Ungu", 5: "Hijau",
    6: "Coklat", 7: "Hitam", 8: "Kuning-Hitam",
}
BALL_BGR = {
    0: (0, 255, 255),
    1: (255, 0, 0),
    2: (0, 0, 255),
    3: (180, 105, 255),
    4: (211, 0, 148),
    5: (0, 200, 0),
    6: (19, 69, 139),
    7: (0, 0, 0),
    8: ((0, 255, 255), (0, 0, 0)),
}

CONF_THRESHOLD = 0.35

SHOW_CONTACT_DEBUG = True

SHOT_START_VEL = 8.0
SHOT_END_VEL = 5.0

SHOT_END_SECS = 1.0
SCRATCH_REST_SECS = 1.0
CUE_MISSING_SECS = 12.0
CUE_OFF_TABLE_SECS = 1.5
POCKET_DISAPPEAR_SECS = 0.5
FOUL_BANNER_SECS = 3.0

MOVEMENT_CONFIRM_FRAMES = 12
MOVEMENT_THRESHOLD_PX = 8
CONTACT_BBOX_PAD = 4

CLASS_VOTE_WINDOW = 50

RAIL_MARGIN_PX = 40
BOTTOM_RAIL_PAD_PX = 2
POST_SHOT_GRACE_FRAMES = 45

TRAJECTORY_PROXIMITY_PX = 35

```

```

TRAJECTORY_LOOKAHEAD_PX = 400
LINE_OF_SIGHT_REWIND_FRAMES = 4
LINE_OF_SIGHT_BLOCK_RADIUS_PX =
TRAJECTORY_PROXIMITY_PX
POCKET_REPLACEMENT_OVERLAP = 0.45

def bbox_center(b):
    return ((b[0] + b[2]) * 0.5, (b[1] + b[3]) * 0.5)

def pt_dist(a, b):
    return ((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2) ** 0.5

def boxes_overlap(b1, b2, pad=0):
    a = (b1[0] - pad, b1[1] - pad, b1[2] + pad, b1[3] + pad)
    return not (a[2] < b2[0] or b2[2] < a[0] or
                a[3] < b2[1] or b2[3] < a[1])

def bbox_overlap_ratio(b1, b2):
    x1 = max(b1[0], b2[0])
    y1 = max(b1[1], b2[1])
    x2 = min(b1[2], b2[2])
    y2 = min(b1[3], b2[3])
    inter = max(0, x2 - x1) * max(0, y2 - y1)
    area1 = max(1, (b1[2] - b1[0]) * (b1[3] - b1[1]))
    area2 = max(1, (b2[2] - b2[0]) * (b2[3] - b2[1]))
    return inter / min(area1, area2)

def bbox_inside(inner, outer):
    return (inner[0] >= outer[0] and inner[1] >= outer[1] and
            inner[2] <= outer[2] and inner[3] <= outer[3])

def pt_to_line_segment_dist(p, v, w):
    px, py = p
    vx, vy = v
    wx, wy = w

    l2 = (wx - vx)**2 + (wy - vy)**2
    if l2 == 0.0:
        return pt_dist(p, v)

    t = max(0, min(1, ((px - vx) * (wx - vx) + (py - vy) * (wy - vy)) / l2))

    proj_x = vx + t * (wx - vx)
    proj_y = vy + t * (wy - vy)

    return pt_dist(p, (proj_x, proj_y))

```

```

@dataclasses.dataclass
class Track:
    track_id: int
    bbox: list
    stable_cls: int
    center: tuple

@dataclasses.dataclass
class PendingContact:
    contacted_track_id: int
    contacted_class: int
    pos_at_contact: tuple
    created_frame: int
    shot_target_class: Optional[int]

class FoulDetectorApp:

    def __init__(self, root: tk.Tk):
        self.root = root
        self.root.title("9-Ball Foul Detector — Camera Recording")
        self.root.geometry("1280x860")
        self.root.configure(bg="white")

        self.model: Optional[YOLO] = None
        self.model_path: str = DEFAULT_MODEL_PATH
        self.video_path: Optional[str] = None
        self.output_dir: Optional[str] = None

        self.cap: Optional[cv2.VideoCapture] = None
        self.video_writer: Optional[cv2.VideoWriter] = None
        self.foul_log = None
        self.debug_log = None

        self.is_playing = False
        self.is_paused = False
        self.fps = 30.0
        self.frame_idx = 0
        self._foul_count = 0

        self.camera_index: Optional[int] = None
        self.available_cams: list = []
        self.is_recording: bool = False
        self.recorded_path: Optional[str] = None
        self._record_thread = None
        self._rec_start_time: float = 0.0
        self.cap_width = 1280

```

```

self.cap_height = 720
self.cap_fps    = 60
self._frame_stride: int = 1
self._src_frame_no: int = 0

self.class_history: defaultdict = defaultdict(
    lambda: deque(maxlen=CLASS_VOTE_WINDOW))
self.ball_positions: defaultdict = defaultdict(
    lambda: deque(maxlen=90))
self.ball_position_frames: defaultdict = defaultdict(
    lambda: deque(maxlen=90))

self.prev_cue_center: Optional[tuple] = None
self._last_cue_bbox: Optional[tuple] = None
self._last_pocket_bboxes: list = []
self.cue_velocity: float = 0.0
self.shot_in_progress: bool = False
self.lowest_at_shot_start: Optional[int] = None
self.is_break_shot: bool = False
self.rack_frames_since_seen: int = 999
self.checked_contact_tids: set = set()
self._low_vel_run: int = 0
self.pending_contacts: List[PendingContact] = []

self.ball_start_info: dict = {}
self.balls_left_rail_this_shot: set = set()
self.contact_occurred_this_shot: bool = False
self.post_contact_rail_hit: bool = False
self.post_contact_pocketed: bool = False
self.active_ball_ids_at_contact: dict = {}

self.event_log: deque = deque(maxlen=6)
self.balls_hit_rail_this_shot: set = set()
self.potted_balls: set = set()
self.stable_pockets: list = []
self.stable_pocket_bboxes: list = []
self.pocket_candidates: dict = {}

self.cue_missing_frames: int = 0
self.cue_in_pocket_frames: int = 0
self._scratch_fired: bool = False
self._missing_fired: bool = False
self._off_table_fired: bool = False

self.cue_was_near_pocket: bool = False
self.cue_near_pocket_streak: int = 0
self.cue_gone_after_pocket_frames: int = 0
self._pocket_disappear_fired: bool = False
self._cue_pocket_replace_fired: bool = False
self.post_shot_eval_frame: Optional[int] = None

```



```

self._no_legal_ball_hit_fired: bool = False

self.table_bounds: Optional[tuple] = None

self.rail_polygon: Optional[np.ndarray] = None
self.rail_locked: bool = False
self.pocket_positions: dict = {}

self.foul_banner_text: str = ""
self.foul_banner_until_frame: int = 0

self._build_ui()
self._populate_models()
self.refresh_cameras()

def _build_ui(self):
    bar = tk.Frame(self.root, bg="white", pady=7)
    bar.pack(side=tk.TOP, fill=tk.X)

    tk.Label(bar, text="Camera:", bg="white").pack(side=tk.LEFT, padx=(6,
2))
    self.cam_var = tk.StringVar(value="Detecting...")
    self.cam_menu = tk.OptionMenu(bar, self.cam_var, "Detecting...")
    self.cam_menu.config(state=tk.DISABLED)
    self.cam_menu.pack(side=tk.LEFT, padx=4)
    tk.Button(bar, text="Refresh", command=self.refresh_cameras
    ).pack(side=tk.LEFT, padx=2)
    tk.Button(bar, text="Output Folder",
        command=self.select_output).pack(side=tk.LEFT, padx=5)
    tk.Label(bar, text="Model:", bg="white").pack(side=tk.LEFT, padx=(8, 2))
    self.model_var = tk.StringVar(value="Scanning...")
    self.model_menu = tk.OptionMenu(bar, self.model_var, "Scanning...")
    self.model_menu.config(state=tk.DISABLED)
    self.model_menu.pack(side=tk.LEFT, padx=4)
    tk.Button(bar, text="Browse...",
        command=self.load_model_dialog).pack(side=tk.LEFT, padx=2)

    tk.Frame(bar, bg="#cccccc", width=2, height=36).pack(
        side=tk.LEFT, padx=14, fill=tk.Y)

    self.upload_btn = tk.Button(bar, text="Upload Video",
        command=self.upload_video,
        state=tk.DISABLED)
    self.upload_btn.pack(side=tk.LEFT, padx=4)
    self.rec_btn = tk.Button(bar, text="Start Recording",
        command=self.start_recording,
        state=tk.DISABLED)
    self.rec_btn.pack(side=tk.LEFT, padx=4)
    self.stoprec_btn = tk.Button(bar, text="Stop Recording",

```

```

        command=self.stop_recording,
        state=tk.DISABLED)
self.stoprec_btn.pack(side=tk.LEFT, padx=4)
self.pause_btn = tk.Button(bar, text="Pause",
        command=self.toggle_pause,
        state=tk.DISABLED)
self.pause_btn.pack(side=tk.LEFT, padx=4)

self.status_lbl = tk.Label(bar, text="Loading model...", bg="white")
self.status_lbl.pack(side=tk.RIGHT, padx=14)

self.video_lbl = tk.Label(self.root, bg="black")
self.video_lbl.pack(fill=tk.BOTH, expand=True, padx=8, pady=6)

log_frame = tk.Frame(self.root, bg="#12121f", pady=4)
log_frame.pack(side=tk.BOTTOM, fill=tk.X, padx=8)
tk.Label(log_frame, text="Foul Log", fg="white", bg="#12121f",
        font=("Segoe UI", 11, "bold")).pack(anchor=tk.W)
self.foul_listbox = tk.Listbox(
    log_frame, height=5,
    bg="#0d0d1f", fg="#ff7070",
    font=("Consolas", 10),
    selectbackground="#2b2b4a",
    highlightthickness=0, borderwidth=0,
)
self.foul_listbox.pack(fill=tk.X, pady=2)

def _load_model_async(self, path: str):
    def _do():
        try:
            self.model = None

            m = YOLO(path)
            self.model = m
            self.model_path = path
            self.root.after(0, lambda: self.status_lbl.config(
                text=f"{os.path.basename(path)}", fg="black"))
            self.root.after(0, self._check_ready)
        except Exception as exc:
            msg = str(exc)
            self.root.after(0, lambda: self.status_lbl.config(
                text=f"Model error: {msg}", fg="#ff7070"))
        threading.Thread(target=_do, daemon=True).start()

def load_model_dialog(self):
    if self.is_playing:
        messagebox.showwarning(
            "Busy",
            "Stop the current job before loading a different model.")

```

```

        return
    path = filedialog.askopenfilename(
        title="Select YOLO weights (.pt)",
        filetypes=[("Ultralytics YOLO weights", "*.pt"), ("All files", "*.*")],
    )
    if path:
        self.model_var.set(os.path.basename(path))
        self.status_lbl.config(text="Loading...", fg="black")
        self._load_model_async(path)

def _discover_models(self):
    names = []
    try:
        for entry in sorted(os.listdir(MODELS_ROOT)):
            if os.path.isfile(_model_pt_path(entry)):
                names.append(entry)
    except OSError:
        pass
    return names

def _populate_models(self):
    names = self._discover_models()
    menu = self.model_menu["menu"]
    menu.delete(0, tk.END)

    if not names:
        self.model_var.set("No models found")
        self.model_menu.config(state=tk.DISABLED)
        self.status_lbl.config(
            text=f"No models in {MODELS_ROOT} — use Browse...",
            fg="#ff7070")
        return

    for name in names:
        menu.add_command(
            label=name,
            command=lambda n=name: self._select_model(n))
    self.model_menu.config(state=tk.NORMAL)

    self.model_var.set("Select a model")
    self.status_lbl.config(text="Select a model to begin", fg="black")

def _select_model(self, name):
    if self.is_playing:
        messagebox.showwarning(
            "Busy",
            "Stop the current job before loading a different model.")
        return
    self.model_var.set(name)
    self.status_lbl.config(text="Loading...", fg="black")

```

```

        self._load_model_async(_model_pt_path(name))

    def refresh_cameras(self):
        self.status_lbl.config(text="Scanning for cameras...", fg="black")
        threading.Thread(target=self._detect_cameras_worker,
            daemon=True).start()

    def _detect_cameras_worker(self):
        found = []
        for idx in range(6):
            cap = cv2.VideoCapture(idx, cv2.CAP_DSHOW)
            if cap is not None and cap.isOpened():
                ok, _ = cap.read()
                if ok:
                    found.append(idx)
            if cap is not None:
                cap.release()
        self.root.after(0, lambda: self._populate_cameras(found))

    def _populate_cameras(self, found):
        self.available_cams = found
        menu = self.cam_menu["menu"]
        menu.delete(0, tk.END)
        if not found:
            self.cam_var.set("No camera found")
            self.cam_menu.config(state=tk.DISABLED)
            self.camera_index = None
            self.status_lbl.config(
                text="No camera detected — check the connection", fg="#ff7070")
            self._check_ready()
            return
        for idx in found:
            label = f"Camera {idx}"
            menu.add_command(
                label=label,
                command=lambda i=idx, l=label: self._select_camera(i, l))
        self._select_camera(found[0], f"Camera {found[0]}")
        self.cam_menu.config(state=tk.NORMAL)
        self.status_lbl.config(text=f"{len(found)} camera(s) found", fg="black")

    def _select_camera(self, idx, label):
        self.camera_index = idx
        self.cam_var.set(label)
        self._check_ready()

    def select_output(self):
        path = filedialog.askdirectory(title="Select output folder")
        if path:
            self.output_dir = path
            self._check_ready()

```

```

def _check_ready(self):
    ready = (self.camera_index is not None
              and self.output_dir and self.model is not None
              and not self.is_recording)
    if hasattr(self, "rec_btn"):
        self.rec_btn.config(state=tk.NORMAL if ready else tk.DISABLED)
    upload_ready = bool(self.output_dir and self.model is not None
                        and not self.is_recording and not self.is_playing)
    if hasattr(self, "upload_btn"):
        self.upload_btn.config(
            state=tk.NORMAL if upload_ready else tk.DISABLED)

def upload_video(self):
    if self.is_recording or self.is_playing:
        messagebox.showwarning(
            "Busy", "Stop the current job before uploading a video.")
        return
    if not (self.output_dir and self.model):
        messagebox.showerror(
            "Not ready",
            "Select an output folder and load a model first.")
        return
    path = filedialog.askopenfilename(
        title="Select a video file",
        filetypes=[
            ("Video files",
             "*.mp4 *.avi *.mov *.mkv *.m4v *.wmv *.flv *.mpg *.mpeg"),
            ("All files", "*.*"),
        ],
    )
    if not path:
        return
    self.recorded_path = path
    self.cam_menu.config(state=tk.DISABLED)
    self._begin_detection(path)

def start_recording(self):
    if self.is_recording:
        return
    if not (self.camera_index is not None and self.output_dir and self.model):
        messagebox.showerror(
            "Not ready",
            "Select a camera, an output folder, and load a model first.")
        return

    cap = cv2.VideoCapture(self.camera_index, cv2.CAP_DSHOW)
    if not cap.isOpened():
        messagebox.showerror(

```

```

        "Error", f"Could not open camera {self.camera_index}.")
    return
cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*"MJPG"))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, self.cap_width)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, self.cap_height)
cap.set(cv2.CAP_PROP_FPS, self.cap_fps)

W = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)) or self.cap_width
H = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT)) or self.cap_height
fps = cap.get(cv2.CAP_PROP_FPS)
if not fps or fps < 1:
    fps = float(self.cap_fps)
self.cap = cap
self.fps = fps

ts = datetime.now().strftime("%Y%m%d_%H%M%S")
self.recorded_path = os.path.join(self.output_dir, f"recording_{ts}.mp4")
writer, codec = self._make_h264_writer(self.recorded_path, fps, (W, H))
if writer is None:
    cap.release()
    self.cap = None
    messagebox.showerror("Error", "Could not open a video writer.")
    return
self.video_writer = writer

self.is_recording = True
self._rec_start_time = time.time()
self._rec_frame_count = 0
self.rec_btn.config(state=tk.DISABLED)
self.stoprec_btn.config(state=tk.NORMAL)
self.cam_menu.config(state=tk.DISABLED)
self.status_lbl.config(
    text=f"REC {W}x{H} @ {fps:.0f}fps ({codec})", fg="#ff7070")

self._record_thread = threading.Thread(
    target=self._record_loop, daemon=True)
self._record_thread.start()

def _record_loop(self):
    try:
        while self.is_recording and self.cap and self.cap.isOpened():
            ret, frame = self.cap.read()
            if not ret:
                break
            if self.video_writer:
                self.video_writer.write(frame)
                self._rec_frame_count += 1
            prev = frame.copy()
            elapsed = time.time() - self._rec_start_time
            cv2.circle(prev, (32, 32), 12, (0, 0, 255), -1)

```

```

        cv2.putText(
            prev, f"REC {int(elapsed // 60):02d}:{int(elapsed % 60):02d}",
            (54, 42), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 0, 255), 2)
        self._display_frame(prev)
    finally:
        if self.cap:
            self.cap.release()
            self.cap = None
        if self.video_writer:
            self.video_writer.release()
            self.video_writer = None

def stop_recording(self):
    if not self.is_recording:
        return
    self.is_recording = False
    self._rec_stop_time = time.time()
    self.stoprec_btn.config(state=tk.DISABLED)
    self.status_lbl.config(text="Finishing recording...", fg="black")
    threading.Thread(target=self._finish_and_detect, daemon=True).start()

def _finish_and_detect(self):
    rt = self._record_thread
    if rt is not None and rt.is_alive():
        rt.join(timeout=5.0)
    path = self.recorded_path
    if not path or not os.path.exists(path):
        self.root.after(0, lambda: self.status_lbl.config(
            text="Recording file missing — nothing to detect", fg="#ff7070"))
        self.root.after(0, self._check_ready)
    return

    elapsed = max(1e-6, getattr(self, "_rec_stop_time", time.time())
        - self._rec_start_time)
    frames = getattr(self, "_rec_frame_count", 0)
    real_fps = frames / elapsed if frames > 0 else None
    if real_fps is not None and not (1.0 <= real_fps <= 1000.0):
        real_fps = None
    self.root.after(0, lambda: self._begin_detection(path,
        fps_override=real_fps))

def _make_h264_writer(self, path, fps, size):
    for codec in ("avc1", "H264", "X264"):
        w = cv2.VideoWriter(path, cv2.VideoWriter_fourcc(*codec), fps, size)
        if w.isOpened():
            return w, codec
        w.release()
    w = cv2.VideoWriter(path, cv2.VideoWriter_fourcc(*"mp4v"), fps, size)
    if w.isOpened():
        return w, "mp4v (H264 unavailable)"

```

```

return None, None

def _begin_detection(self, video_path, fps_override=None):
    self.video_path = video_path
    self.cap = cv2.VideoCapture(video_path)
    if not self.cap.isOpened():
        messagebox.showerror("Error", "Could not open the recorded video.")
        self._check_ready()
        return

    if fps_override and 1.0 <= fps_override <= 1000.0:
        src_fps = fps_override
    else:
        src_fps = self.cap.get(cv2.CAP_PROP_FPS) or 30.0
    if not src_fps or src_fps < 1 or src_fps > 1000:
        src_fps = 30.0
    W = int(self.cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    H = int(self.cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

    self._frame_stride = max(1, int(round(src_fps / 30.0)))
    self._src_frame_no = 0
    self._src_fps = src_fps
    self._last_annotated = None
    self.fps = src_fps / self._frame_stride

    ts = datetime.now().strftime("%Y%m%d_%H%M%S")
    writer, _ = self._make_h264_writer(
        os.path.join(self.output_dir, f"annotated_{ts}.mp4"),
        self._src_fps, (W, H))
    self.video_writer = writer
    self.foul_log = open(
        os.path.join(self.output_dir, f"foul_log_{ts}.txt"),
        "w", encoding="utf-8",
    )
    self.foul_log.write(
        f"Foul Log — {ts}\nSource: {self.video_path}\n\n")

    self.debug_log = open(
        os.path.join(self.output_dir, f"debug_positions_{ts}.txt"),
        "w", encoding="utf-8",
    )
    self.debug_log.write(
        f"Ball Position Debug Log — {ts}\nSource: {self.video_path}\n\n")

    self._reset_state()

    self.is_playing = True
    self.is_paused = False
    self.rec_btn.config(state=tk.DISABLED)
    self.stoprec_btn.config(state=tk.DISABLED)

```



```

self.upload_btn.config(state=tk.DISABLED)
self.pause_btn.config(state=tk.NORMAL, text="Pause")
self.status_lbl.config(
    text=f"Detecting on recording (stride {self._frame_stride})...",
    fg="black")

self._worker = threading.Thread(target=self._process_loop, daemon=True)
self._worker.start()

def _reset_state(self):
    self.frame_idx = 0
    self._foul_count = 0
    self.class_history = defaultdict(
        lambda: deque(maxlen=CLASS_VOTE_WINDOW))
    self.ball_positions = defaultdict(lambda: deque(maxlen=90))
    self.ball_position_frames = defaultdict(lambda: deque(maxlen=90))
    self.prev_cue_center = None
    self._last_cue_bbox = None
    self._last_pocket_bboxes = []
    self.cue_velocity = 0.0
    self.shot_in_progress = False
    self.lowest_at_shot_start = None
    self.is_break_shot = False
    self.rack_frames_since_seen = 999
    self.checked_contact_tids = set()
    self.balls_near_pocket = {}
    self.disappeared_near_pocket_frames = defaultdict(int)
    self._low_vel_run = 0
    self.pending_contacts = []
    self.stable_pockets: list = []
    self.stable_pocket_bboxes: list = []
    self.pocket_candidates: dict = {}
    self.balls_moved_this_shot: dict = {}
    self.trajectory_candidates: set = set()
    self.first_trajectory_hit_fired: bool = False
    self._no_legal_ball_hit_fired = False

    self.ball_start_info = {}
    self.balls_left_rail_this_shot.clear()
    self.contact_occurred_this_shot = False
    self.post_contact_rail_hit = False
    self.post_contact_pocketed = False
    self.active_ball_ids_at_contact = {}
    self.event_log.clear()
    self.balls_hit_rail_this_shot.clear()
    self.potted_balls.clear()

    self.cue_missing_frames = 0
    self.cue_in_pocket_frames = 0
    self._scratch_fired = False

```

```

self._missing_fired      = False
self._off_table_fired    = False
self.cue_was_near_pocket = False
self.cue_near_pocket_streak = 0
self.cue_gone_after_pocket_frames = 0
self._pocket_disappear_fired = False
self._cue_pocket_replace_fired = False
self.post_shot_eval_frame = None
self._last_annotated      = None
self.table_bounds         = None
self.rail_polygon         = None
self.rail_locked          = False
self.pocket_positions     = {}
self.foul_banner_text     = ""
self.foul_banner_until_frame = 0
self.foul_listbox.delete(0, tk.END)

def toggle_pause(self):
    self.is_paused = not self.is_paused
    self.pause_btn.config(
        text="Resume" if self.is_paused else "Pause")

def stop_processing(self):
    self.is_playing = False

def _cleanup(self):
    if self.cap:
        self.cap.release()
        self.cap = None
    if self.video_writer:
        self.video_writer.release()
        self.video_writer = None
    if self.foul_log:
        self.foul_log.close()
        self.foul_log = None
    if self.debug_log:
        self.debug_log.close()
        self.debug_log = None

ready = bool(self.camera_index is not None
             and self.output_dir and self.model)
self.rec_btn.config(state=tk.NORMAL if ready else tk.DISABLED)
self.stoprec_btn.config(state=tk.DISABLED)
self.pause_btn.config(state=tk.DISABLED, text="Pause")
if hasattr(self, "upload_btn"):
    self.upload_btn.config(
        state=tk.NORMAL if (self.output_dir and self.model)
        else tk.DISABLED)
if hasattr(self, "cam_menu") and self.available_cams:
    self.cam_menu.config(state=tk.NORMAL)

```

```

def _update_table_bounds(self, pockets: list):
    if len(self.stable_pocket_bboxes) != 6 and len(pockets) == 6:
        ordered = sorted(pockets, key=lambda p: (p.center[1], p.center[0]))
        self.stable_pocket_bboxes = [list(p.bbox) for p in ordered]
        self.stable_pockets = [bbox_center(b) for b in
self.stable_pocket_bboxes]
        self._add_event_log("6 pocket locations locked")

    if not pockets:
        return

    for p in pockets:
        px, py = p.center
        matched = False

        for i, (sx, sy) in enumerate(self.stable_pockets):
            if pt_dist((px, py), (sx, sy)) < 60:
                self.stable_pockets[i] = (sx * 0.95 + px * 0.05, sy * 0.95 + py *
0.05)
                matched = True
                break

        if not matched:
            matched_candidate = None
            for (cx, cy) in list(self.pocket_candidates.keys()):
                if pt_dist((px, py), (cx, cy)) < 60:
                    self.pocket_candidates[(cx, cy)] += 1
                    matched_candidate = (cx, cy)
                    if self.pocket_candidates[(cx, cy)] > 15:
                        self.stable_pockets.append((cx, cy))
                        del self.pocket_candidates[(cx, cy)]
                        break

            if not matched_candidate:
                self.pocket_candidates[(px, py)] = 1

    if len(self.stable_pockets) >= 4:
        xs = [sx for (sx, sy) in self.stable_pockets]
        ys = [sy for (sx, sy) in self.stable_pockets]

        self.table_bounds = (min(xs), min(ys), max(xs), max(ys))

    if not self.rail_locked and len(self.stable_pockets) == 6:
        self._build_static_rail(self.stable_pockets)

def _build_static_rail(self, centers: list):
    if len(centers) != 6:
        return

```

```

by_y = sorted(centers, key=lambda c: c[1])
top = sorted(by_y[:3], key=lambda c: c[0])
bottom = sorted(by_y[3:], key=lambda c: c[0])

self.pocket_positions = {
    "top_left": top[0],
    "top_middle": top[1],
    "top_right": top[2],
    "bottom_left": bottom[0],
    "bottom_middle": bottom[1],
    "bottom_right": bottom[2],
}

br = (bottom[2][0], bottom[2][1] - BOTTOM_RAIL_PAD_PX)
bl = (bottom[0][0], bottom[0][1] - BOTTOM_RAIL_PAD_PX)
ordered = [
    top[0], top[1], top[2],
    br, bl,
]
self.rail_polygon = np.array(
    [(int(round(x)), int(round(y))) for (x, y) in ordered],
    dtype=np.int32,
)
self.rail_locked = True
self._add_event_log("Static rail locked (6 pockets)")

def _dist_to_rail(self, point) -> Optional[float]:
    if self.rail_polygon is None:
        return None
    pts = self.rail_polygon
    n = len(pts)
    return min(
        pt_to_line_segment_dist(point, tuple(pts[i]), tuple(pts[(i + 1) % n]))
        for i in range(n)
    )

def _ball_at_rail(self, bbox: list, frame_shape: tuple) -> bool:
    bx1, by1, bx2, by2 = bbox

    if self.rail_polygon is not None:
        cx, cy = (bx1 + bx2) * 0.5, (by1 + by2) * 0.5
        radius = ((bx2 - bx1) + (by2 - by1)) * 0.25
        d = self._dist_to_rail((cx, cy))
        return d is not None and d <= RAIL_MARGIN_PX + radius

    h, w = frame_shape[:2]
    if self.table_bounds is not None:
        rx1, ry1, rx2, ry2 = self.table_bounds
    else:

```

```

        rx1, ry1, rx2, ry2 = 0, 0, w, h

    return (
        bx1 <= rx1 + RAIL_MARGIN_PX or
        bx2 >= rx2 - RAIL_MARGIN_PX or
        by1 <= ry1 + RAIL_MARGIN_PX or
        by2 >= ry2 - RAIL_MARGIN_PX
    )

def _add_event_log(self, msg: str):
    t = self.frame_idx / max(self.fps, 1e-6)
    ts_str = f"[{int(t // 60):02d}:{int(t % 60):02d}]"
    self.event_log.append(f"{ts_str} {msg}")

def _mark_ball_potted(self, tid: int, cls: int):
    if tid in self.potted_balls:
        return
    self.potted_balls.add(tid)
    if self.contact_occurred_this_shot:
        self.post_contact_pocketed = True
    name = CLASS_NAMES.get(cls, "ball")
    self._add_event_log(f"{name} potted")

def _ball_replaced_saved_pocket(self, ball: Track, pockets: List[Track]) ->
bool:
    if len(self.stable_pocket_bboxes) != 6:
        return False

    for pocket_bbox in self.stable_pocket_bboxes:
        pocket_visible = any(
            bbox_overlap_ratio(p.bbox, pocket_bbox) >= 0.35
            for p in pockets
        )
        if pocket_visible:
            continue
        if bbox_overlap_ratio(ball.bbox, pocket_bbox) >=
POCKET_REPLACEMENT_OVERLAP:
            return True
    return False

def _evaluate_rail_checks(self):
    if not self.contact_occurred_this_shot:
        return

    if self.post_contact_pocketed:
        return

    if self.post_contact_rail_hit:
        return

```

```

        self._register_foul("No rail contact: Neither the cue nor object balls
reached a rail or pocket after contact")

def _register_no_legal_ball_hit(self):
    if self._no_legal_ball_hit_fired:
        return
    self._register_foul("No legal ball hit")
    self._no_legal_ball_hit_fired = True

def _history_center_at_frame(self, tid: int, target_frame: int, fallback=None):
    hist = self.ball_positions.get(tid)
    if not hist:
        return fallback

    frames = self.ball_position_frames.get(tid)
    if frames and len(frames) == len(hist):
        for idx in range(len(frames) - 1, -1, -1):
            if frames[idx] <= target_frame:
                return hist[idx]
        return fallback if fallback is not None else hist[0]

    frames_back = self.frame_idx - target_frame
    if frames_back < 0:
        return hist[-1]
    idx = len(hist) - 1 - frames_back
    if 0 <= idx < len(hist):
        return hist[idx]
    return fallback if fallback is not None else hist[0]

def _line_of_sight_first_hit(self, moved_tid: int):
    moved_frame = self.balls_moved_this_shot.get(moved_tid)
    moved_info = self.ball_start_info.get(moved_tid)
    if moved_frame is None or moved_info is None:
        return moved_tid, None

    cue_tid = None
    for tid, info in self.ball_start_info.items():
        if info.get("cls") == CUE_CLASS:
            cue_tid = tid
            break
    if cue_tid is None:
        return moved_tid, None

    cue_info = self.ball_start_info.get(cue_tid, {})
    snapshot_frame = moved_frame -
LINE_OF_SIGHT_REWIND_FRAMES
    cue_pos = self._history_center_at_frame(
        cue_tid, snapshot_frame, cue_info.get("center"))
    moved_pos = self._history_center_at_frame(

```

```

        moved_tid, snapshot_frame, moved_info.get("center"))
    if cue_pos is None or moved_pos is None:
        return moved_tid, None

    vx = moved_pos[0] - cue_pos[0]
    vy = moved_pos[1] - cue_pos[1]
    line_len2 = vx * vx + vy * vy
    if line_len2 < 1.0:
        return moved_tid, None

    blockers = []
    for tid, info in self.ball_start_info.items():
        if tid in (cue_tid, moved_tid):
            continue
        if info.get("cls") not in BALL_CLASSES:
            continue
        center = self._history_center_at_frame(
            tid, snapshot_frame, info.get("center"))
        if center is None:
            continue

        wx = center[0] - cue_pos[0]
        wy = center[1] - cue_pos[1]
        t = (wx * vx + wy * vy) / line_len2
        if t <= 0.0 or t >= 1.0:
            continue

        proj = (cue_pos[0] + t * vx, cue_pos[1] + t * vy)
        dist_to_line = pt_dist(center, proj)
        if dist_to_line <= LINE_OF_SIGHT_BLOCK_RADIUS_PX:
            blockers.append((pt_dist(cue_pos, center), tid))

    if not blockers:
        return moved_tid, None

    blockers.sort()
    blocker_tid = blockers[0][1]
    return blocker_tid, moved_tid

def _process_loop(self):
    scratch_frames = max(1, int(SCRATCH_REST_SECS * self.fps))
    off_table_thresh = max(1, int(CUE_OFF_TABLE_SECS * self.fps))
    shot_end_frames = max(1, int(SHOT_END_SECS * self.fps))
    pocket_disappear_thresh = max(1, int(POCKET_DISAPPEAR_SECS *
self.fps))

    try:
        while self.is_playing and self.cap and self.cap.isOpened():
            if self.is_paused:
                time.sleep(0.04)

```

```

        continue

    ret, frame = self.cap.read()
    if not ret:
        break
    self._src_frame_no += 1
    if (self._frame_stride > 1
        and (self._src_frame_no % self._frame_stride) != 0):
        if self.video_writer is not None:
            self.video_writer.write(
                self._last_annotated
                if self._last_annotated is not None else frame)
        continue
    self.frame_idx += 1

    try:
        yolo_out = self.model.predict(
            frame, verbose=False,
            conf=CONF_THRESHOLD,
        )[0]
    except Exception as exc:
        err_msg = f"Inference Error: {exc}"
        print(err_msg)

        cv2.putText(frame, err_msg[:70], (20, 50),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)
        self._display_frame(frame)

        time.sleep(0.03)
        continue

    balls:    List[Track] = []
    cue:      Optional[Track] = None
    pockets:  List[Track] = []
    cue_sticks: List[Track] = []

    rack_detected_this_frame = False

    if len(yolo_out.bboxes) > 0:
        sorted_boxes = sorted(yolo_out.bboxes, key=lambda b:
b.conf.item(), reverse=True)
        seen_ball_classes = set()

        for i, box in enumerate(sorted_boxes):
            raw_cls = int(box.cls.item())

            if raw_cls in BALL_CLASSES or raw_cls == CUE_CLASS:
                if raw_cls in seen_ball_classes:
                    continue
                seen_ball_classes.add(raw_cls)

```



```

        tid = raw_cls
    else:
        tid = 1000 + i

    x1, y1, x2, y2 = box.xyxy[0].cpu().numpy().tolist()
    raw_cls = int(box.cls.item())
    bbox = [x1, y1, x2, y2]
    center = bbox_center(bbox)

    stable_cls = raw_cls
    self.ball_positions[tid].append(center)
    self.ball_position_frames[tid].append(self.frame_idx)

    t = Track(track_id=tid, bbox=bbox,
              stable_cls=stable_cls, center=center)

    if stable_cls in BALL_CLASSES:
        balls.append(t)
    elif stable_cls == CUE_CLASS:
        if cue is None:
            cue = t
    elif stable_cls == POCKET_CLASS:
        pockets.append(t)
    elif stable_cls == 10:
        cue_sticks.append(t)
    elif stable_cls == RACK_CLASS:
        rack_detected_this_frame = True

    if rack_detected_this_frame:
        self.rack_frames_since_seen = 0
    else:
        self.rack_frames_since_seen += 1

    lowest_ball = min((b.stable_cls for b in balls), default=None)
    ball_by_id = {b.track_id: b for b in balls}

    if self.debug_log:
        self.debug_log.write(f"--- Frame {self.frame_idx} ---\n")
        if self.table_bounds:
            rx1, ry1, rx2, ry2 = self.table_bounds
            self.debug_log.write(f" Table Bounds: ({rx1:.1f}, {ry1:.1f}) to ({rx2:.1f}, {ry2:.1f})\n")
        else:
            self.debug_log.write(" Table Bounds: None\n")

    for b in balls:
        if b.stable_cls in (6, 8):
            ball_name = "7-ball" if b.stable_cls == 6 else "9-ball"
            self.debug_log.write(f" {ball_name}:<6> (ID: {b.track_id:3d}) | Center: ({b.center[0]:6.1f}, {b.center[1]:6.1f}) | BBox: {b.bbox}\n")

```

```

        if cue:
            self.debug_log.write(f" CUE   (ID: {cue.track_id:3d}) | Center:
({cue.center[0]:6.1f}, {cue.center[1]:6.1f}) | BBox: {cue.bbox}\n")
            self.debug_log.write("\n")

    self._update_table_bounds(pockets)

    current_ball_ids = {b.track_id for b in balls}

    for b in balls:
        if b.track_id not in self.potted_balls:
            if self._ball_replaced_saved_pocket(b, pockets):
                self._mark_ball_potted(b.track_id, b.stable_cls)
                self.balls_near_pocket.pop(b.track_id, None)
                self.disappeared_near_pocket_frames.pop(b.track_id, None)
                continue

            if any(boxes_overlap(b.bbox, p.bbox, pad=10) for p in pockets):
                self.balls_near_pocket[b.track_id] = b.stable_cls
            else:
                self.balls_near_pocket.pop(b.track_id, None)
                self.disappeared_near_pocket_frames.pop(b.track_id, None)

    if (cue is not None
        and not self._cue_pocket_replace_fired
        and self._ball_replaced_saved_pocket(cue, pockets)):
        self._register_foul("Scratch: cue ball entered pocket")
        self._cue_pocket_replace_fired = True
        self._add_event_log("Cue ball scratch")

    for tid, cls in list(self.balls_near_pocket.items()):
        if tid not in current_ball_ids:
            self.disappeared_near_pocket_frames[tid] += 1

            if self.disappeared_near_pocket_frames[tid] > 5:
                self._mark_ball_potted(tid, cls)
                self.balls_near_pocket.pop(tid, None)
                self.disappeared_near_pocket_frames.pop(tid, None)
            else:
                self.disappeared_near_pocket_frames[tid] = 0

    if self.potted_balls:
        balls = [b for b in balls if b.track_id not in self.potted_balls]
        ball_by_id = {b.track_id: b for b in balls}
        lowest_ball = min((b.stable_cls for b in balls), default=None)

    all_moving = balls + ([cue] if cue else [])

```

```

for b in all_moving:
    start_info = self.ball_start_info.get(b.track_id)
    if start_info and start_info["on_rail"]:
        if b.track_id not in self.balls_left_rail_this_shot:
            left_zone = False
            if self.rail_polygon is not None:
                bx1, by1, bx2, by2 = b.bbox
                radius = ((bx2 - bx1) + (by2 - by1)) * 0.25
                d = self._dist_to_rail(b.center)
                left_zone = (d is not None and
                             d > RAIL_MARGIN_PX + 10 + radius)
            elif self.table_bounds is not None:
                rx1, ry1, rx2, ry2 = self.table_bounds
                bx1, by1, bx2, by2 = b.bbox
                m = RAIL_MARGIN_PX + 10
                left_zone = (bx1 > rx1 + m and bx2 < rx2 - m and
                             by1 > ry1 + m and by2 < ry2 - m)

            dist_moved = pt_dist(b.center, start_info["center"])
            if left_zone or dist_moved > 40:
                self.balls_left_rail_this_shot.add(b.track_id)

if (self.shot_in_progress
    or self.post_shot_eval_frame is not None):
    for b in all_moving:
        if b.track_id not in self.balls_hit_rail_this_shot:
            if self._ball_at_rail(b.bbox, frame.shape):

                start_center = None
                started_on_rail = False

                if b.track_id in self.ball_start_info:
                    start_center = self.ball_start_info[b.track_id]["center"]
                    started_on_rail =
self.ball_start_info[b.track_id]["on_rail"]
                elif self.ball_start_info:
                    _, nearest_info = min(
                        self.ball_start_info.items(),
                        key=lambda item: pt_dist(b.center, item[1]["center"])
                    )
                    if pt_dist(b.center, nearest_info["center"]) < 40:
                        start_center = nearest_info["center"]
                        started_on_rail = nearest_info["on_rail"]

                valid_rail_hit = False

            if start_center is None:
                self.ball_start_info[b.track_id] = {
                    "center": b.center,
                    "on_rail": True

```

```

        }
    else:
        dist_moved = pt_dist(b.center, start_center)

        if dist_moved >= 60:
            if started_on_rail:
                if b.track_id in self.balls_left_rail_this_shot or
dist_moved > 60:
                    valid_rail_hit = True
                else:
                    valid_rail_hit = True

            if valid_rail_hit:
                self.balls_hit_rail_this_shot.add(b.track_id)
                self.post_contact_rail_hit = True
                name = CLASS_NAMES.get(b.stable_cls, "ball")
                self._add_event_log(f"{name} hit rail")

self._last_pocket_bboxes = [tuple(p.bbox) for p in pockets]
self._last_cue_bbox = tuple(cue.bbox) if cue is not None else None

if cue is not None:
    self.cue_missing_frames = 0
    self._missing_fired = False
    self._off_table_fired = False
    cc = cue.center

    if self.prev_cue_center is not None:
        self.cue_velocity = pt_dist(cc, self.prev_cue_center)
        self.prev_cue_center = cc

    if (not self.shot_in_progress
        and self.cue_velocity > SHOT_START_VEL):
        self.shot_in_progress = True
        self.lowest_at_shot_start = lowest_ball
        self.checked_contact_tids.clear()
        self._low_vel_run = 0
        self.balls_hit_rail_this_shot.clear()
        self.post_shot_eval_frame = None
        self.balls_moved_this_shot.clear()
        self.trajectory_candidates.clear()
        self.first_trajectory_hit_fired = False
        self._no_legal_ball_hit_fired = False

    _all_start_balls = balls + ([cue] if cue else [])
    self.ball_start_info = {
        _b.track_id: {
            "center": _b.center,
            "on_rail": self._ball_at_rail(_b.bbox, frame.shape),

```

```

        "cls": _b.stable_cls
    } for _b in _all_start_balls
}
self.balls_left_rail_this_shot.clear()

self.is_break_shot = (self.rack_frames_since_seen < 30)

log_msg = "Shot started (Break Shot)" if self.is_break_shot else
"Shot started"
self._add_event_log(log_msg)

if self.shot_in_progress and not self.contact_occurred_this_shot:
    for ball in balls:
        if ball.track_id not in self.checked_contact_tids:

            is_hit = False

            if boxes_overlap(cue.bbox, ball.bbox,
pad=CONTACT_BBOX_PAD):
                is_hit = True

            elif self.prev_cue_center is not None and self.cue_velocity
> 5.0:
                path_dist = pt_to_line_segment_dist(ball.center,
self.prev_cue_center, cue.center)

                if path_dist < 20.0:
                    is_hit = True

            if is_hit:
                self.checked_contact_tids.add(ball.track_id)
                self.pending_contacts.append(PendingContact(
                    contacted_track_id = ball.track_id,
                    contacted_class    = ball.stable_cls,
                    pos_at_contact     = ball.center,
                    created_frame      = self.frame_idx,
                    shot_target_class  = self.lowest_at_shot_start,
                ))

if self.shot_in_progress:
    for b in balls:
        if b.track_id not in self.balls_moved_this_shot:
            start_info = self.ball_start_info.get(b.track_id)
            if start_info:
                dist = pt_dist(b.center, start_info["center"])
                if dist >= MOVEMENT_THRESHOLD_PX:
                    self.balls_moved_this_shot[b.track_id] =
self.frame_idx

cue_corridor_px = (cue.bbox[3] - cue.bbox[1]) * 0.5

```

```

cue_hist = self.ball_positions[cue.track_id]
lookback = max(1, min(int(self.fps * 0.15), len(cue_hist) - 1))
if len(cue_hist) >= 2:
    past_pos = cue_hist[max(0, len(cue_hist) - lookback - 1)]
    curr_pos = cue.center
    tdx = curr_pos[0] - past_pos[0]
    tdy = curr_pos[1] - past_pos[1]
    tspeed = (tdx**2 + tdy**2) ** 0.5
    if tspeed > 3.0:
        extend = TRAJECTORY_LOOKAHEAD_PX / tspeed
        far_x = curr_pos[0] + tdx * extend
        far_y = curr_pos[1] + tdy * extend
        for _b in balls:
            if _b.track_id not in self.trajectory_candidates:
                _d = pt_to_line_segment_dist(
                    _b.center, curr_pos, (far_x, far_y))
                if _d < cue_corridor_px:
                    self.trajectory_candidates.add(_b.track_id)

if (not self.contact_occurred_this_shot
    and not self.first_trajectory_hit_fired
    and self.trajectory_candidates):
    moved_candidates = {
        tid: frm
        for tid, frm in self.balls_moved_this_shot.items()
        if tid in self.trajectory_candidates
    }
    if moved_candidates:
        first_hit_tid = min(
            moved_candidates, key=moved_candidates.get)
        first_hit_tid, _blocked_moved_tid = (
            self._line_of_sight_first_hit(first_hit_tid))
        first_hit_cls = self.ball_start_info.get(
            first_hit_tid, {}).get("cls")
        if first_hit_cls is not None:
            self.first_trajectory_hit_fired = True
            self.contact_occurred_this_shot = True
            self.post_contact_rail_hit = False
            self.post_contact_pocketed = False
            self.active_ball_ids_at_contact = {
                _b.track_id: _b.stable_cls for _b in balls}
            hit_name = CLASS_NAMES.get(
                first_hit_cls, f"#{first_hit_tid}")
            self._add_event_log(
                f"Traj 1st hit: {hit_name}")
            if not self.is_break_shot:
                target = self.lowest_at_shot_start
                if (target is not None
                    and first_hit_cls != target):
                    self._register_no_legal_ball_hit()

```

```

if self.shot_in_progress:
    if self.cue_velocity < SHOT_END_VEL:
        self._low_vel_run += 1
        if self._low_vel_run >= shot_end_frames:
            self.shot_in_progress = False
            self._low_vel_run = 0
            self._add_event_log("Shot ended")
            self.post_shot_eval_frame = (
                self.frame_idx + POST_SHOT_GRACE_FRAMES)
    else:
        self._low_vel_run = 0

in_pocket = any(
    boxes_overlap(cue.bbox, p.bbox) for p in pockets)
if in_pocket and self.cue_velocity < SHOT_END_VEL:
    self.cue_in_pocket_frames += 1
    if (self.cue_in_pocket_frames >= scratch_frames
        and not self._scratch_fired):
        self._register_foul(
            "Scratch: cue ball resting in pocket")
        self._scratch_fired = True
    else:
        self.cue_in_pocket_frames = 0
        self._scratch_fired = False

if in_pocket:
    self.cue_near_pocket_streak = 8
else:
    self.cue_near_pocket_streak = max(
        0, self.cue_near_pocket_streak - 1)

self.cue_was_near_pocket = (
    self.cue_near_pocket_streak > 0)
self.cue_gone_after_pocket_frames = 0
self._pocket_disappear_fired = False

else:
    self.cue_velocity = 0.0
    self.cue_in_pocket_frames = 0
    self.cue_missing_frames += 1

if (not self.cue_was_near_pocket
    and self.prev_cue_center is not None
    and self.stable_pockets):
    for _px, _py in self.stable_pockets:
        if pt_dist(self.prev_cue_center, (_px, _py)) < 70:
            self.cue_was_near_pocket = True
            break

```

```

        if self.cue_was_near_pocket:
            self.cue_gone_after_pocket_frames += 1
            if (self.cue_gone_after_pocket_frames >=
pocket_disappear_thresh
                and not self._pocket_disappear_fired):
                self._register_foul(
                    "Scratch: cue ball entered pocket")
                self._pocket_disappear_fired = True
                self._add_event_log("Cue ball scratch")

        elif self.prev_cue_center is not None:
            if (self.cue_missing_frames >= off_table_thresh
                and not self._off_table_fired):
                self._register_foul(
                    "Cue ball off the table")
                self._off_table_fired = True
                self._add_event_log("Cue ball off table")

        if (self.post_shot_eval_frame is not None
            and self.frame_idx >= self.post_shot_eval_frame):

            if not self.contact_occurred_this_shot and
self.balls_moved_this_shot:
                first_moved_id = min(self.balls_moved_this_shot,
key=self.balls_moved_this_shot.get)
                first_moved_id, blocked_moved_id = (
                    self._line_of_sight_first_hit(first_moved_id))

            if first_moved_id in self.ball_start_info:
                first_moved_cls = self.ball_start_info[first_moved_id]["cls"]

                self.contact_occurred_this_shot = True
                hit_name = CLASS_NAMES.get(first_moved_cls,
f"#{first_moved_id}")
                if blocked_moved_id is not None:
                    self._add_event_log(
                        f"Fallback hit: {hit_name} blocked line of sight")
                else:
                    self._add_event_log(
                        f"Fallback hit: {hit_name} moved first")

            if not self.is_break_shot:
                target = self.lowest_at_shot_start
                if target is not None and first_moved_cls != target:
                    self._register_no_legal_ball_hit()
            if not self.contact_occurred_this_shot:
                self._register_no_legal_ball_hit()

        self._evaluate_rail_checks()
        self.contact_occurred_this_shot = False

```



```

        self.is_break_shot          = False
        self.post_shot_eval_frame   = None

    still_pending: List[PendingContact] = []
    for pc in self.pending_contacts:
        age = self.frame_idx - pc.created_frame

        if pc.contacted_track_id in ball_by_id:
            cur_pos = ball_by_id[pc.contacted_track_id].center
            moved   = pt_dist(cur_pos, pc.pos_at_contact)
        else:
            moved = 0.0

        if moved >= MOVEMENT_THRESHOLD_PX:
            if not self.contact_occurred_this_shot:
                target = pc.shot_target_class
                hit_name = CLASS_NAMES[pc.contacted_class]

                self._add_event_log(f"Cue hit {hit_name}")

                if not self.is_break_shot:
                    if target is not None and pc.contacted_class != target:
                        self._register_no_legal_ball_hit()

                self.contact_occurred_this_shot = True
                self.post_contact_rail_hit = False
                self.post_contact_pocketed = False
                self.active_ball_ids_at_contact = {
                    b.track_id: b.stable_cls for b in balls
                }
            elif age < MOVEMENT_CONFIRM_FRAMES:
                still_pending.append(pc)
            else:
                self.checked_contact_tids.discard(pc.contacted_track_id)

    self.pending_contacts = still_pending

    annotated = self._draw(
        frame, balls, cue, pockets, cue_sticks, lowest_ball)

    if self.video_writer:
        self.video_writer.write(annotated)
    self._last_annotated = annotated
    self._display_frame(annotated)

    time.sleep(max(0.0, 1.0 / self.fps - 0.015))

finally:
    self.is_playing = False
    _eov_fouls_before = self._foul_count

```

```

if self.shot_in_progress or self.post_shot_eval_frame is not None:
    if self.contact_occurred_this_shot:
        self._evaluate_rail_checks()
    else:
        self._register_no_legal_ball_hit()

if (not self._scratch_fired and not self._pocket_disappear_fired
    and not self._cue_pocket_replace_fired
    and self._last_cue_bbox is not None
    and any(bbox_inside(self._last_cue_bbox, pb)
             for pb in self._last_pocket_bboxes)):
    self._register_foul("Scratch: cue ball resting in pocket")
    self._scratch_fired = True
    self._add_event_log("Cue ball scratch")

cue_pocketed = (self._scratch_fired or self._pocket_disappear_fired
                or self._cue_pocket_replace_fired)
if (self.cue_missing_frames > 0
    and not cue_pocketed
    and not self._off_table_fired):
    self._register_foul("Cue ball off the table")
    self._off_table_fired = True
    self._add_event_log("Cue ball off table")

if (self._foul_count > _eov_fouls_before
    and self._last_annotated is not None):
    annotated = self._render_foul_banner(self._last_annotated.copy())
    self._last_annotated = annotated
    self._display_frame(annotated)
    if self.video_writer:
        for _ in range(max(1, int(FOUL_BANNER_SECS * self.fps))):
            self.video_writer.write(annotated)

self.root.after(0, self._cleanup)

def _draw(self, frame, balls: List[Track], cue: Optional[Track],
          pockets: List[Track], cue_sticks: List[Track],
          lowest_ball: Optional[int]):

    out = frame.copy()
    h, w = out.shape[:2]

    if self.rail_polygon is not None:
        cv2.polylines(out, [self.rail_polygon], isClosed=True,
                      color=(0, 160, 160), thickness=2)
        _short = {
            "top_left": "TL", "top_middle": "TM", "top_right": "TR",
            "bottom_left": "BL", "bottom_middle": "BM", "bottom_right": "BR",
        }

```

```

for _name, (_px, _py) in self.pocket_positions.items():
    cv2.circle(out, (int(_px), int(_py)), 4, (0, 200, 200), -1)
    cv2.putText(out, _short.get(_name, _name),
                (int(_px) + 6, int(_py) - 6),
                cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0, 200, 200), 1)
cv2.putText(out, "rail",
            (int(self.rail_polygon[0][0]) + 4,
             int(self.rail_polygon[0][1]) + 16),
            cv2.FONT_HERSHEY_SIMPLEX, 0.38, (0, 160, 160), 1)
elif self.table_bounds is not None:
    rx1, ry1, rx2, ry2 = (int(v) for v in self.table_bounds)
    cv2.rectangle(out, (rx1, ry1), (rx2, ry2), (0, 160, 160), 1)
    m = RAIL_MARGIN_PX
    cv2.rectangle(out,
                  (rx1 + m, ry1 + m),
                  (rx2 - m, ry2 - m),
                  (0, 100, 100), 1)
    cv2.putText(out, "rail zone (forming...)", (rx1 + 4, ry1 + 16),
                cv2.FONT_HERSHEY_SIMPLEX, 0.38, (0, 160, 160), 1)

needs_rail = (self.contact_occurred_this_shot and
              not self.post_contact_rail_hit and
              not self.post_contact_pocketed)

for p in pockets:
    x1, y1, x2, y2 = map(int, p.bbox)
    cv2.rectangle(out, (x1, y1), (x2, y2), (0, 0, 180), 2)
    cv2.putText(out, "pocket", (x1, max(0, y1 - 5)),
                cv2.FONT_HERSHEY_SIMPLEX, 0.42, (0, 0, 180), 1)

for s in cue_sticks:
    x1, y1, x2, y2 = map(int, s.bbox)
    cv2.rectangle(out, (x1, y1), (x2, y2), (0, 140, 255), 2)

for b in balls:
    x1, y1, x2, y2 = map(int, b.bbox)
    is_target = (b.stable_cls == lowest_ball)
    is_waiting_rail = (needs_rail and is_target)

    if is_waiting_rail:
        color, thick = (0, 140, 255), 3
    elif is_target:
        color, thick = (0, 255, 60), 3
    else:
        color, thick = (0, 215, 215), 2

    cv2.rectangle(out, (x1, y1), (x2, y2), color, thick)

    label = CLASS_NAMES.get(b.stable_cls, f"#{b.track_id}")
    if is_waiting_rail:

```

```

        label += " ⚠ rail?"
        label += f" #{b.track_id}"

        cv2.putText(out, label, (x1, max(0, y1 - 6)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.42, color, 1)

    if cue is not None:
        x1, y1, x2, y2 = map(int, cue.bbox)
        cv2.rectangle(out, (x1, y1), (x2, y2), (255, 255, 255), 2)
        cv2.putText(out, f"cue #{cue.track_id}",
                    (x1, max(0, y1 - 6)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.42, (255, 255, 255), 1)

        history = self.ball_positions[cue.track_id]

        lookback_frames = int(self.fps * 0.2)

        if len(history) >= 2:
            past_idx = max(0, len(history) - lookback_frames - 1)
            past_center = history[past_idx]
            curr_center = cue.center

            dx = curr_center[0] - past_center[0]
            dy = curr_center[1] - past_center[1]

            dist = (dx**2 + dy**2)**0.5

            if dist > 5.0:
                scale = 2.5

                pred_x = int(curr_center[0] + dx * scale)
                pred_y = int(curr_center[1] + dy * scale)

                cv2.line(out, (int(curr_center[0]), int(curr_center[1])),
                        (pred_x, pred_y), (255, 255, 0), 2)

                cv2.circle(out, (pred_x, pred_y), 4, (0, 0, 255), -1)

        panel_bottom = 120 + (len(self.event_log) * 25)

        ov = out.copy()
        cv2.rectangle(ov, (8, 8), (600, panel_bottom), (0, 0, 0), -1)
        cv2.addWeighted(ov, 0.55, out, 0.45, 0, out)
        cv2.rectangle(out, (8, 8), (600, panel_bottom), (0, 200, 200), 2)

        cv2.putText(out, "LOWEST BALL ON TABLE",
                    (22, 40), cv2.FONT_HERSHEY_SIMPLEX,
                    0.65, (160, 200, 200), 2)

    if lowest_ball is not None:

```

```

label = CLASS_NAMES[lowest_ball]
col_name = BALL_COLOR_ID.get(lowest_ball, "")
swatch = BALL_BGR.get(lowest_ball)
sx, sy, sr = 34, 74, 16

if isinstance(swatch, tuple) and swatch and isinstance(swatch[0], tuple):
    cv2.ellipse(out, (sx, sy), (sr, sr), 0, 180, 360, swatch[0], -1)
    cv2.ellipse(out, (sx, sy), (sr, sr), 0, 0, 180, swatch[1], -1)
elif swatch is not None:
    cv2.circle(out, (sx, sy), sr, swatch, -1)
cv2.circle(out, (sx, sy), sr, (255, 255, 255), 2)

text = f"{label} [{col_name}]" if col_name else label
cv2.putText(out, text, (sx + sr + 14, sy + 9),
            cv2.FONT_HERSHEY_SIMPLEX, 0.95, (255, 255, 255), 2)
else:
    cv2.putText(out, "NONE DETECTED", (22, 85),
                cv2.FONT_HERSHEY_SIMPLEX, 1.0, (130, 130, 130), 3)

y_offset = 125
for event_str in self.event_log:
    cv2.putText(out, event_str, (22, y_offset),
                cv2.FONT_HERSHEY_SIMPLEX, 0.55, (220, 220, 220), 2)
    y_offset += 25

if SHOW_CONTACT_DEBUG:
    def _names(ids):
        if not ids:
            return "-"
        return ", ".join(CLASS_NAMES.get(i, f"#{i}") for i in sorted(ids))

    dbg_lines = [
        f"shot={self.shot_in_progress}"
contact=f"{self.contact_occurred_this_shot}",
        f"traj_cand: {_names(self.trajectory_candidates)}",
        f"moved: {_names(self.balls_moved_this_shot.keys())}",
        f"potted: {_names(self.potted_balls)}",
    ]
    dh = out.shape[0]
    y0 = dh - 18 - (len(dbg_lines) - 1) * 22
    for i, line in enumerate(dbg_lines):
        y = y0 + i * 22
        cv2.putText(out, line, (12, y), cv2.FONT_HERSHEY_SIMPLEX,
                    0.5, (0, 0, 0), 3)
        cv2.putText(out, line, (12, y), cv2.FONT_HERSHEY_SIMPLEX,
                    0.5, (80, 255, 255), 1)

out = self._render_foul_banner(out)

return out

```

```

def _render_foul_banner(self, out):
    if not (self.foul_banner_text
            and self.frame_idx < self.foul_banner_until_frame):
        return out

    w = out.shape[1]
    banner = f" FOUL: {self.foul_banner_text} "
    font = cv2.FONT_HERSHEY_SIMPLEX
    thick = 2

    scale = 0.85
    (tw, th), _ = cv2.getTextSize(banner, font, scale, thick)
    while tw > (w - 20) and scale > 0.4:
        scale -= 0.05
        (tw, th), _ = cv2.getTextSize(banner, font, scale, thick)

    by = 20
    bx = max(10, w - tw - 10)
    cv2.rectangle(out, (bx, by),
                  (bx + tw, by + th + 20), (0, 0, 190), -1)
    cv2.rectangle(out, (bx, by),
                  (bx + tw, by + th + 20), (80, 80, 255), 2)
    cv2.putText(out, banner, (bx, by + th + 7),
                font, scale, (255, 255, 255), thick)
    return out

def _register_foul(self, reason: str):
    t = self.frame_idx / max(self.fps, 1e-6)
    ts_str = f"[{int(t // 60):02d}:{int(t % 60):02d}]"
    entry = f"[{ts_str}] {reason}"

    self._foul_count += 1
    self.foul_banner_text = reason
    self.foul_banner_until_frame = (
        self.frame_idx + int(FOUL_BANNER_SECS * self.fps))

    if self.foul_log:
        self.foul_log.write(entry + "\n")
        self.foul_log.flush()

    def _add():
        self.foul_listbox.insert(tk.END, entry)
        self.foul_listbox.see(tk.END)
        self.root.after(0, _add)

def _display_frame(self, frame_bgr):
    h, w = frame_bgr.shape[:2]
    max_w = 1200

```

```

        if w > max_w:
            scale = max_w / w
            frame_bgr = cv2.resize(frame_bgr, (max_w, int(h * scale)))
            rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)

        def _update():
            img = Image.fromarray(rgb)
            imgtk = ImageTk.PhotoImage(image=img)
            self.video_lbl.imgtk = imgtk
            self.video_lbl.config(image=imgtk)
            self.root.after(0, _update)

if __name__ == "__main__":
    root = tk.Tk()
    app = FoulDetectorApp(root)

    def _on_close():
        try:
            app.is_recording = False
            app.is_playing = False
        except Exception:
            pass
        root.destroy()

    root.protocol("WM_DELETE_WINDOW", _on_close)
    root.mainloop()

```

LAMPIRAN 2 (Model_Testing.py)

```

import os
import sys
import glob
import time
import threading
import traceback
from contextlib import nullcontext

os.environ.setdefault('KMP_DUPLICATE_LIB_OK', 'TRUE')

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))

MODEL_DIRS = {
    "YOLO11n": "../Trained_Models/YOLO11n",
    "YOLOv8n": "../Trained_Models/YOLOv8n",
    "YOLOv10n": "../Trained_Models/YOLOv10n",
}

```

```

OUTPUT_ROOT = "../Output_Folder"

IMG_EXTS = (".jpg", ".jpeg", ".png", ".bmp", ".webp", ".tif", ".tiff")

def anchor(path):
    if not path:
        return path
    if os.path.isabs(path):
        return os.path.normpath(path)
    return os.path.normpath(os.path.join(SCRIPT_DIR, path))

def _find_weights(model_name, exts):
    folder = MODEL_DIRS.get(model_name)
    if not folder:
        return []
    folder = anchor(folder)
    if not os.path.isdir(folder):
        return []
    hits = []
    for ext in exts:
        hits += glob.glob(os.path.join(folder, "*" + ext))
        hits += glob.glob(os.path.join(folder, "*", "*" + ext))
        hits += glob.glob(os.path.join(folder, "*", "*", "*" + ext))
    return sorted(set(hits))

def resolve_pt(model_name):
    pts = _find_weights(model_name, (".pt",))
    if not pts:
        return None
    key = model_name.lower().replace("yolo", "")
    for p in pts:
        if key in os.path.basename(p).lower():
            return p
    for target in ("best.pt", "last.pt"):
        for p in pts:
            if os.path.basename(p).lower() == target:
                return p
    return pts[0]

def resolve_hef(model_name):
    hefs = _find_weights(model_name, (".hef",))
    if not hefs:
        return None
    key = model_name.lower().replace("yolo", "")
    for h in hefs:

```



```

        if key in os.path.basename(h).lower():
            return h
    return hefs[0]

def resolve_split(data_yaml, split):
    try:
        import yaml
        with open(data_yaml, "r", encoding="utf-8") as f:
            cfg = yaml.safe_load(f) or {}
    except Exception:
        return split
    if cfg.get(split) is not None:
        return split
    fallback = "val" if split != "val" else "test"
    if cfg.get(fallback) is not None:
        print(f"[WARN] Split '{split}' not found in dataset.yaml; using
'{fallback}'.")
        return fallback
    print(f"[WARN] dataset.yaml has no '{split}' entry and no obvious fallback.
")
    f"Trying '{split}' anyway.")
    return split

def load_names(cfg):
    names = cfg.get("names")
    if isinstance(names, dict):
        return {int(k): str(v) for k, v in names.items()}
    if isinstance(names, (list, tuple)):
        return {i: str(n) for i, n in enumerate(names)}
    return {}

def gather_split_images(data_yaml, split):
    import yaml
    with open(data_yaml, "r", encoding="utf-8") as f:
        cfg = yaml.safe_load(f) or {}

    names = load_names(cfg)
    yaml_dir = os.path.dirname(os.path.abspath(data_yaml))
    base = cfg.get("path") or ""
    base = base if os.path.isabs(base) else
os.path.normpath(os.path.join(yaml_dir, base or "."))

    entry = cfg.get(split)
    if entry is None:
        raise ValueError(f"dataset.yaml has no '{split}' entry.")
    entries = entry if isinstance(entry, (list, tuple)) else [entry]

```

```

images = []
for e in entries:
    p = e if os.path.isabs(e) else os.path.normpath(os.path.join(base, e))
    if os.path.isdir(p):
        for root, _dirs, files in os.walk(p):
            for fn in files:
                if fn.lower().endswith(IMG_EXTS):
                    images.append(os.path.join(root, fn))
    elif os.path.isfile(p) and p.lower().endswith(".txt"):
        with open(p, "r", encoding="utf-8") as fh:
            for line in fh:
                line = line.strip()
                if not line:
                    continue
                ip = line if os.path.isabs(line) else
os.path.normpath(os.path.join(base, line))
                images.append(ip)
    elif os.path.isfile(p) and p.lower().endswith(IMG_EXTS):
        images.append(p)
    else:
        print(f"[WARN] Could not interpret split entry: {p}")

images = sorted(set(images))
return images, names

def label_path_for(image_path):
    parts = image_path.replace("\\", "/").split("/")
    for i in range(len(parts) - 1, -1, -1):
        if parts[i] == "images":
            parts[i] = "labels"
            break
    stem = os.path.splitext(parts[-1])[0] + ".txt"
    parts[-1] = stem
    return os.path.normpath("/".join(parts))

def load_gt(image_path, w0, h0):
    lp = label_path_for(image_path)
    import numpy as np
    if not os.path.isfile(lp):
        return np.zeros((0, 4), dtype=np.float32), np.zeros((0,), dtype=np.int64)
    boxes, classes = [], []
    with open(lp, "r", encoding="utf-8") as f:
        for line in f:
            v = line.split()
            if len(v) < 5:
                continue
            c, cx, cy, bw, bh = int(float(v[0])), *map(float, v[1:5])
            x1 = (cx - bw / 2.0) * w0

```

```

        y1 = (cy - bh / 2.0) * h0
        x2 = (cx + bw / 2.0) * w0
        y2 = (cy + bh / 2.0) * h0
        boxes.append([x1, y1, x2, y2])
        classes.append(c)
    if not boxes:
        return np.zeros((0, 4), dtype=np.float32), np.zeros((0,), dtype=np.int64)
    return np.asarray(boxes, dtype=np.float32), np.asarray(classes,
dtype=np.int64)

```

```

class CPUPeakMonitor:

```

```

    def __init__(self, interval=0.1):
        self.interval = interval
        self.peak = None
        self.ram_peak_used = None
        self.ram_total = None
        self._stop = threading.Event()
        self._thread = None
        try:
            import psutil
            self._psutil = psutil
            self.ram_total = int(psutil.virtual_memory().total)
        except Exception:
            self._psutil = None

```

```

    def _run(self):
        self._psutil.cpu_percent(interval=None)
        self.peak = 0.0
        self.ram_peak_used = 0
        while not self._stop.is_set():
            c = self._psutil.cpu_percent(interval=None)
            if c > self.peak:
                self.peak = c
            vm = self._psutil.virtual_memory()
            used = int(vm.total - vm.available)
            if used > self.ram_peak_used:
                self.ram_peak_used = used
            time.sleep(self.interval)

```

```

    def start(self):
        if self._psutil is not None:
            self._thread = threading.Thread(target=self._run, daemon=True)
            self._thread.start()
        return self

```

```

    def stop(self):
        self._stop.set()
        if self._thread is not None:
            self._thread.join(timeout=1.0)

```

```

        return self.peak

def preprocess(img_bgr, net_w, net_h, to_rgb):
    import cv2
    import numpy as np
    h0, w0 = img_bgr.shape[:2]
    if to_rgb:
        img_bgr = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    out = cv2.resize(img_bgr, (net_w, net_h),
interpolation=cv2.INTER_LINEAR)
    meta = (w0, h0)
    return np.ascontiguousarray(out, dtype=np.uint8), meta

def boxes_to_original(nboxes_xyxy, meta):
    import numpy as np
    w0, h0 = meta
    b = nboxes_xyxy.astype(np.float32).copy()
    b[:, [0, 2]] *= w0
    b[:, [1, 3]] *= h0
    b[:, [0, 2]] = b[:, [0, 2]].clip(0, w0)
    b[:, [1, 3]] = b[:, [1, 3]].clip(0, h0)
    return b

def box_iou_np(a, b):
    import numpy as np
    if len(a) == 0 or len(b) == 0:
        return np.zeros((len(a), len(b)), dtype=np.float32)
    area_a = (a[:, 2] - a[:, 0]).clip(0) * (a[:, 3] - a[:, 1]).clip(0)
    area_b = (b[:, 2] - b[:, 0]).clip(0) * (b[:, 3] - b[:, 1]).clip(0)
    lt = np.maximum(a[:, None, :2], b[None, :, :2])
    rb = np.minimum(a[:, None, 2:], b[None, :, 2:])
    wh = (rb - lt).clip(0)
    inter = wh[..., 0] * wh[..., 1]
    union = area_a[:, None] + area_b[None, :] - inter
    return inter / np.maximum(union, 1e-9)

def match_one_image(pred_boxes, pred_cls, gt_boxes, gt_cls, iou_thr):
    import numpy as np
    correct = np.zeros((len(pred_boxes), len(iou_thr)), dtype=bool)
    if len(pred_boxes) == 0 or len(gt_boxes) == 0:
        return correct
    iou = box_iou_np(pred_boxes, gt_boxes)
    same_cls = gt_cls[None, :] == pred_cls[:, None]
    iou = np.where(same_cls, iou, 0.0)
    for ti, thr in enumerate(iou_thr):
        yx = np.nonzero(iou >= thr)

```

```

        if yx[0].size == 0:
            continue
        m = np.stack(yx, axis=1)
        if m.shape[0] > 1:
            ious = iou[m[:, 0], m[:, 1]]
            m = m[np.argsort(-ious)]
            _, keep = np.unique(m[:, 1], return_index=True)
            m = m[keep]
            _, keep = np.unique(m[:, 0], return_index=True)
            m = m[keep]
            correct[m[:, 0], ti] = True
        return correct

def _ap(recall, precision):
    import numpy as np
    mrec = np.concatenate([[0.0], recall, [1.0]])
    mpre = np.concatenate([[1.0], precision, [0.0]])
    mpre = np.flip(np.maximum.accumulate(np.flip(mpre)))
    x = np.linspace(0, 1, 101)
    return float(np.trapz(np.interp(x, mrec, mpre), x))

def compute_metrics(correct, conf, pred_cls, target_cls, names, iou_thr):
    import numpy as np
    order = np.argsort(-conf)
    correct, conf, pred_cls = correct[order], conf[order], pred_cls[order]
    unique_classes, n_gt = np.unique(target_cls, return_counts=True)
    niou = correct.shape[1]
    i50 = int(np.argmax(np.abs(iou_thr - 0.5)))
    i75 = int(np.argmax(np.abs(iou_thr - 0.75)))

    per_class = []
    ap_all = []
    p_all, r_all = [], []
    for ci, c in enumerate(unique_classes):
        sel = pred_cls == c
        ngd = n_gt[ci]
        if sel.sum() == 0 or ngd == 0:
            ap_row = np.zeros(niou)
            per_class.append((int(c), 0.0, 0.0, 0.0, float(ap_row.mean())))
            ap_all.append(ap_row)
            p_all.append(0.0)
            r_all.append(0.0)
            continue
        tp = correct[sel]
        tpc = tp.cumsum(0)
        fpc = (~tp).cumsum(0)
        recall = tpc / (ngd + 1e-9)
        precision = tpc / (tpc + fpc + 1e-9)

```

```

ap_row = np.array([_ap(recall[:, j], precision[:, j]) for j in range(niou)])
pc, rc = precision[:, i50], recall[:, i50]
f1 = 2 * pc * rc / (pc + rc + 1e-9)
k = int(f1.argmax())
per_class.append((int(c), float(pc[k]), float(rc[k]),
                    float(ap_row[i50]), float(ap_row.mean()))))
ap_all.append(ap_row)
p_all.append(float(pc[k]))
r_all.append(float(rc[k]))

ap_all = np.array(ap_all) if ap_all else np.zeros((0, niou))
result = {
    "precision": float(np.mean(p_all)) if p_all else 0.0,
    "recall": float(np.mean(r_all)) if r_all else 0.0,
    "map50": float(ap_all[:, i50].mean()) if len(ap_all) else 0.0,
    "map75": float(ap_all[:, i75].mean()) if len(ap_all) else 0.0,
    "map": float(ap_all.mean()) if len(ap_all) else 0.0,
    "per_class": [{
        "name": str(names.get(c, c)), "p": p, "r": r, "ap50": a50, "ap": a
    } for (c, p, r, a50, a) in per_class],
}
return result

```

class ConfusionMatrix:

```

def __init__(self, nc, iou_thres=0.45):
    import numpy as np
    self.nc = int(nc)
    self.iou_thres = float(iou_thres)
    self.matrix = np.zeros((self.nc + 1, self.nc + 1), dtype=np.int64)

def process_batch(self, pred_boxes, pred_cls, gt_boxes, gt_cls):
    import numpy as np
    nc = self.nc
    gt_cls = np.asarray(gt_cls, dtype=np.int64).reshape(-1)
    pred_cls = np.asarray(pred_cls, dtype=np.int64).reshape(-1)

    if gt_cls.shape[0] == 0:
        for dc in pred_cls:
            if 0 <= dc < nc:
                self.matrix[dc, nc] += 1
        return

    if pred_cls.shape[0] == 0:
        for gc in gt_cls:
            if 0 <= gc < nc:
                self.matrix[nc, gc] += 1
        return

```

```

iou = box_iou_np(gt_boxes, pred_boxes)
x = np.nonzero(iou > self.iou_thres)
if x[0].shape[0]:
    ious = iou[x[0], x[1]]
    matches = np.concatenate((np.stack(x, axis=1),
                               ious[:, None]), axis=1)
    if x[0].shape[0] > 1:
        matches = matches[matches[:, 2].argsort()[::-1]]
        matches = matches[np.unique(matches[:, 1], return_index=True)[1]]
        matches = matches[matches[:, 2].argsort()[::-1]]
        matches = matches[np.unique(matches[:, 0], return_index=True)[1]]
    else:
        matches = np.zeros((0, 3))

n = matches.shape[0] > 0
m_gt = matches[:, 0].astype(int)
m_pred = matches[:, 1].astype(int)

for i, gc in enumerate(gt_cls):
    if not (0 <= gc < nc):
        continue
    sel = m_gt == i
    if n and sel.sum() == 1:
        dc = int(pred_cls[m_pred[sel][0]])
        self.matrix[dc if 0 <= dc < nc else nc, gc] += 1
    else:
        self.matrix[nc, gc] += 1

if n:
    for i, dc in enumerate(pred_cls):
        if not (m_pred == i).any() and 0 <= dc < nc:
            self.matrix[dc, nc] += 1

def plot(self, names, save_dir, split="", normalize=True):
    import os
    import numpy as np
    nc = self.nc
    labels = [str(names.get(i, i)) for i in range(nc)] + ["background"]
    written = []
    os.makedirs(save_dir, exist_ok=True)
    suffix = f"_{split}" if split else ""

    try:
        csv_path = os.path.join(save_dir,
f"confusion_matrix{suffix}_hailo.csv")
        with open(csv_path, "w", encoding="utf-8") as f:
            f.write("predicted\\true," + ",".join(labels) + "\n")
            for i, rl in enumerate(labels):
                f.write(rl + "," + ",".join(str(int(v)) for v in self.matrix[i]) + "\n")
        written.append(csv_path)
    
```

```

except Exception as e:
    print(f"[WARN] Could not write confusion-matrix CSV: {e}")

try:
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
except Exception as e:
    print(f"[WARN] matplotlib unavailable; skipped the plot (CSV still "
          f"written). pip install matplotlib to get the PNG. ({e})")
    return written

array = self.matrix.astype(np.float64)
if normalize:
    col = array.sum(axis=0, keepdims=True)
    array = array / (col + 1e-9)
array[array < 0.005] = np.nan

n = nc + 1
vmax = 1.0 if normalize else float(np.nan_to_num(np.nanmax(array)) or
1.0)
cmap = plt.get_cmap("Blues").copy()
cmap.set_bad(color="white")

side = max(6.0, 0.55 * n + 3.0)
fig, ax = plt.subplots(figsize=(side, side * 0.85), dpi=200)
im = ax.imshow(array, cmap=cmap, vmin=0.0, vmax=vmax,
aspect="auto")
fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)

ax.set_xticks(range(n)); ax.set_yticks(range(n))
ax.set_xticklabels(labels, rotation=90, ha="center", fontsize=8)
ax.set_yticklabels(labels, fontsize=8)
ax.set_xlabel("True"); ax.set_ylabel("Predicted")
ax.set_title("Hailo Confusion Matrix" +
              (" (normalised)" if normalize else " (counts)"))

fmt = "{:.2f}" if normalize else "{:.0f}"
cut = 0.5 * vmax
for i in range(n):
    for j in range(n):
        v = array[i, j]
        if not np.isnan(v):
            ax.text(j, i, fmt.format(v), ha="center", va="center",
                    fontsize=7, color="white" if v > cut else "black")

fig.tight_layout()
try:
    png = os.path.join(
        save_dir,

```



```

        f"confusion_matrix_normalized{suffix}_hailo.png" if normalize
        else f"confusion_matrix{suffix}_hailo.png")
    fig.savefig(png)
    written.append(png)
except Exception as e:
    print(f"[WARN] Could not save confusion-matrix PNG: {e}")
finally:
    plt.close(fig)
return written

def _describe(x, depth=0, max_depth=4):
    import numpy as np
    pad = " " * depth
    if isinstance(x, dict):
        return "\n".join(f"{pad}dict[{k!r}]:\n{_describe(v, depth + 1,
max_depth)}"
                        for k, v in x.items())
    if isinstance(x, np.ndarray):
        head = f"{pad}ndarray shape={x.shape} dtype={x.dtype}"
        if x.dtype == object and x.size and depth < max_depth:
            return head + "\n" + _describe(x.reshape(-1)[0], depth + 1, max_depth)
        return head
    if isinstance(x, (list, tuple)):
        head = f"{pad}{type(x).__name__} len={len(x)}"
        if x and depth < max_depth:
            return head + "\n" + _describe(x[0], depth + 1, max_depth)
        return head
    return f"{pad}{type(x).__name__}={x!r}"

def _det_rows(elem):
    import numpy as np
    a = np.asarray(elem, dtype=np.float32)
    if a.size == 0:
        return np.zeros((0, 5), dtype=np.float32)
    return a.reshape(-1, a.shape[-1])

def _find_per_class(x, num_classes, depth=0):
    import numpy as np
    if isinstance(x, np.ndarray) and x.dtype == object:
        x = list(x)
    elif isinstance(x, np.ndarray):
        return None
    if isinstance(x, (list, tuple)):
        x = list(x)
        if len(x) == num_classes:
            return x
        if len(x) == 1 and depth < 6:

```

```

        return _find_per_class(x[0], num_classes, depth + 1)
    if x and depth < 6:
        try:
            widths = {(_det_rows(e).shape[1]) for e in x}
            if widths and widths.issubset({5, 6}):
                return x
        except Exception:
            pass
    return None

def parse_hailo_output(raw, num_classes, debug=False):
    import numpy as np
    if isinstance(raw, dict):
        if not raw:
            return []
        raw = next(iter(raw.values()))

    if debug:
        print("[DEBUG] Hailo output structure:")
        print(_describe(raw))

    dets = []

    per_class = _find_per_class(raw, num_classes)
    if per_class is not None:
        for cls_id, elem in enumerate(per_class):
            arr = _det_rows(elem)
            for row in arr:
                ymin, xmin, ymax, xmax, score = row[:5]
                dets.append((cls_id, float(score), float(ymin), float(xmin),
                             float(ymax), float(xmax)))
        return dets

    arr = np.asarray(raw)
    if arr.dtype != object:
        a = arr[0] if (arr.ndim and arr.shape[0] == 1) else arr

        if a.ndim == 3 and a.shape[-1] == 5:
            for cls_id in range(a.shape[0]):
                for row in a[cls_id]:
                    ymin, xmin, ymax, xmax, score = row[:5]
                    if score <= 0:
                        continue
                    dets.append((cls_id, float(score), float(ymin), float(xmin),
                                 float(ymax), float(xmax)))
            return dets

        if a.ndim == 2 and a.shape[-1] >= 6:
            for row in a:

```

```

        ymin, xmin, ymax, xmax, score, cls_id = row[:6]
        dets.append((int(cls_id), float(score), float(ymin), float(xmin),
                    float(ymax), float(xmax)))
    return dets

    raise RuntimeError(
        "Could not interpret the Hailo output. Re-run with debug=true and send
me "
        "the printed structure. The .hef may have no on-chip NMS (raw feature
maps "
        "need a YOLO decode step) or an unfamiliar output layout."
    )

def run_hailo(hef_path, images, names, conf, iou, to_rgb, debug, max_images,
             save_dir=None, split=""):
    import numpy as np
    import cv2
    try:
        from hailo_platform import (HEF, VDevice, HailoStreamInterface,
InferVStreams,
                                ConfigureParams, InputVStreamParams,
                                OutputVStreamParams, FormatType)
    except Exception as e:
        raise RuntimeError(
            "The Hailo Python package (hailo_platform) is not importable. On a Pi
5 "
            "with the AI Kit this comes from the 'hailo-all' / HailoRT install. "
            f"Underlying error: {e}"
        )

    if max_images and max_images > 0:
        images = images[:max_images]
    num_classes = (max(names.keys()) + 1) if names else 80

    iou_thr = np.linspace(0.5, 0.95, 10)
    all_correct, all_conf, all_pcls, all_tcls = [], [], [], []
    cm = ConfusionMatrix(num_classes, iou_thres=iou)
    t_pre = t_inf = t_post = 0.0
    n = 0

    hef = HEF(hef_path)
    in_info = hef.get_input_vstream_infos()[0]
    net_h, net_w, _ = in_info.shape

    with VDevice() as target:
        cfg = ConfigureParams.create_from_hef(hef,
interface=HailoStreamInterface.PCIe)
        network_group = target.configure(hef, cfg)[0]
        ng_params = network_group.create_params()

```

```

    in_params = InputVStreamParams.make(network_group,
format_type=FormatType.UINT8)
    out_params = OutputVStreamParams.make(network_group,
format_type=FormatType.FLOAT32)

    try:
        act_ctx = network_group.activate(ng_params)
    except Exception:
        act_ctx = nullcontext()

    with act_ctx:
        with InferVStreams(network_group, in_params, out_params) as
pipeline:
        for idx, ipath in enumerate(images):
            img = cv2.imread(ipath)
            if img is None:
                print(f"[WARN] Could not read image: {ipath}")
                continue
            h0, w0 = img.shape[:2]

            t0 = time.perf_counter()
            inp, meta = preprocess(img, net_w, net_h, to_rgb)
            batch = inp[None, ...]
            t1 = time.perf_counter()

            raw = pipeline.infer({in_info.name: batch})
            t2 = time.perf_counter()

            dets = parse_hailo_output(raw, num_classes, debug=(debug and
idx == 0))
            boxes_n, scores, classes = [], [], []
            for cls_id, score, ymin, xmin, ymax, xmax in dets:
                if score < conf:
                    continue
                boxes_n.append([xmin, ymin, xmax, ymax])
                scores.append(score)
                classes.append(cls_id)

            if boxes_n:
                boxes_n = np.asarray(boxes_n, dtype=np.float32)
                pred_boxes = boxes_to_original(boxes_n, meta)
                pred_cls = np.asarray(classes, dtype=np.int64)
                pred_conf = np.asarray(scores, dtype=np.float32)
            else:
                pred_boxes = np.zeros((0, 4), dtype=np.float32)
                pred_cls = np.zeros((0,), dtype=np.int64)
                pred_conf = np.zeros((0,), dtype=np.float32)

            gt_boxes, gt_cls = load_gt(ipath, w0, h0)

```

```

        correct = match_one_image(pred_boxes, pred_cls, gt_boxes,
gt_cls, iou_thr)
        cm.process_batch(pred_boxes, pred_cls, gt_boxes, gt_cls)
        t3 = time.perf_counter()

        all_correct.append(correct)
        all_conf.append(pred_conf)
        all_pcls.append(pred_cls)
        all_tcls.append(gt_cls)

        t_pre += (t1 - t0)
        t_inf += (t2 - t1)
        t_post += (t3 - t2)
        n += 1
        if n % 50 == 0:
            print(f" ...processed {n}/{len(images)} images")

    if n == 0:
        raise RuntimeError("No images were processed - check the split path in
dataset.yaml.")

    correct = np.concatenate(all_correct) if all_correct else np.zeros((0, 10),
dtype=bool)
    conf_arr = np.concatenate(all_conf) if all_conf else np.zeros((0,))
    pcls = np.concatenate(all_pcls) if all_pcls else np.zeros((0,), dtype=np.int64)
    tcls = np.concatenate(all_tcls) if all_tcls else np.zeros((0,), dtype=np.int64)

    result = compute_metrics(correct, conf_arr, pcls, tcls, names, iou_thr)

    if save_dir:
        cm_files = cm.plot(names, save_dir, split=split, normalize=True)
        if cm_files:
            print("\nConfusion matrix written:")
            for p in cm_files:
                print(f" {p}")

    speed = {
        "preprocess": 1000.0 * t_pre / n,
        "inference": 1000.0 * t_inf / n,
        "postprocess": 1000.0 * t_post / n,
    }
    return result, speed, n

def run_pt(model_path, data_yaml, split, imgsz, conf, iou, device, project,
run_name):
    from ultralytics import YOLO
    model = YOLO(model_path)
    metrics = model.val(
        data=data_yaml, split=split, imgsz=imgsz, conf=conf, iou=iou,

```

```

        device=device, project=project, name=run_name,
        exist_ok=True, plots=True, verbose=True,
    )
    box = metrics.box
    names = metrics.names
    idxs = list(box.ap_class_index) if len(box.ap_class_index) else []
    per_class = []
    for i, ci in enumerate(idxs):
        p, r, ap50, ap = box.class_result(i)
        per_class.append({"name": str(names.get(ci, ci)),
                        "p": float(p), "r": float(r),
                        "ap50": float(ap50), "ap": float(ap)})
    result = {
        "precision": float(box.mp), "recall": float(box.mr),
        "map50": float(box.map50), "map75": float(box.map75),
        "map": float(box.map), "per_class": per_class,
    }
    speed = getattr(metrics, "speed", {}) or {}
    return result, speed, str(metrics.save_dir)

def fmt_fps(speed):
    inf = float(speed.get("inference", 0) or 0)
    e2e = sum(float(speed.get(k, 0) or 0) for k in ("preprocess", "inference",
"postprocess"))
    inf_fps = (1000.0 / inf) if inf > 0 else None
    e2e_fps = (1000.0 / e2e) if e2e > 0 else None
    return inf, inf_fps, e2e, e2e_fps

def _fmt_gb(num_bytes):
    return f"{num_bytes / (1024 ** 3):.2f} GB"

def write_summary(result, speed, model_name, weights, split, backend,
                imgsz, conf, iou, device, cpu_peak, save_dir, n_images,
                ram_peak_used=None, ram_total=None):
    lines = []
    lines.append(f"Model:  {model_name}")
    lines.append(f"Backend: {backend}")
    lines.append(f"Weights: {weights}")
    lines.append(f"Split:  {split} (images: {n_images})")
    lines.append(f"Config: imgsz={imgsz} conf={conf} iou={iou}
device={device}")
    lines.append(f"Saved:  {save_dir}")
    lines.append("")
    lines.append("Overall")
    lines.append(f" Precision (mean): {result['precision']:.4f}")
    lines.append(f" Recall (mean):   {result['recall']:.4f}")
    lines.append(f" mAP@50:         {result['map50']:.4f}")

```

```

lines.append(f" mAP@75:      {result['map75']:.4f}")
lines.append(f" mAP@50-95:    {result['map']:.4f}")
lines.append("")
inf, inf_fps, e2e, e2e_fps = fmt_fps(speed)
lines.append("Performance")
if inf_fps:
    lines.append(f" Inference only:  {inf_fps:7.2f} FPS ({inf:.2f} ms/img)")
if e2e_fps:
    lines.append(f" End-to-end:      {e2e_fps:7.2f} FPS ({e2e:.2f} ms/img
incl. pre/post)")
cpu_s = f"{cpu_peak:.0f}%" if cpu_peak is not None else "n/a"
lines.append(f" CPU peak:      {cpu_s} (system-wide; 100% = all 4 cores
maxed)")
if ram_peak_used is not None and ram_total:
    ram_pct = ram_peak_used / ram_total * 100.0
    lines.append(f" RAM peak:      {_fmt_gb(ram_peak_used)} /
{_fmt_gb(ram_total)} "
f"({ram_pct:.1f}% system-wide)")
else:
    lines.append(" RAM peak:      n/a (install psutil to enable RAM
monitoring)")
lines.append("")

header = f"{'Class':<20}{'P':>9}{'R':>9}{'mAP50':>10}{'mAP50-95':>11}"
lines.append("Per class")
lines.append(header)
lines.append("-" * len(header))
for pc in result["per_class"]:
    lines.append(f"{'pc['name']':<20}{'pc['p']':>9.4f}{'pc['r']':>9.4f}"
f"{'pc['ap50']':>10.4f}{'pc['ap']':>11.4f}")

text = "\n".join(lines)

saved = []
os.makedirs(save_dir, exist_ok=True)
try:
    txt_path = os.path.join(save_dir,
f"accuracy_summary_{split}_{backend_tag(backend)}.txt")
    with open(txt_path, "w", encoding="utf-8") as f:
        f.write(text + "\n")
    saved.append(txt_path)
except Exception as e:
    text += f"\n\n(Could not save summary .txt: {e})"
try:
    csv_path = os.path.join(save_dir,
f"per_class_metrics_{split}_{backend_tag(backend)}.csv")
    with open(csv_path, "w", encoding="utf-8") as f:
        f.write("class,precision,recall,mAP50,mAP50-95\n")
        for pc in result["per_class"]:
            f.write(f"{'pc['name']},{pc['p']:.4f},{pc['r']:.4f},"

```

```

        f"{pc['ap50']:.4f},{pc['ap']:.4f}\n")
    saved.append(csv_path)
except Exception:
    pass

if saved:
    text += "\n\nFiles written by this script:\n" + "\n".join(f" {s}" for s in
saved)
    return text

def backend_tag(backend):
    return "hailo" if "Hailo" in backend else "cpu"

def main():
    params = {
        "model": "YOLOv10n",
        "data": "../Dataset/dataset.yaml",
        "split": "test",
        "imgsz": 640,
        "conf": 0.001,
        "iou": 0.7,
        "device": "cpu",
        "hailo": False,
        "rgb": True,
        "debug": False,
        "max_images": 0,
    }

    def parse_bool(v):
        return str(v).strip().lower() in ("1", "true", "yes", "y", "on")

    for arg in sys.argv[1:]:
        if "=" not in arg:
            continue
        key, value = arg.split("=", 1)
        key = key.strip()
        if key == "testfolder":
            key = "data"
        if key not in params:
            params[key] = value
            continue
        default = params[key]
        if isinstance(default, bool):
            params[key] = parse_bool(value)
        elif isinstance(default, int):
            params[key] = int(value)
        elif isinstance(default, float):
            params[key] = float(value)

```



```

else:
    params[key] = value

model_name = params["model"]
data_yaml = anchor(params["data"])
split = params["split"]
imgsz = params["imgsz"]
conf = params["conf"]
iou = params["iou"]
device = str(params["device"])
use_hailo = bool(params["hailo"])
to_rgb = bool(params["rgb"])
debug = bool(params["debug"])
max_images = int(params["max_images"])

if not os.path.exists(data_yaml):
    print(f"[ERROR] Could not find dataset config: {data_yaml}")
    sys.exit(1)
split = resolve_split(data_yaml, split)

project = anchor(OUTPUT_ROOT)
run_name = f"{model_name}_Hailo" if use_hailo else
f"{model_name}_CPU"
save_dir = os.path.join(project, run_name)

monitor = CPUPeakMonitor().start()
t_start = time.perf_counter()

try:
    if use_hailo:
        hef_path = resolve_hef(model_name)
        if not hef_path or not os.path.isfile(hef_path):
            print(f"[ERROR] Could not find a .hef for '{model_name}' in: "
                  f"{anchor(MODEL_DIRS.get(model_name, 'unknown'))}")
            monitor.stop()
            sys.exit(1)

        backend = "Hailo (.hef)"
        print(f"Starting Hailo eval for {model_name}...")
        print(f"Weights: {hef_path}")
        print(f>Data: {data_yaml} (split: {split})")
        print(f"Resize: stretch rgb={to_rgb} conf>={conf}")
        print("Loading .hef and running inference... please wait.\n")

        images, names = gather_split_images(data_yaml, split)
        if not images:
            print("[ERROR] No images found for that split.")
            monitor.stop()
            sys.exit(1)

```

```

result, speed, n_images = run_hailo(
    hef_path, images, names, conf, iou, to_rgb, debug, max_images,
    save_dir=save_dir, split=split)
weights = hef_path

else:
    pt_path = resolve_pt(model_name)
    if not pt_path or not os.path.isfile(pt_path):
        print(f"[ERROR] Could not find a .pt for '{model_name}' in: "
              f"{anchor(MODEL_DIRS.get(model_name, 'unknown'))}")
        monitor.stop()
        sys.exit(1)

    if device.lower() != "cpu":
        try:
            import torch
            if not torch.cuda.is_available():
                print(f"[WARN] device='{device}' requested but no CUDA
GPU is "
                    f"available (expected on a Pi 5). Falling back to CPU.")
                device = "cpu"
        except ImportError:
            print(f"[ERROR] PyTorch is not installed. Install ultralytics (which
"
                  "pulls in a CPU torch build) for the hailo=false path.")
            monitor.stop()
            sys.exit(1)

    backend = "PyTorch (CPU)" if device.lower() == "cpu" else f"PyTorch
({device})"
    print(f"Starting CPU eval for {model_name}...")
    print(f"Weights: {pt_path}")
    print(f>Data: {data_yaml} (split: {split})")
    print(f"Config: imgsz={imgsz} conf={conf} iou={iou}
device={device}")
    print("Loading model and running validation... please wait.\n")

    result, speed, save_dir = run_pt(
        pt_path, data_yaml, split, imgsz, conf, iou, device, project,
run_name)
    weights = pt_path
    try:
        imgs, _ = gather_split_images(data_yaml, split)
        n_images = len(imgs)
    except Exception:
        n_images = -1

    cpu_peak = monitor.stop()
    ram_peak_used = monitor.ram_peak_used
    ram_total = monitor.ram_total

```

```

total_s = time.perf_counter() - t_start

text = write_summary(result, speed, model_name, weights, split, backend,
                    imgsz, conf, iou, device, cpu_peak, save_dir, n_images,
                    ram_peak_used=ram_peak_used, ram_total=ram_total)

print("\n" + "=" * 80)
print(text)
print(f"\nWall-clock total: {total_s:.1f}s")
print("=" * 80 + "\nDone.")

except Exception:
    monitor.stop()
    print("\n[ERROR] An exception occurred during execution:")
    traceback.print_exc()
    sys.exit(1)

if __name__ == "__main__":
    main()

```

LAMPIRAN 3 (Augmentation.ipynb)

```

import os
import shutil
import random

# --- Configuration ---
source_img_dir = "Annotated_Images/train/images"
source_lbl_dir = "Annotated_Images/train/labels"
base_out_dir = "YOLO_Dataset"

# Splitting ratios
train_ratio, val_ratio, test_ratio = 0.7, 0.2, 0.1

# Create new directory structure
dirs_to_make = [
    f"{base_out_dir}/images/train", f"{base_out_dir}/labels/train",
    f"{base_out_dir}/images/val", f"{base_out_dir}/labels/val",
    f"{base_out_dir}/images/test", f"{base_out_dir}/labels/test"
]
for d in dirs_to_make:
    os.makedirs(d, exist_ok=True)

# Get all images and labels
images = [f for f in os.listdir(source_img_dir) if f.endswith((''.jpg', '.jpeg',
'.png'))]
data_pairs = []

```

```

for img_name in images:
    lbl_name = os.path.splitext(img_name)[0] + ".txt"
    if os.path.exists(os.path.join(source_lbl_dir, lbl_name)):
        data_pairs.append((img_name, lbl_name))

print(f"Found {len(data_pairs)} matching image-label pairs.")

# Shuffle and split
random.seed(42)
random.shuffle(data_pairs)

total = len(data_pairs)
train_end = int(total * train_ratio)
val_end = train_end + int(total * val_ratio)

train_pairs = data_pairs[:train_end]
val_pairs = data_pairs[train_end:val_end]
test_pairs = data_pairs[val_end:]

def copy_files(pairs, split_name):
    for img_name, lbl_name in pairs:
        shutil.copy(os.path.join(source_img_dir, img_name),
os.path.join(base_out_dir, f"images/{split_name}", img_name))
        shutil.copy(os.path.join(source_lbl_dir, lbl_name),
os.path.join(base_out_dir, f"labels/{split_name}", lbl_name))
    print(f"Copied {len(pairs)} files to {split_name}.")

copy_files(train_pairs, "train")
copy_files(val_pairs, "val")
copy_files(test_pairs, "test")

import os
os.environ['KMP_DUPLICATE_LIB_OK'] = 'TRUE'

import albumentations as A
import cv2
import os

# --- Configuration ---
train_img_dir = "YOLO_Dataset/images/train"
train_lbl_dir = "YOLO_Dataset/labels/train"
augment_multiplier = 3 # Increased from 2 to get more variety from the richer
pipeline

transform = A.Compose([
    # --- Geometry ---
    A.HorizontalFlip(p=0.5),

    A.ShiftScaleRotate(
        shift_limit=0.04,

```

```

        scale_limit=0.08,
        rotate_limit=12,
        border_mode=0,
        p=0.4
    ),

    A.RandomScale(scale_limit=0.12, p=0.3),

    A.Perspective(
        scale=(0.01, 0.04),
        p=0.2
    ),

    # --- Blur / Focus / Motion ---
    A.MotionBlur(blur_limit=(3, 7), p=0.25),
    A.GaussianBlur(blur_limit=(3, 5), p=0.15),

    # --- Lighting, safer than hue/brightness extreme ---
    A.CLAHE(
        clip_limit=2.0,
        tile_grid_size=(8, 8),
        p=0.15
    ),

    A.RandomShadow(
        shadow_roi=(0, 0, 1, 1),
        num_shadows_lower=1,
        num_shadows_upper=2,
        shadow_dimension=4,
        p=0.15
    ),

    # --- Camera / Video Quality ---
    A.ImageCompression(
        quality_lower=70,
        quality_upper=100,
        p=0.3
    ),

    A.GaussNoise(
        var_limit=(5.0, 20.0),
        p=0.2
    ),

    ], bbox_params=A.BboxParams(
        format='yolo',
        label_fields=['class_labels'],
        min_visibility=0.2
    ))

```

```

images = [f for f in os.listdir(train_img_dir)
           if f.endswith(('.jpg', '.jpeg', '.png')) and not f.startswith('aug_')]
total_augmented = 0

for img_name in images:
    img_path = os.path.join(train_img_dir, img_name)
    lbl_path = os.path.join(train_lbl_dir, os.path.splitext(img_name)[0] + ".txt")

    # Read image
    image = cv2.imread(img_path)
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # Read labels
    bboxes = []
    class_labels = []
    with open(lbl_path, 'r') as f:
        for line in f.readlines():
            parts = line.strip().split()
            if len(parts) == 5:
                class_labels.append(int(float(parts[0])))
                bboxes.append([float(parts[1]), float(parts[2]), float(parts[3]),
float(parts[4])])

    # Generate augmented copies
    for i in range(augment_multiplier):
        try:
            augmented = transform(image=image, bboxes=bboxes,
class_labels=class_labels)
            aug_img = augmented['image']
            aug_bboxes = augmented['bboxes']
            aug_labels = augmented['class_labels']

            # Save augmented image
            aug_img_name = f"aug_{i}_{img_name}"
            aug_img_path = os.path.join(train_img_dir, aug_img_name)
            cv2.imwrite(aug_img_path, cv2.cvtColor(aug_img,
cv2.COLOR_RGB2BGR))

            # Save augmented labels
            aug_lbl_name = os.path.splitext(aug_img_name)[0] + ".txt"
            aug_lbl_path = os.path.join(train_lbl_dir, aug_lbl_name)
            with open(aug_lbl_path, 'w') as f:
                for bbox, label in zip(aug_bboxes, aug_labels):
                    f.write(f"{label} {bbox[0]:.6f} {bbox[1]:.6f} {bbox[2]:.6f}
{bbox[3]:.6f}\n")

            total_augmented += 1
        except Exception as e:
            pass

```

```
print(f"Augmentation complete. Generated {total_augmented} new training images.")
```

LAMPIRAN 4 (Training.ipynb)

```
import yaml
from ultralytics import YOLO
import os
os.environ['KMP_DUPLICATE_LIB_OK'] = 'TRUE'
os.environ.pop("ALBUMENTATIONS_DISABLE", None)

# 1. Create the dataset.yaml file required by YOLO
yaml_content = {
    'path': os.path.abspath('./Images/YOLO_Dataset'),
    'train': 'images/train',
    'val': 'images/val',
    'test': 'images/test',
    'names': {
        0: '1-ball',
        1: '2-ball',
        2: '3-ball',
        3: '4-ball',
        4: '5-ball',
        5: '6-ball',
        6: '7-ball',
        7: '8-ball',
        8: '9-ball',
        9: 'cue',
        10: 'cue_stick',
        11: 'pocket',
        12: 'rack'
    }
}

with open('dataset.yaml', 'w') as f:
    yaml.dump(yaml_content, f, sort_keys=False)

print("dataset.yaml created successfully.")

# 2. Download YOLO11n and Train
model = YOLO('yolov10n.pt')

print("Starting training...")
results = model.train(
    data='dataset.yaml',
    epochs=100,
```

```

    imgsz=640,
    batch=8,
    patience=10,
    workers=4,
    close_mosaic=10,
    cos_lr=True,
    project=os.path.abspath('../Trained_Models'),
    name='YOLOv10n'
)

```

LAMPIRAN 5 (ONNX_Conversion.py)

```

from ultralytics import YOLO

model = YOLO(r"YOLOv10n-2\weights\best.pt")

path = model.export(
    format="onnx",
    dynamic=False,
    imgsz=640,
    project="ONNX_Models",
    opset=11,
    simplify=True,
    batch=1,
)

print(f"Model successfully saved to: {path}")

```

LAMPIRAN 6 (Hailo Compilation Command)

```

hailomz compile yolov8n \
  --ckpt yolov8n/best.onnx \
  --calib-path calibration_images \
  --hw-arch hailo8l \
  --classes 13 \
  --performance

hailomz compile yolov10n \
  --ckpt yolov10n/best.onnx \
  --calib-path calibration_images \
  --hw-arch hailo8l \
  --classes 13 \
  --performance

hailomz compile yolov11n \
  --ckpt yolo11n/best.onnx \
  --calib-path calibration_images \
  --hw-arch hailo8l \
  --classes 13 \

```


--performance